SUP'COM

المدرسة العليا للمواصلات بتونس

ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Cycle de formation des ingénieurs en Télécommunications

Option :

CySeD

Rapport de Projet de fin d'études

Thème :

« Formalisation de la Méthodologie ATHENA pour les Pentests IoT»

Réalisé par :

Yannick Kouame

Encadrants:

M. Ahmed Badreddine

M. Slim Rekhis

Travail proposé et réalisé en collaboration avec



Année universitaire : 2020-2021

RESUME ET MOTS CLÉS

Les objets connectés, avec leur croissance exponentielle, occupent de plus en plus d'importance et de place dans le quotidien des utilisateurs. Cependant ils suscitent une vive inquiétude quant au volet de la cybersécurité car réputés très vulnérables et couramment ciblés par les cyberattaques. Ceci impose une variété de mesures de sécurité dont les tests d'intrusion qui visent à évaluer/tester le niveau de sécurité des systèmes informatiques.

Toutefois il n'existe pas de méthodologie et de standards qui soient efficacement adaptés et qui abordent de façon complète les pentests (tests d'intrusion) IoT. Et c'est précisément ce problème que ce PFE se propose d'adresser, pour guider les pentests IoT de l'entreprise ATHENA.

Pour ce faire nous avons exploré divers standards et sources de référence en matière d'IoT pour proposer une méthodologie de pentest qui concilie complétude et non redondance. Notre méthodologie aborde le pentest IoT en 4 phases, ciblant respectivement 4 couches (réseau, applicative, firmware et physique selon les surfaces d'attaques des IoT). Chacune d'elles décrit des étapes corrélées permettant d'exploiter les canaux d'attaques spécifiques à la couche qu'elle aborde.

Mots-clés: IoT, tests d'intrusion, rétro-ingénierie de firmware

REMERCIEMENTS

Avant tout, je tiens à exprimer ma profonde gratitude à toutes ces personnes qui m'ont accompagné le long de la réalisation de ce projet de fin d'étude (PFE) et à la rédaction de ce mémoire.

Je voudrais tout d'abord mentionner mon encadrant académique: M. SLIM REKHIS, Enseignant-chercheur en télécommunication et cybersécurité à SUP'COM TUNIS que je ne saurais remercier assez pour le soutien sans faille.

Son assistance principalement dans les orientations techniques m'a dans une large mesure permit de mener à bien ce PFE, aussi bien dans le travail à réaliser que dans la rédaction du mémoire.

J'adresse également mes sincères remerciements à M. Ahmed Badreddine, Mon maître de stage avec qui j'ai travaillé sur le projet, pour son apport essentiellement technique.

Je remercie toute l'équipe pédagogique de SUP'COM ayant été responsable de ma formation, pour les compétences théoriques et pratiques acquises et que j'ai pu mettre en œuvre pour la réalisation de ce projet.

Sans oublier l'équipe Athena-expert qui m'a offert l'opportunité de travailler sur ce projet plus qu'enrichissant.

Enfin je remercie mes parents, pour leur soutien constant et leurs encouragements.

DÉDICACE

A Mon père et à ma mère, qui tout le long de mon cursus académique n'ont ménagé aucun effort pour ma réussite.

Les mots ne seront pas suffisants pour exprimer ma reconnaissance pour leur soutien, tant matériel qu'affectif, et sans lequel j'aurais manqué de relever avec brio bien de défis auxquels j'ai été confronté.

Enfin à mon amie Dominique, pour qui je porte une affection presque fraternelle.

SOMMAIRE

RESUME ET MOTS CLÉS 3

REMERCIEMENTS 4

DÉDICACE 5

SOMMAIRE 6

LISTE DES ABREVIATIONS 11

LISTE DES FIGURES 14

LISTE DES TABLEAUX: 17

INTRODUCTION GENERALE 18

CHAPITRE I: Généralités 20

I. Présentation de la structure d'accueil 20
II. Collecte d'informations et mots-clés 21
1. L'IoT 21
1.1. Généralités sur l'IoT 21
1.1.1. Définition de l'IoT 21
1.1.2. Architecture et fonctionnement [20] 21
1.2. Applications de l'IoT 23
1.2.1. Les maisons intelligentes 23
1.2.2. L'e-Santé 23
2. Le Test d'intrusion 24
2.1. Généralités sur les tests d'intrusion 24
2.1.1. Définition 24
2.1.2. Les types de tests de d'intrusion 24
2.2. Quelques Méthodologies et standards de tests d'intrusion [1] 25
2.2.1. ISSAF (Information Systems Security Assessment Framework) [2] 25
2.2.2. PTES (Penetration Testing Execution Standards) [3] 27
3. Le Test d'intrusion en matière d'IoT 29
3.1. Statistiques et chiffres clés en matière de sécurité IoT 29
3.1.1. Les appareils IoT les plus exposés [4] 29
3.1.2. Trafic non sécurisé des appareils IoT [5] 29
3.1.3. Les menaces et vulnérabilités les plus rencontrées dans la sphère IoT (OWASP's Top 10 IoT Vulnerabilities) [6] 29
3.2. Les surfaces d'attaques des écosystèmes IoT [7] 31

3.2.1. Les attaques sur la surface d'attaque physique 32
3.2.2. Attaques sur les systèmes IoT ciblant la mémoire 34
3.2.3. Attaques sur le firmware 35
3.2.4. Attaque ciblant le mécanisme de mise à jour 35
3.2.5. Attaques et vecteur d'attaques de la couche applicative mobile 36
3.2.6. La couche applicative web/interface administrateur 36
3.2.7. La couche réseau/communication [15] [16] 36
3.3. Méthodologies / standards / frameworks en pentesting IoT 37
3.3.1. Le ISVS (IoT Security Verification Standard) [17] 37
3.3.2. FSTM (Firmware Security Testing Methodology) [18] 40
3.3.2. La méthodologie de pentest IoT proposée par Rapid7 [19] 42
III. Analyse et critique de l'existant 43
1. Existant d'ATHENA 43
2. Existant au-delà d'ATHENA 43
3. Les couches et les vecteurs/surfaces d'attaques 44
3.1. Exploration des surfaces d'attaque IoT selon OWASP 44
3.2. Exploration du standard ISVS (IoT Security Verification Standard) 46
3.3. Exploration du standard "Code of Practice for Consumer IoT Security" 46
3.4. Conclusion Partielle 47
IV. Cahier de charges et Objectifs 48

CHAPITRE II: Elaboration de la méthodologie 49
I. ARCHITECTURE DE LA CIBLE D'ÉVALUATION 49
II. MODÈLE DE L'ATTAQUANT 50
1. Les connaissances et ressources préalables du testeur d'intrusion 51
2. Positions de l'attaquant vis-à-vis des équipements. 51
2.1 Accès Physique 51
2.2 Accès depuis internet 51
2.3 Dans la zone de couverture du réseau 52
III. LA MÉTHODOLOGIE 53
A: Test de la couche réseau 55
I. Collecte d'informations sur le réseau cible et Scanning 56
1. Découverte et mappage 57
II. Identification des vulnérabilités et modélisation des attaques 59
1. Scan de vulnérabilités 59
2. Choix des attaques à mener / tests à effectuer 60
III. Exploitation 61
1. Exploitation WI-FI 62
1.1 Sniffing du trafic WI-FI 62

1.2 craquage de Mot de passe via attaque par dictionnaire 63
1.3. Attaques Conjointes du DOS sur DHCP, du Serveur Rogue DHCP et du MITM 63
2. Exploitation Zigbee 64
2.1 Sniffing de trafic Zigbee 64
2.2 Attaque Replay 64
3. Exploitation BlueTooth/BLE 65
3.1 Sniffing 65
3.2 Attaque Replay 65
4. Attaques sur VOIP 66
4.1. Reconnaissance 66
4.2. Interception des flux VOIP/ relecture des conversations 66
4.3. Craquage de Mot de passe SIP 67
B. test de la Couche d'application 68
C: Test de la couche Firmware 69
I. Collecte d'informations 70
II. Obtention/Extraction du firmware 71
III. Analyse sommaire du firmware 72
IV. Extraction du système de fichier 73
V. Analyse du contenu des systèmes de fichiers et rétro-ingénierie binaire 74
VI. Implantation de backdoors /modification du firmware 75
VII. Emulation & Analyse dynamique (facultatif) 76
VIII. Exploitation et/ou flash du firmware altéré 77
D. Test de la couche physique 78
I. Collecte d'Informations 79
1. Inspection externe 80
2. Inspection interne 80
3. Recherches à travers internet 81
4. Révision de la fiche technique 81
II. Choix des Attaques 82
III. Exploitation 83
1. Exploitation UART 83
1.1 obtention de shell root sur l'équipement à travers une connexion UART 83
2. Exploitation SPI 84
2.1 Lecture et écriture à travers la mémoire EEPROM SPI 84
3. Exploitation JTAG 85
3.1 Écrire un nouveau firmware 85
3.2 Lire les données sensibles du firmware 86
3.3 Extraire le firmware 87
4. Exploitation I²C 88

4.1 Lecture de contenu de la mémoire EEPROM I²C 88
5. Autres attaques (construites à partir des menaces modélisées, ou spécifiques à des équipements particuliers) 89
5.1 Le déblocage des modems 3G/4G 89
5.2 Le déblocage des Routeurs 3G/4G 90

CHAPITRE III: Validation de la méthodologie (tests) 91
I. Architecture / environnement de tests 91
1. La carte Raspberry pi 92
2. La DEL (Diode ÉlectroLuminescente) 92
3. Le smartphone 93
4. Montage de l'environnement 93
5. Hypothèses de départ 96
6. Outils utilisés pour les tests 97
6.1 La suite aircrack-ng 97
6.2 Nmap 97
6.3 Binwalk 97
6.4 Hopper 97
II. Déroulement des tests 98
La couche réseau 98
1. Collecte d'informations sur le réseau cible et Scanning 99
Découverte et mappage: identifions les réseaux à proximité. 99
Scan de ports (services actifs) 100
2. Identification des vulnérabilités et modélisation des attaques 101
Scan de vulnérabilités 101
Choix des attaques à mener / tests à effectuer 101
3. Exploitation 102
Attaque par dictionnaire 102
4. Intrusion dans le réseau et retour à la phase de scan de ports 104
Scan de Ports. 105
Scan de vulnérabilités 106
Choix des attaques 106
Pour aller plus loin 106
5. Conclusion du test d'intrusion au niveau de la couche réseau. 109
La couche firmware 110
1. La collecte d'information 111
L'architecture de processeur prise en charge: 111
Aux vulnérabilités Connues: 112
Bilan de la phase de collecte d'informations 112

2. Obtention du firmware 113
3. Analyse sommaire du firmware 114
4. Extraction du système de fichier 115
5. Analyse du contenu des systèmes de fichiers et rétro-ingénierie binaire 117
Commençons par effectuer une analyse statique Manuelle 117
Menons une analyse statique automatisée 119
Menons la phase de rétro-ingénierie binaire 120
Bilan de la phase d'analyse du système de fichiers 124
6. Conclusion de la phase de test du firmware 125

CONCLUSION GENERALE 126

BIBLIOGRAPHIE & WEBOGRAPHIE 127

ANNEXES 129

LISTE DES ABREVIATIONS

IoT: Internet Of Things

OWASP: Open Web Application Security Project

WSTG: Web Security Testing Guide

ISVS: IoT Security Verification Standard

FSTG: Firmware Security Testing Guide

MSTG: Mobile Security Testing Guide

QSA: Qualified Security Assessor

PCI DSS: Payment Card Industry Data Security Standard

PCI SSC: Payment Card Industry Security Standards Council

SOC: Security Operation Center

PECB: Professional Evaluation and Certification Board

BLE: Bluetooth Low Energy

6LOWPAN: IPv6 Low power Wireless Personal Area Network

ADSL: Asymmetric Digital Subscriber Line

PTES: Penetration Testing Execution Standard

ISSAF: Information System Security Assessment Framework

OSINT: Open Source Intelligence

IDS: Intrusion Detection System

CVE: Common Vulnerabilities and Exposures

DDOS: Distributed Denial of Service

SPA: Single Power Analysis

JTAG: Joint Test Action Group

FI: Fault Injection

RSA: Rivest, Shamir, et Adleman

DES: Data Encryption Standard

BED: Bruteforce Exploit Detector

SSL/TLS: Secure Sockets Layer / Transport Layer Security

HTTP(S): Hypertext Transfer Protocol (Secure)

LDAP: Lightweight Directory Access Protocol

OS: Operating System

MITM: Man In The Middle

UART: Universal Asynchronous Receiver-transmitter

API: Application Programming Interface

SSH: Secure Shell

FACT: Firmware Analysis and Comparison Tool

FMK: Firmware-Mod-Kit

OWASP ZAP: OWASP Zed Attack Proxy

ROP: Return-oriented programming

LAN: Local Area Network

WAN: Wide Area Network

WEP: Wired Equivalent Privacy

WPA: Wi-Fi Protected Access

TCP/IP: Transmission Control Protocol/Internet Protocol

STRIDE: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of service, and Elevation of privilege

WI-FI: Wireless Fidelity

DHCP: Dynamic Host Configuration Protocol

DOS: Denial of Service

USB: Universal Serial Bus

CRC: Cyclic Redundancy Check

SIP: Session Initiation Protocol

PBX: Private Branch eXchange

VOIP: Voice over Internet Protocol

ARM: Advanced RISC Machines

MIPS: Microprocessor without Interlocked Pipelined Stages

FCC: Federal Communications Commission

SPI: Serial Peripheral Interface

SOC: Security Operation Center

I2C: Inter-Integrated Circuit

EEPROM: Electrically Erasable Programmable Read-Only Memory

SIM: Subscriber Identification Module

IMEI: International Mobile Equipment Identity

DEL: Diode ÉlectroLuminescente

NAT: Network Address Translation

LISTE DES FIGURES

Figure 1: *Architecture d'un environnement IoT*----------21

Figure 2: *Maison intelligente*----------23

Figure 3: *Collecte et analyse de données médicales à distances* ----------23

Figure 4: *Surfaces d'attaque IoT*----------31

Figure 5: *Types d'attaques physiques*----------32

Figure 6: *équipement requis pour mener une attaque SPA*----------32

Figure 7: *PulseView* ----------33

Figure 8: *Microprobing*----------34

Figure 9: *l'IoT Security Verification Standard* ----------37

Figure 10: *logo du FSTM*---------- 40

Figure 11: *les exigences du "Code of Practice for Consumer IoT Security"* ----------46

Figure 12: *Architecture de la cible.* ----------49

Figure 13: *Modèle de l'attaquant.* ----------50

Figure 14: *La méthodologie schématisée* ----------53

Figure 15: *La méthodologie pour la couche reseau*----------55

Figure 16: *La méthodologie pour la couche firmware*----------69

Figure 17: *La méthodologie pour la couche physique*----------78

Figure 18: *L'architecture de l'environnement de tests*----------91

Figure 19: *La carte raspberry pi*----------92

Figure 20: *Smartphone manipulant une DEL connectée.*----------93

Figure 21: *Raspberry Pi OS installé*----------93

Figure : *DEL*----------94

Figure 22: *Installation d'apache et de ses dépendances pour mettre en place notre serveur web*----93

Figure 23: *Installation de la librairie wiringpi*----------94

Figure 24: *Les broches de la Raspberry Pi*----------94

Figure 25: *Schéma du montage*----------95

Figure 26: *Smartphone manipulant la DEL connectée.*----------96

Figure 27: *Logo Aircrack-ng*----------97

Figure 28: *Logo Nmap* ----------97

Figure 29: *Binwalk* ----------97

Figure 30: *hopper* ----------98

Figure 31: *La méthodologie de pentest réseau à titre de rappel.*----------98

Figure 32: *On interrompt les processus en cours et on passe en mode moniteur.*----------99

Figure 33 : *découverte des points d'accès WI-FI aux alentours.* ----------99

Figure 34: *découvertes des terminaux connectés à chacun des ces points d'accès* ----------99

Figure 35: Aperçu de la *couche réseau - sous catégorie WI-FI* de la check-list. choix de l'attaque *craquage de mot de passe via attaque par dictionnaire.* ----------101

Figure 36: *déconnection des terminaux du point d'accès.* ----------102

Figure 37: *Nos paquets de 4-way handshakes et la clé que nous souhaitons obtenir (cryptée).*----102

Figure 38: *clé craquée.*----------103

Figure 39: *un IoT découvert (en rouge).*----------104

Figure 40: *les services ssh et http actif sur le IoT cible* ----------105

Figure 41: *Scan du smartphone Huawei.* ----------105

Figure 42: *Scan de vulnérabilités sur la raspberry pi.* ----------106

Figure 43: *Connexion réussie via ssh avec les identifiants par défaut* ----------107

Figure 44: *présence d'une page web.* ----------*107*

Figure 45: *Le code confirme nos soupçons.* ----------*107*

Figure 46: *Insertion d'un code malicieux dans le serveur web.* ---------- *108*

Figure 47: *Génération d'une clé pré-partagée et transfert vers la machine de l'attaquant.*-------- *109*

Figure 48: *La méthodologie firmware à titre de rappel.* ----------*110*

Figure 49: *Architecture des modèles de la série DWR de D-LINK est donc de la famille MIPS*---*111*

Figure 50: *vulnérabilités connues des routeurs D-Link* ----------*112*

Figure 51: *Téléchargement du firmware*----------*113*

Figure 52: *Informations sommaires sur le fichier téléchargé.* ----------*114*

Figure 53: *contenu du fichier .zip originel après avoir été décompressé.* ----------*115*

Figure 54: *Extraction du système de fichier dans un nouveau répertoire nommé root-yaffs.* ------*115*

Figure 55: *Aperçu du système de fichier extrait* ----------*116*

Figure 56: *liste des fichiers de configurations*----------*117*

Figure 57: Identifiants trouvés dans le fichier *./etc/inadyn-mt.conf* ----------*117*

Figure 58: Un compte root identifié---------- 118

Figure 59: *mot de passe obtenu. Le compte root a comme mot de passe 1234.* ----------*118*

Figure 60: *Fichiers sensibles découverts par l'outil Firmwalker.* ----------*119*

Figure 61: *Hopper depuis le menu.* ----------*120*

Figure 62: *Chargement du binaire appmgr* ----------*120*

Figure 63: *Hopper disassembler* nous fournit le code assembleur du binaire, ainsi que les fonctions qui le composent. ----------121

Figure 64: *le firmware semble fournir des services d'accès à distance (en l'occurrence telnet).*----122

Figure 65: *Pseudo-code de la fonction de laquelle provient l'instruction lançant telnetd*----------*123*

LISTE DES TABLEAUX:

Tableau 1: *Surfaces d'attaques des IoT selon OWASP* ----------45

INTRODUCTION GENERALE

Avec une estimation de 46 milliards d'équipements IoT actifs d'ici fin 2021, l'Internet Of Things occupe manifestement une place de plus en plus prépondérante dans l'industriel et dans le quotidien des utilisateurs. Cette croissance fulgurante de l'usage des IoT ne manque toutefois pas de susciter des craintes quant aux risques liés à la sécurité: une étude stipule que 83% des IoT dans le domaine médical fonctionnent avec des firmwares obsolètes et donc hautement vulnérables. Une autre avance que 98% de tous les trafics d'équipement IoT sont non cryptés. Ces chiffres plus qu'alarmant publiés lors d'une étude de Unit 42 poussent à reconsidérer l'importance à accorder à la cybersécurité dans le secteur des IoT.

Dès lors, il devient vital de s'assurer de la robustesse des équipements IoT en termes de cybersécurité. Pour ce faire, il est de plus en plus fréquent d'avoir recours à des tests d'intrusion pour évaluer à quel point l'environnement IoT est paré aux cyber-attaques. Des standards et guides sont mis à la disposition des spécialistes pour mener de façon efficace leur tests d'intrusion et tests de sécurité. Parmi eux nous avons OWASP Web security testing guide (WSTG) pour les tests d'intrusion orientés web, et aussi IoT Security Verification Standard (ISVS) Pour la vérification du niveau de sécurité des équipements IoT.

Dans ce même élan et dans un souci de formalisation du processus de ses tests d'intrusion, ATHENA à jugé pertinent l'**élaboration d'une méthodologie pour ses pentests IoT.**

Athena ne dispose pas de méthodologie formelle pour mener ses tests d'intrusion en milieu IoT. Par ailleurs, il n'existe pas de standard exclusivement dédié à des tests d'intrusion en la matière. Certes il en existe certains qui guident les tests ciblant des couches spécifiques adaptables au contexte IoT, comme OWASP FSTG (firmware security testing guide) pour tester la couche firmware. Toutefois aucun des standards existant n'adresse toutes les couches pour aborder de façon complète les tests d'intrusion IoT.

Comment concevoir une méthodologie pratique et efficace pour guider les testeurs d'intrusion en milieu IoT?

Sur quels standards s'appuyer pour l'élaboration d'une telle méthodologie?

L'élaboration de cette méthodologie s'est essentiellement basée sur des standards existants. Des standards en matière de sécurité, directement ou indirectement liés aux IoT. En l'occurrence Le FSTM (firmware Security testing guide), et le MSTG (mobile security testing guide). Nous avons

également consulté des livres et documentation de natures diverses, proposant des méthodes de test d'intrusion en matière d'IoT.

Ce rapport sera structuré en 3 chapitres dont le premier se chargera d'aborder des questions de généralités sur le projet, son contexte et ses mots clés. Dans les deux derniers il sera respectivement question du déroulement de la méthodologie, et de sa validation à travers des tests pratiques menés sur des architectures cibles.

CHAPITRE I: Généralités

Ce chapitre sera le lieu d'aborder les questions d'ordre général. Nous nous attellerons dans un premier point, à présenter brièvement la structure d'accueil. Ensuite, après la phase de collecte d'informations et d'analyse de l'existant, nous allons décrire le cahier des charges du projet.

I. Présentation de la structure d'accueil

Fondée en 2007, ATHENA est un cabinet de service spécialisé dans la sécurité de l'information et la gouvernance. ATHENA offre ses services aux entreprises en les accompagnant dans les aspects techniques, organisationnels et fonctionnels liés à leurs domaines de spécialisation. ATHENA étend ses services à l'international à travers 4 filiales implantées respectivement à:

- Paris (France)
- Bucarest (Roumanie)
- Tunis (Tunisie)
- Kinshasa (République Démocratique du Congo)

ATHENA TUNISIE c'est aussi une structure qualifiée QSA (Qualified Security Assessor) par le PCI SSC. Elle soutient ainsi ses clients dans leur démarche de conformité PCI DSS.

La mission d'ATHÉNA consiste à accompagner ses clients sur les différentes problématiques liées à la sécurité de leurs informations. La richesse de son capital humain lui permet d'opérer sur un large périmètre d'intervention qui inclut les aspects techniques, opérationnels, d'organisation, de gouvernance et de formation liées à la sécurité de l'information.

Par ailleurs, ATHENA offre des services allant de l'analyse forensique à la mise en place de SOC.

II. Collecte d'informations et mots-clés

Dans cette section du chapitre I, nous allons aborder les concepts d'IOT, de test d'intrusion, puis de test d'intrusion en matière d'IOT.

1. L'IoT

1.1. Généralités sur l'IoT

1.1.1. Définition de l'IoT

L'IoT, Internet Of Things, est un concept qui fait référence à l'interconnexion d'objets physiques (things) à travers internet. Ces objets sont divers et peuvent être des objets domestiques ordinaires ou des équipements industriels sophistiqués. Pour ce faire, il leur sont intégrés des capteurs et logiciels dans le but de les faire échanger des informations entre eux et à travers internet.

1.1.2. Architecture et fonctionnement [20]

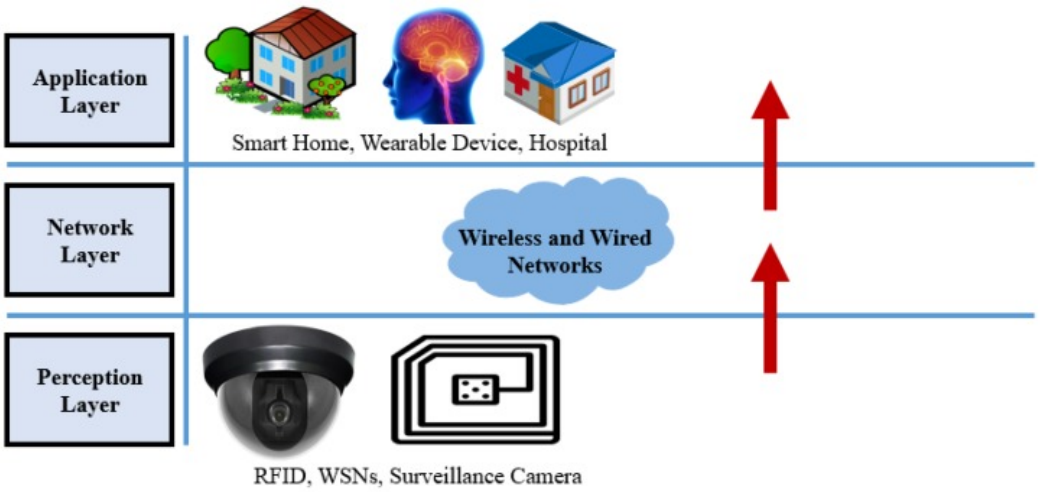


Figure 1*: Architecture d'un environnement IoT*

L'architecture d'un environnement IoT peut varier d'une solution à une autre, selon les objectifs visés. Toutefois, il est possible de substantiellement décrire cette architecture selon 3 couches que sont la couche physique, la couche réseau et la couche application.

- **La couche physique**

C'est la couche qui va abriter les composants tels que les capteurs et les actionneurs, pour collecter des données provenant de l'environnement ou réagir selon ces informations.

- **La couche réseau/communication**

Cette couche regroupe l'ensemble des moyens, composants, protocoles et équipements intervenant dans la transmission/réception des informations. Selon l'architecture et la solution déployée, cette couche peut disposer de deux autres fonctionnalités, l'agrégation des données et leur traitement. Cela s'avère nécessaire pour la transformation par exemple des données analogiques en provenance des capteurs en données numériques, leur filtrage, leur agrégation avant acheminement vers le cloud...

Comme protocoles, on peut distinguer ZIGBEE, BLE, ou 6LowPAN, pour la communication sans fil. Pour ce qui est des équipements, on peut avoir les box ADSL.

- **La couche application**

La couche application définit l'usage auquel sera employé la solution IoT (d'un point de vue de l'utilisateur final). C'est l'interface entre l'utilisateur et la solution IoT en elle-même. On peut à titre illustratif citer les maisons/villes intelligentes, les logiciels/applications par lesquelles l'utilisateur accède aux services IoT.

1.2. Applications de l'IoT

1.2.1. Les maisons intelligentes



Figure 2*: Maison intelligente*

L'IoT est la technologie sous-jacente du concept de maison intelligente. Il transforme le quotidien de l'utilisateur en lui permettant de manipuler les objets domestiques usuels de façon commode, à distance. Il lui épargne aussi des contraintes ergonomiques tels les déplacements et les vérifications à travers une automatisation intelligente des tâches.

1.2.2. L'e-Santé



Figure 3: *Collecte et analyse de données médicales à distance*

L'IoT se montre également fort utile dans le secteur médical. Il permet d'assister les patients à distance à travers des gadgets qui font parvenir leurs données médicales aux médecins (pression sanguine, taux de glycémie, ...).

2. Le Test d'intrusion

2.1. Généralités sur les tests d'intrusion

2.1.1. Définition

Les tests d'intrusion, aussi connus sous l'appellation pentests sont des cyberattaques simulées autorisées, à l'encontre d'un système dans le but d'en évaluer le niveau de sécurité et en identifier les vulnérabilités. Ces attaques sont effectuées par des hackers éthiques, ou par des spécialistes internes à l'entreprise cible (auditeurs internes...).

2.1.2. Les types de tests de d'intrusion

Selon les informations initiales dont dispose le hacker éthique, le test d'intrusion peut être:

- **Black box**

Aucune information ou très peu d'informations sont fournies au hacker. Son but est de simuler une attaque externe. L'inconvénient avec ce type de test d'intrusion est qu'il est relativement coûteux en temps et ressources financières. Par ailleurs, il manque parfois certaines surfaces vulnérables de la cible.

- **White box**

Toutes les informations nécessaires sur la cible sont fournies au testeur en avance. Il s'agit ici de simuler une attaque interne. Comparativement au test en black box, il est plus rapide et facile à effectuer et aussi moins coûteux en ressources.

- **Grey box**

Il s'agit en quelque sorte d'une combinaison des 2 types précités. Ce type de test consiste à fournir au test une quantité d'informations limitée pour effectuer le test.

2.2. Quelques Méthodologies et standards de tests d'intrusion [1]

Plusieurs standards/méthodes pour les tests d'intrusion sont en vigueur. Parmi ceux-ci, on pourrait citer OWASP WSTG, PTES, ISSAF, etc. nous allons juste en aborder quelques-uns (en l'occurrence ISSAF et PTES).

2.2.1. ISSAF (Information Systems Security Assessment Framework) [2]

ISSAF est un standard pour l'évaluation du niveau de sécurité d'une entreprise. Il se décline en 3 phases que sont: (I) planning et préparation , (II) évaluation du niveau de sécurité, et (III) Rapport et nettoyage des traces.

- **Planning et préparation (Planning and preparation)**

Cette phase consiste à établir un contrat formel entre le testeur et la cible. Il spécifie les points tels que la période du test, les actifs impliqués, les méthodologies utilisées, etc.

- **Évaluation du niveau de sécurité (Assessment)**

C'est la phase où se déroule le test d'intrusion proprement dit. Elle se répartit selon les étapes suivantes:

Collecte d'informations (Information gathering): Cette phase consiste essentiellement à recueillir des informations sur la cible. Il s'agit substantiellement des informations sur les systèmes, les technologies employées dans le réseau.

Cartographie du réseau (network mapping): Le but ici est d'avoir le plus d'information possible sur la topologie du réseau, ces composants, ses protocoles, etc. pour cela le testeur aura recours à (parmi tant d'autres) Diverses techniques telles que le scanning (scan de ports, d'hôtes, ...).

Identification des vulnérabilités: Dans cette phase, il s'agit d'utiliser des scanners tels que Nessus ou Nikto pour relever les vulnérabilités du système. Puis de les classifier et de les prioriser.

Pénétration du système et augmentation de privilège: Le testeur apporte la preuve de l'exploitabilité des vulnérabilités qu'il a détectées. Il s'introduit dans le système et exploite les vulnérabilités. Une fois introduit dans le système, il augmente ses privilèges.

Énumération avancée: En usant surtout des privilèges élevés acquis, l'attaquant recueille davantage de données d'identification et d'authentification. Le testeur peut même mieux cartographier le réseau.

Compromission des utilisateurs/sites distants: Un seul utilisateur distant compromis suffit pour exposer le réseau entier. Cette phase va donc consister à exploiter ce vecteur d'attaque.

Maintenir l'accès au système: Pour ce faire, le testeur aura recours à diverses techniques, notamment l'exploitation des canaux de communication cachés et l'installation de backdoors.

Couvrir les traces/brouiller les pistes: À ce stade, le test tire pratiquement à sa fin. Le testeur fait disparaître toute trace des exploits effectués, de tous fichiers suspects créés. Il se sert de techniques telles que l'usage de rootkit.

- **Le rapport**

La dernière phase, comme son nom l'indique, consiste à faire un compte rendu (verbal et écrit) du test effectué.

2.2.2. PTES (Penetration Testing Execution Standards) [3]

- **Le Pré-engagement (Pré-engagement Interactions)**

Cette phase initiale est le lieu pour le testeur et la cible d'aborder les points tels que la portée du test (actifs impliqués, blocs d'adresses IP à attaquer...), la période de test, les considérations légales ainsi que les modalités de paiement.

- **La Collecte d'informations (intelligence gathering)**

Elle aussi se repartie en diverses étapes dont:

OSINT: Open Source Intelligence Gathering il s'agit de recueillir des informations publiques.

Collecte d'informations non publiques (Covert Gathering): Collecte d'informations non publiques sur la cible comme les informations disponibles sur sites (physiques), la location des datacenters et les chaînes d'approvisionnement.

Footprinting: Recueille d'information de façon plus technique, à l'aide d'outils et de techniques comme les scanners de réseaux et de vulnérabilités, ...

Identification des mécanismes de protection: Il s'agit ici de vérifier l'existence d'éventuels pare-feux, IDS, Anti-Virus...

- **Modélisation des menaces (Threat Modeling)**

C'est un processus qui à pour but d'identifier les menaces auxquelles sont exposés les actifs, les prioriser et appliquer des mesures conséquentes.

- **Analyse des Vulnérabilités (Vulnerability Analysis)**

Cette étape consiste à identifier les vulnérabilités et les valider. Elle se déroule de la façon suivante:

Phase de test: La phase testing peut être soit active (interaction avec la cible par le biais de moyens automatiques comme les scanneurs de vulnérabilités ou manuellement), soit passive (étude de métadonnées ou analyse de trafic).

Validation: Elle vise à corréler les vulnérabilités détectées (cas de plusieurs outils utilisés) et de s'assurer de leur crédibilité (faux positifs...). En ayant recours à des vérifications via des attaques à mener manuellement, ou à des recherches (bases de données CVE...).

- **Exploitation**

c'est un processus qui à pour but d'identifier les menaces auxquelles sont exposés les actifs, les prioriser et appliquer des mesures conséquentes.

- **Post-exploitation**

C'est ici que l'attaquant, après avoir eu accès au système, va tenter d'y maintenir son accès pour des contrôles distants ultérieurs. En l'occurrence installer des portes dérobées ou créer une chaîne de communication cachée et exfiltrer des données.

- **Rédaction du rapport (Reporting)**

Enfin, il va s'agir de consigner les points tels que les objectifs du test, les résultats du test et les recommandations dans un rapport.

3. Le Test d'intrusion en matière d'IoT

3.1. Statistiques et chiffres clés en matière de sécurité IoT

3.1.1. Les appareils IoT les plus exposés [4]

Selon une étude publiée par Symantec Data, **75%** des appareils IoT infectés seraient des **routeurs**. Les **caméras IP** quant à elles occupent la seconde place avec **15%**.

3.1.2. Trafic non sécurisé des appareils IoT [5]

Selon une étude publiée en 2020 par Unit 42 (l'équipe de threat intelligence de Palo Alto Networks), **98%** des trafics des environnements IoT ne sont **pas cryptés**. Compromettant ainsi la confidentialité sur le réseau.

3.1.3. Les menaces et vulnérabilités les plus rencontrées dans la sphère IoT (OWASP's Top 10 IoT Vulnerabilities) [6]

- **Mot de passe faible ou codé en dur**

Il est très fréquent de voir des appareils IoT livrés avec des mots de passe par défaut. Ces derniers restent parfois inchangés. Pire, certains mots de passes sont fixes/codés en dur pour des accès à des backdoors à des fins de débogages. Il s'agit là d'une véritable aubaine pour les pirates.

- **Services réseaux non sécurisés**

Les réseaux IoT sont fort menacés par des cyberattaques en raison des ports (et par ricochet les services qui leurs sont associés) non sécurisés et/ou non nécessaires qui sont actifs. Ce qui rend les composants IoT très vulnérables aux attaques du type DDOS, où ces derniers sont utilisés dans des botnets.

- **Interfaces non sécurisées**

Il s'agit des interfaces web, mobiles, des API côté serveur, du cloud. Elles pourraient constituer des vecteurs d'attaques, car souffrant très souvent de vulnérabilités: manque de cryptage, mauvais processus d'authentification, etc

- **Manque mécanismes de sécurisation des mises à jour**

Les mises à jour ne doivent pas transiter en clair. Elles doivent employer des mesures cryptographiques adéquates. Des vérifications du firmware (intégrité) doivent être effectuées. Et des mécanismes de protection contre les attaques comme downgrade (faire passer le logiciel à une version antérieure moins sécurisée).

- **Usage de composants pas à jour et non sécurisés**

Les appareils IoT sont vulnérables parfois en raison des composants matériels et logiciels inappropriés qu'ils utilisent. Cela va des librairies/logiciels vulnérables aux gadgets provenant d'une chaîne d'approvisionnement compromise.

- **Pas de mécanisme efficace de protection de la confidentialité de l'utilisateur**

Les données de l'utilisateur transitent parfois en clair dans l'écosystème IoT, compromettant la confidentialité de ce dernier.

- **Transfert et stockage non sécurisés**

À ce niveau il est question d'un manque de cryptage des données au repos, en cours de transmission et lors du traitement

- **Manque de gestion des appareils**

Les appareils une fois installés et configurés pour la première fois doivent subir des processus réguliers de gestion pour la mise à jour, l'inventaire, le monitoring, etc. Sans cela, ils sont exposés à des cyberattaques.

- **Mauvaise configuration par défaut**

Une autre vulnérabilité à laquelle on assiste lorsque les conditions suivantes sont réunies: d'abord l'appareil IoT vient de chez le constructeur avec des configurations initiales pas assez robustes d'un point de vue de la sécurité. Mais aussi et surtout quand celui-ci contient des restrictions quant aux modifications qu'on pourrait apporter à la configuration

- **Manque de sécurité physique**

Les appareils IoT souffrent parfois de cette vulnérabilité, ce qui permet aux attaquants par exemple d'accéder aux ports de débogages, d'extraire les données de la carte mémoire.

3.2. Les surfaces d'attaques des écosystèmes IoT [7]

Un écosystème IoT combine divers vecteurs d'attaques qui constituent ensemble la surface d'attaque de l'écosystème. Ce sont ces vecteurs que vont cibler le testeur durant le pentest. La fondation OWASP à classifié ces vecteurs d'attaques en plusieurs surfaces d'attaques qui ont été substantiellement consignées ci-dessous par un travail de recherche que nous avons exploré.

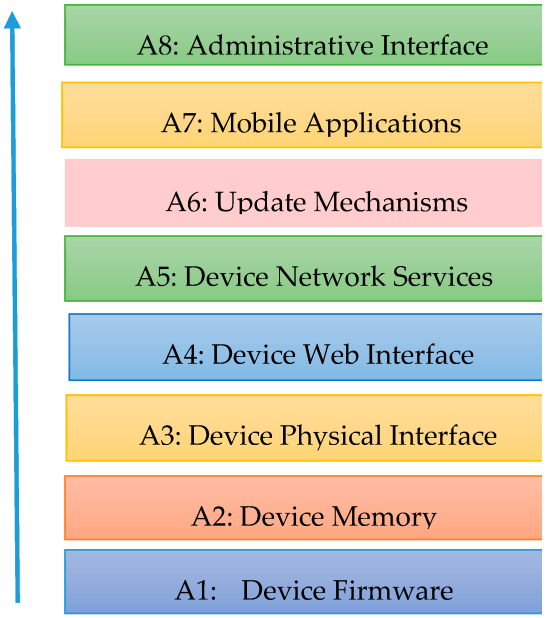


Figure 4: *Surfaces d'attaque IoT*

3.2.1. Les attaques sur la surface d'attaque physique

À ce niveau se distinguent deux formes d'attaques, les attaques discrètes (covert attacks) et les attaques non discrètes (overt attacks).



Figure 5: *Types d'attaques physiques*

- **Les attaques discrètes (covert attacks)** [8]

Les attaques du type SPA (single power analysis):

Ce type d'attaque consiste en une analyse de signal émis par l'équipement cible. Il s'agit précisément d'interpréter la consommation en énergie de l'équipement, à travers des graphes permettant de visualiser les activités électriques de ce dernier au cours du temps. Le principe: chaque opération effectuée par le microprocesseur induit une activité électrique particulière.

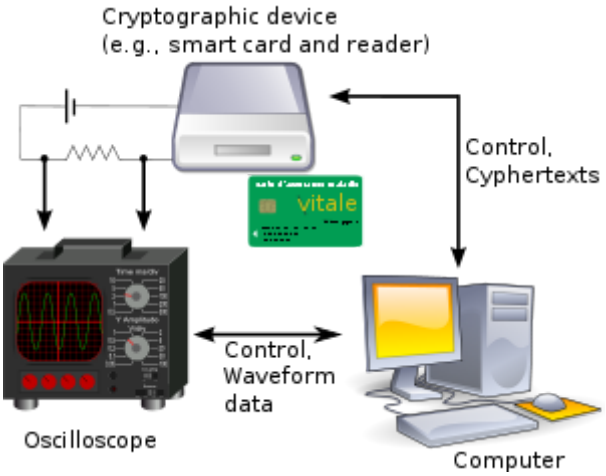


Figure 6: *équipement requis pour mener une attaque SPA*

Attaque via port JTAG (C-JTAG, Covert JTAG Port Attack)

JTAG (Joint Test Action Group) est un standard pour les tests et débogage des circuits imprimés. Un port JTAG est donc une interface permettant d'accéder à d'importantes fonctionnalités de débogage d'un équipement IoT, et par suite un vecteur d'attaque prisé en matière de hardware hacking. Le hacker y ayant accès peut entre autres modifier la mémoire, les valeurs de registres ou avoir accès à des fonctionnalités de débogage.

Une attaque discrète pouvant être menée est le sniffing du port JTAG, ou l'extraction d'informations sensibles au niveau du Chip à partir de ce port.

Pour effectuer l'attaque sniffing, l'on a recours à des outils comme des analyseurs logiques comme Saleae Logic Analyzer, des logiciels comme pulsview pour visualiser les signaux circulant via les ports JTAG.

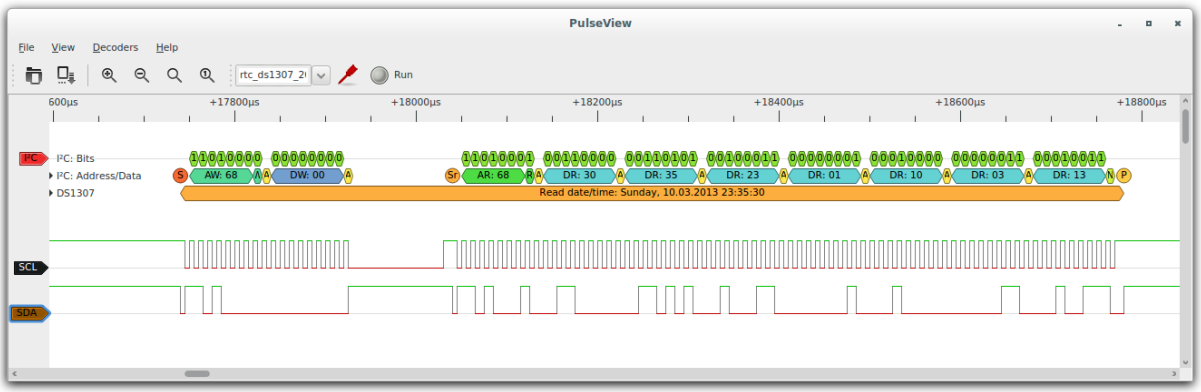


Figure 7*: PulseView*

- **Les attaques non discrètes (overt attacks) [9]**

Ces attaques, contrairement aux précédentes (covert attacks), laissent la victime savoir qu'elle subit une attaque, laissent des traces (crée des dénis de services, ...). Comme exemple on a:

L'attaque FI (fault injection attack) [10]:

Cette attaque consiste à injecter dans l'équipement un signal perturbateur en vue d'altérer son fonctionnement. Cette attaque cible généralement l'alimentation électrique des contrôleurs ou les signaux d'horloge. Dans le cadre des attaques hardware, la fault injection peut être avec ou sans contact direct. Avec contact direct, elle consiste le plus souvent à injecter du courant additionnel via les broches de circuits ou les ports JTAG pour compromettre altérer le bon fonctionnement du chip. Elle peut aussi consister à créer une perturbation électromagnétique. OpenOCD (Open On Chip

Debugging) est un outil fort répandu permettant de communiquer avec le chip via les ports JTAG, voire y injecter du code sous forme de 0 et 1.

3.2.2. Attaques sur les systèmes IoT ciblant la mémoire

- **Le bus probing ou microprobing [11]**

Cette attaque consiste à se servir d'outils tel qu'un microscope équipé de sondes microscopiques que l'on met en contact avec les bus du chip en cours d'exécution; cela peut être utilisé soit pour lire des données sensibles internes (sous forme de bits) qui ne sont pas destinées à quitter la mémoire de la puce (notaments des clés cryptographiques). Il peut être utilisé pour des attaques d'injection (hardware fault injection).

Dans le premier cas, il s'agit d'une technique de lecture du contenu sensible d'une mémoire. Certaines attaques basées sur cette technique ont permis de casser des algorithmes cryptographiques tels que RSA et DES.

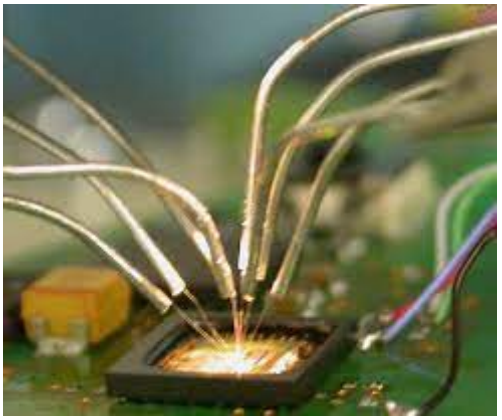


Figure 8*: Microprobing*

- **Buffer Overflow**

Cette attaque qui peut être classée parmi les attaques de la couche applicative, figure dans la couche mémoire en raison de son but: corruption de mémoire et exécution de code arbitraire.

Elle consiste à fournir en entrées des données dont la taille est supérieure à celle du buffer (qui est aussi un espace mémoire). Ce débordement à pour effet d'écraser des données localisées dans les zones mémoires environnantes. Si parmi les données écrasées se trouve une adresse de retour d'une fonction, cette adresse de retour prend désormais une nouvelle valeur. Et c'est là que se trouve

l'utilité de cette attaque dans le cadre de corruption de mémoire. Il suffit à l'attaquant d'écraser l'adresse retour avec une adresse pointant vers un code malicieux à exécuter.

Des outils de fuzzing tels que BED (Bruteforce exploit detector) ou l'outil metasploit présents dans la distribution Kali sont très utiles pour tester cette vulnérabilité.

3.2.3. Attaques sur le firmware

Parmi les attaques de cette catégorie, on peut mentionner l'implantation de backdoor.

- **L'implantation de backdoor [12] [13] [14]**

Elle consiste à insérer dans le code du firmware un programme permettant d'accéder à l'équipement à travers un shell distant. Cette attaque est très souvent menée sur les systèmes n'ayant pas de mécanisme de vérification de l'intégrité du firmware.

3.2.4. Attaque ciblant le mécanisme de mise à jour

- **L'attaque downgrade/version rollback**

Cette attaque consiste à faire passer un système/protocole/logiciel à un niveau/une version antérieur(e) et donc moins amélioré(e) et moins sécurisé(e). L'attaque downgrade est le plus souvent rencontrée dans une communication employant les protocoles SSL/TLS ou HTTP(S). Dans ce second exemple l'attaquant va par faire passer le trafic du protocole HTTPS au HTTP qui lui permettra de sniffer voir modifier le trafic.

Aussi l'attaquant peut tout simplement installer une version antérieure du firmware, s'il en possède l'habileté (droit d'accès privilégié, ...).

3.2.5. Attaques et vecteur d'attaques de la couche applicative mobile

La fondation OWASP à travers le classement "Top 10 Mobile Risks" fait cas des vulnérabilités et canaux d'attaque les plus rencontrés dans le monde des applications mobiles. Parmi ces canaux d'attaques:

- **Stockage de données non sécurisé**

Ce canal d'attaque occupe la **seconde place** dans le classement OWASP. Il est question ici des procédés de stockage vulnérables employant par exemple des mesures de **cryptage faibles ou inexistantes des données** sensibles au repos. Un accès physique à l'équipement serait suffisant pour recueillir des clés et informations personnelles.

3.2.6. La couche applicative web/interface administrateur

De même que pour la couche précédente, OWASP définit un classement des canaux d'attaques phares. Parmi elles:

- **Les injections**

Des formes d'attaques englobant l'injection SQL, l'injection LDAP, l'injection OS, ...

- **Mauvais mécanismes d'authentification**

Canal entretenu par la mauvaise gestion des sessions, et du recouvrement de données d'authentification.

3.2.7. La couche réseau/communication [15] [16]

- **Le MITM**

Le MITM (Man in the Middle) consiste à intercepter une communication entre deux nœuds en se faisant passer pour l'autre auprès de l'un, ayant la possibilité de lire et modifier le contenu du trafic.

La version récente du standard BLE implémente des procédés cryptographiques pour l'appairage des équipements (pairing). Les recherches stipulent que plus de 80% des appareils IoT recourant à BLE n'implémentent pas ou implémentent mal cette couche de cryptage. Autant de vecteur d'attaque permettant de mener à bien l'attaque citée plus haut.

Il existe des outils pour réussir une attaque de MITM dans un réseau bluetooth. Parmi eux on a Gattacker ou encore BtleJuice.

3.3. Méthodologies / standards / frameworks en pentesting IoT

Il n'existe pas assez de standards de pentest spécialement pour l'IoT. Par contre, il existe plusieurs standards/framework/méthodologies pour assurer et/ou vérifier la sécurité d'un écosystème IoT. Dans ce chapitre, il sera question d'aborder ces standards, ainsi d'autres méthodologies (non des standards) développées par des particuliers (en l'occurrence Rapid7 les propriétaires de Metasploit).

3.3.1. Le ISVS (IoT Security Verification Standard) [17]

C'est un standard pour les exigences de sécurité d'un environnement IoT, en répartissant ces exigences selon **5 couches**: l'**écosystème**, l'espace **applicatif utilisateur**, l'espace **logiciel**, la couche de **communication**, et la couche **physique**. Aussi ce standard à une approche basée sur des **niveaux de sécurité**: **I**(attaque ciblant la couche logicielle), **II**(attaque ciblant la couche logicielle et la couche physique), et **III**(concerne les équipements pour lesquels des attaques devraient être évitées à tout pris, comme les implants médicaux connectés). Pour des raisons de fluidité, nous ne nous étalons pas sur les détails du contenu du standard. Cette partie se limitera aux grands points.

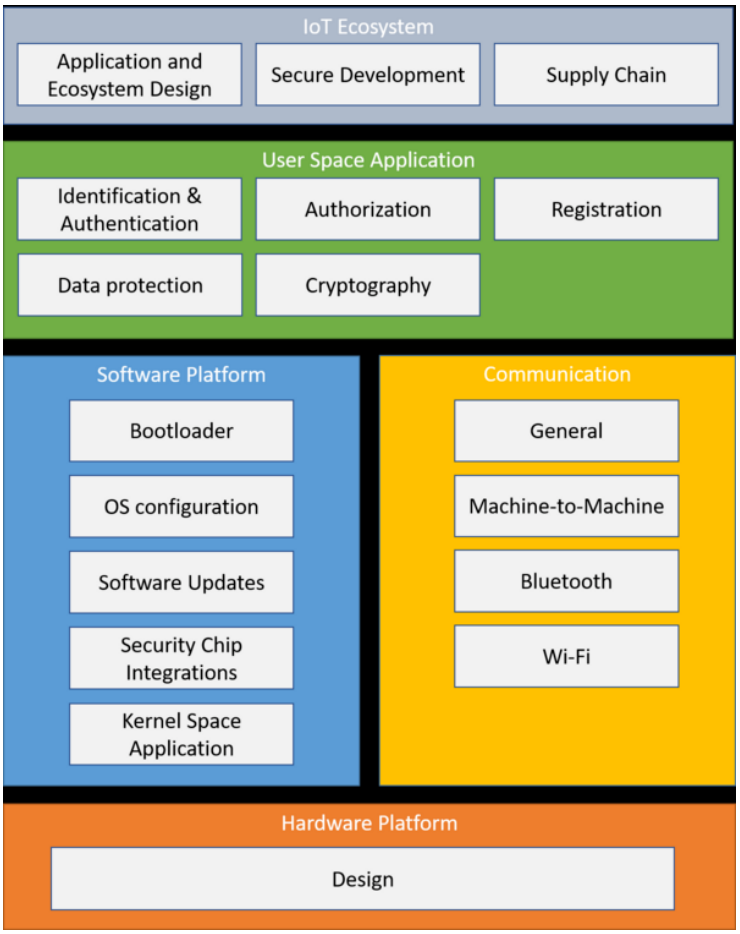


Figure 9*: l'IoT Security Verification Standard*

- **Exigences liées à l'écosystème**

Il contient un ensemble d'exigences que pourrait valider. Ces exigences sont réparties en différentes sous parties que sont:

- ✓ Le design de l'application et l'écosystème (6 contrôles)
- ✓ Le design de l'application et l'écosystème (6 contrôles)
- ✓ La chaîne d'approvisionnement (8 contrôles)
- ✓ Sécurisation du développement (15 contrôles)
- ✓ La journalisation (7 contrôles)

- **Exigences liées à l'espace applicatif utilisateur**

- ✓ Identification et authentification (11 contrôles)
- ✓ Autorisation (4 contrôles)
- ✓ Protection des données (4 contrôles)
- ✓ Cryptographie (6 contrôles)

- **Exigences logicielles**

- ✓ Bootloader (8 contrôles)
- ✓ Configuration du système d'exploitation (11 contrôles)
- ✓ Mises à jour logicielles (12 contrôles)
- ✓ Sécurité au niveau du chip (4 contrôles)
- ✓ Sécurité des applications proches du noyau (2 contrôles)

- **Exigences de la couche communication**

- ✓ Exigences générales (7 contrôles)
- ✓ Machine à machine (3 contrôles)
- ✓ Bluetooth (7 contrôles)
- ✓ WI-FI 4 (contrôles)

- **Exigences de la couches physiques**

Il comprend 11 contrôles abordant par exemple la vérification de la sécurité des ports physiques de débogage.

3.3.2. FSTM (Firmware Security Testing Methodology) [18]

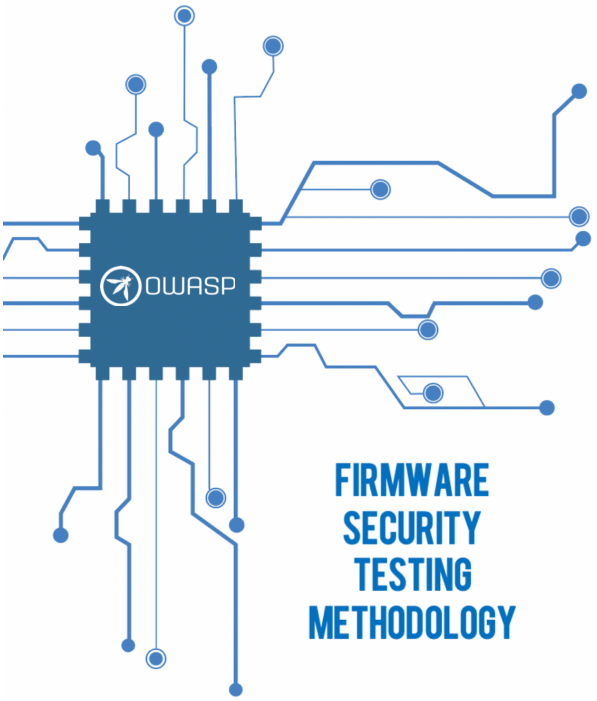


Figure 10*: logo du FSTM*

Ce standard guide les tests d'intrusion IoT en en adressant uniquement la couche firmware. Il est constitué de 9 étapes que sont:

- **Collecte d'Informations**

Les informations ciblées sont entre autres le système d'exploitation, l'architecture du microprocesseur, les configurations du bootloader, les fiches techniques, etc

Les outils qui entrent en jeu sont pour certains, Coverity Scan, LGTM, ou un moteur de recherche.

- **Extraction du firmware**

Il est question d'obtenir une image du firmware. Il existe plusieurs méthodes pour cette étape. Elle peut se faire directement par accès physique au hardware via les ports JTAG, UART... On peut aussi recourir au MITM pendant une mise à jour, ou encore des requêtes google dork pour des binaires sur internet.

- **Analyse du firmware**

Cette phase consiste à recueillir des informations sur le fichier binaire extrait. On peut avoir usage aux commandes telles que file (sur Linux). Les outils tels que Binwalk ou Binvis peuvent entrer en jeu.

- **Extraction du système de fichier**

L'objectif est d'extraire des informations primaires à travers le système de fichier(configurations de l'équipement, code non compilé...). les outils tels que binwalk et la commande dd sont là encore très utiles.

- **Analyse du système de fichier**

C'est la phase précédente, mais en plus approfondie. On cible des informations telles que: Des données d'identifications et d'authentifications codées en dur (noms utilisateurs, clés API, clés SSH...), des détails sur les serveurs back-end, de mise à jour etc, les fichiers de scripts. Comme outils utilisés on peut citer Firewalker, FACT (Firmware Analysis Comparison Toolkit).

- **Émulation du firmware**

Le firmware est émulé pour entre autres vérifier les vulnérabilités. Cette phase est très utile lorsque l'on dispose du firmware et que l'on veut l'exécuter sans avoir recours à l'équipement.

- **Analyse dynamique**

À ce stade, il sera question de faire les tests d'intrusion sur l'équipement abritant le firmware pendant qu'il est actif. Les outils de pentesting classique tels que Nmap, OWASP ZAP, metasploit, sont très utiles. D'autres tels que firmware-mod-kit (FMK) sont plus focus sur les tests d'intégrité du firmware.

- **Analyse d'exécution**

L'analyse d'exécution consiste en la connexion à un processus en cours d'exécution ou à un binaire pendant qu'un périphérique s'exécute dans son environnement normal ou émulé. Des exemples d'outils utiles sont gdb-multiarch, Binary Ninja, etc.

- **Exploitation binaire**

Une fois les vulnérabilités identifiées, il est question de les exploiter. On peut avoir recours à des exploits ou en créer soi- même (des codes utilisant des techniques de ROP par exemple (Return

Oriented Programming)). Des outils existent pour faciliter cette phase. On a par exemple le ROPGadget.

3.3.2. La méthodologie de pentest IoT proposée par Rapid7 [19]

- **Reconnaissance**

En ayant recours à l'OSINT (Open Source INTelligence gathering), on collecte le maximum d'information sur la cible.

- **Test de la surface d'attaque cloud/web**

Pour cette phase l'on a recours au technique de pentest web classique, en utilisant les outils tels que burpsuite, OWASP ZAP, etc.

- **Test de la surface d'attaque mobile**

Les solutions IoT offrent souvent des accès aux services via des applications mobiles. Il s'agit d'identifier les failles associées à ce vecteur d'attaque. OWASP mobile top 10 fournit des menaces pertinentes à adresser à ce niveau.

- **Test de la surface d'attaque réseau**

Cette phase ne diffère pas vraiment du test réseau classique. Elle consiste à se servir des outils comme Nmap, metasploit, etc pour identifier des vulnérabilités associées aux ports et services réseaux de l'écosystème.

- **Test de la surface physique**

Cette phase se focalise sur l'appareil physique IoT. Elle consiste à identifier l'ensemble des ports physiques, ports de débogage (JTAG, ...), les médias de stockage. Il va s'agir de tester l'accès à la console de débogage, la possibilité de modifier la configuration, et bien d'autres.

- **Test des communications radio**

Les communications radio sont très employées dans les solutions IoT. Cette phase consiste à tester la robustesse des mécanismes employés lors des trafics, d'effectuer des attaques comme les "replay attacks".

III. Analyse et critique de l'existant

Lors de cette phase, il sera question d'étudier l'existant, c'est-à-dire la question des méthodologies et standards en matière de pentest IOT. Nous allons ensuite faire des critiques constructives afin de proposer une approche pour solutionner les problèmes de l'existant.

1. Existant d'ATHENA

Athena dispose pour ces tests d'intrusion, de méthodologies qui guident les experts dans leur tâche. On distingue entre autres des méthodologies de tests d'intrusion Web, de tests d'intrusion d'applications mobiles, d'audit de configuration ou encore d'audit d'architecture.

Cependant, ATHENA ne dispose pas de méthodologie pour les tests d'intrusion IoT. Ses méthodologies sont adaptées aux tests d'intrusion dans un contexte plus général et pas spécifique aux IoT. Par ailleurs, certaines couches ne sont pas adressées, notamment la couche Physique (références faites aux attaques physiques).

2. Existant au-delà d'ATHENA

Pour ce qui est des standards et méthodologies, il n'en existe pas qui soit spécifiquement dédié aux tests d'intrusion IoT et qui adresse ce sujet de façon complète (toutes les couches).

Certes il en existe dans le cadre de la sécurité des IoT d'une façon générale, mais il ne s'agit pas de méthodologie de pentest. Seulement des standards de conformité pour les uns (Code of Practice for Consumer IoT Security), ou des standards de vérification de la sécurité et de code de bonnes pratiques pour les autres (IoT Security Verification Standard, ISVS).

Et quand ces standards sont adaptables au contexte du test d'intrusion IoT, il l'aborde partiellement (FSTM, Firmware Security Testing Methodology).

3. Les couches et les vecteurs/surfaces d'attaques

Notre but est de concevoir une méthodologie qui se veut **la plus complète possible** (couvre tous les aspects) tout **en évitant les redondances**.

Nous avons donc proposé une approche par couches. Chaque couche rassemble un ensemble de surface et de vecteurs d'attaques, en s'inspirant de plusieurs standards (en l'occurrence ceux précédemment évoqués).

3.1. Exploration des surfaces d'attaque IoT selon OWASP

Le tableau qui suit contient la liste exhaustive (cellules à droite) des surfaces et vecteurs d'attaque publiée par l'OWASP en matière d'IoT. Nous nous sommes proposés de les regrouper en couches (cellules colorées à droite).

Surfaces d'attaque	Classifications
Device Physical Interfaces (Sensors)	Surface Physique
Hardware	
Device Memory	Surface Physique/Applicative
Device Web Interface	Surface Applicative (Web/Cloud, Mobile)
Administrative Interface	
Cloud Web Interface	
Mobile Application	

Vendor/Third-party Backend APIs	
Authentication/Authorization	
Privacy	
Local Data Storage	**Surface Firmware/Applicative**
Device Firmware	**Surface Firmware**
Update Mechanism	**Surface Firmware/Communication**
Network Traffic	**Surface Communication**
Ecosystem Communication	
Device Network Services	

Tableau 1*: Surfaces d'attaques des IoT selon OWASP*

On peut donc stratifier les surfaces d'attaque proposées par OWASP de la façon suivante: Surface **Physique,** Surface **Applicative,** Surface **Firmware,** Surface **Communication.**

3.2. Exploration du standard ISVS (IoT Security Verification Standard)

Par contre, pour ce qui est du standard ISVS (voir image figure 9), on a les surfaces: **hardware** (assimilables à la surface Physique de notre classification), espace **applicatif** utilisateur (assimilables à la surface Applicative de notre classification), software (qui aborde des points assimilables à la surface **Firmware** de notre classification précédente) et **Communication**.

NB: La surface **écosystème** (général) dans les celles citées par OWASP, ainsi que celle évoquée dans le standard ISVS n'ont pas été prises en compte; nous les trouvons plus adaptées à des contrôles de conformité et de conception sécurisée et non à notre contexte de pentest.

3.3. Exploration du standard "Code of Practice for Consumer IoT Security"

Il en est de même pour le standard "*Code of Practice for Consumer IoT Security*" dont les exigences sont résumées ci-dessous.

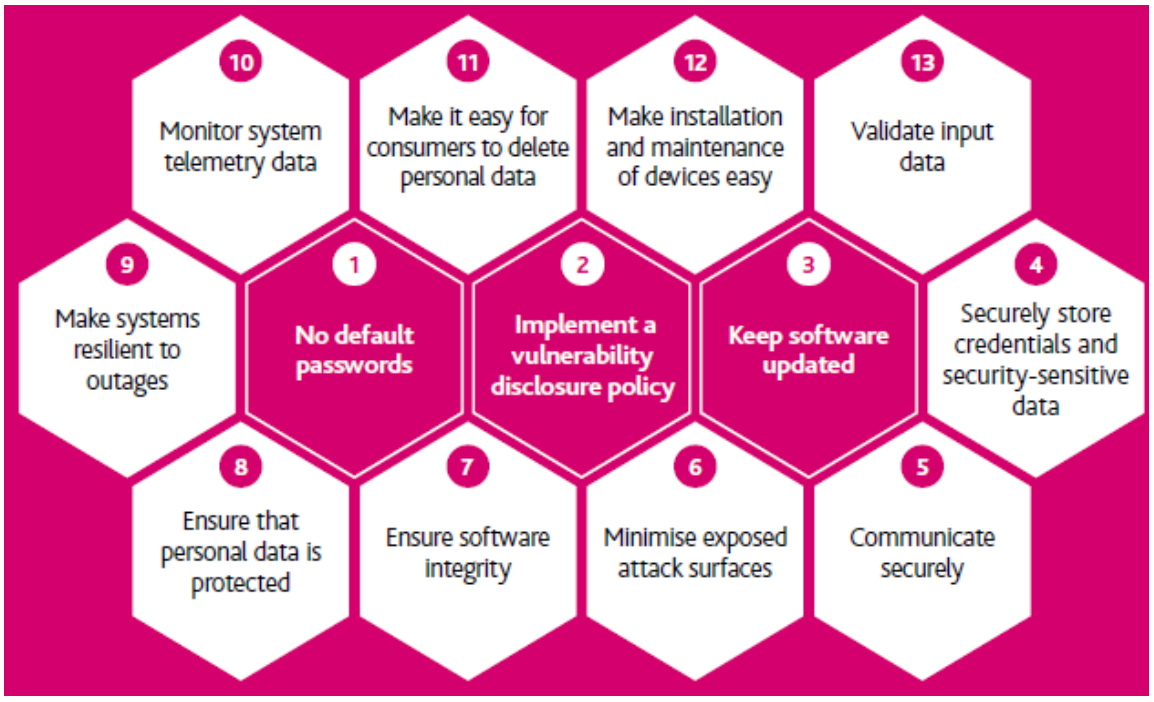


Figure 11: *les exigences du "Code of Practice for Consumer IoT Security"*

Les contrôles 1, 3, 4, 7, 8 peuvent être inclus dans la **couche applicative** (le 7 peut aussi appartenir à notre **couche firmware**)

Les contrôles 5, 10 peuvent être inclus dans la **couche communication**.

Les contrôles 2, 11, 12 ne sont pas des contrôles à prendre en compte dans notre cadre de pentest.

Enfin le contrôle 6 est assez général (concerne plusieurs couches)

3.4. Conclusion Partielle

Les couches pour élaborer notre méthodologie peuvent donc se résumer à: couche **Physique**, couche **Applicative**, couche **Firmware**, couche **Communication**.

IV. Cahier de charges et Objectifs

Comme précédemment évoqué, ce projet vise à élaborer une méthodologie pratique pour guider les experts de ATHENA pendant les tests d'intrusion. Pour ce faire, notre travail a consisté à:

- S'inspirer de divers standards et méthodologies existants pour proposer une méthodologie adaptée aux tests d'intrusion IoT.

- Mettre en place une architecture de type IoT et home network.

- Mener des tests d'intrusions sur l'architecture mise en place, en suivant ladite méthodologie afin de la valider.

CHAPITRE II: Elaboration de la méthodologie

Nous allons maintenant procéder à l'élaboration de la méthodologie. Pour ce faire, nous commençons par décrire l'architecture à laquelle elle sera appliquée et le modèle de l'attaquant. Après quoi nous passerons à l'élaboration proprement dite.

I. ARCHITECTURE DE LA CIBLE D'ÉVALUATION

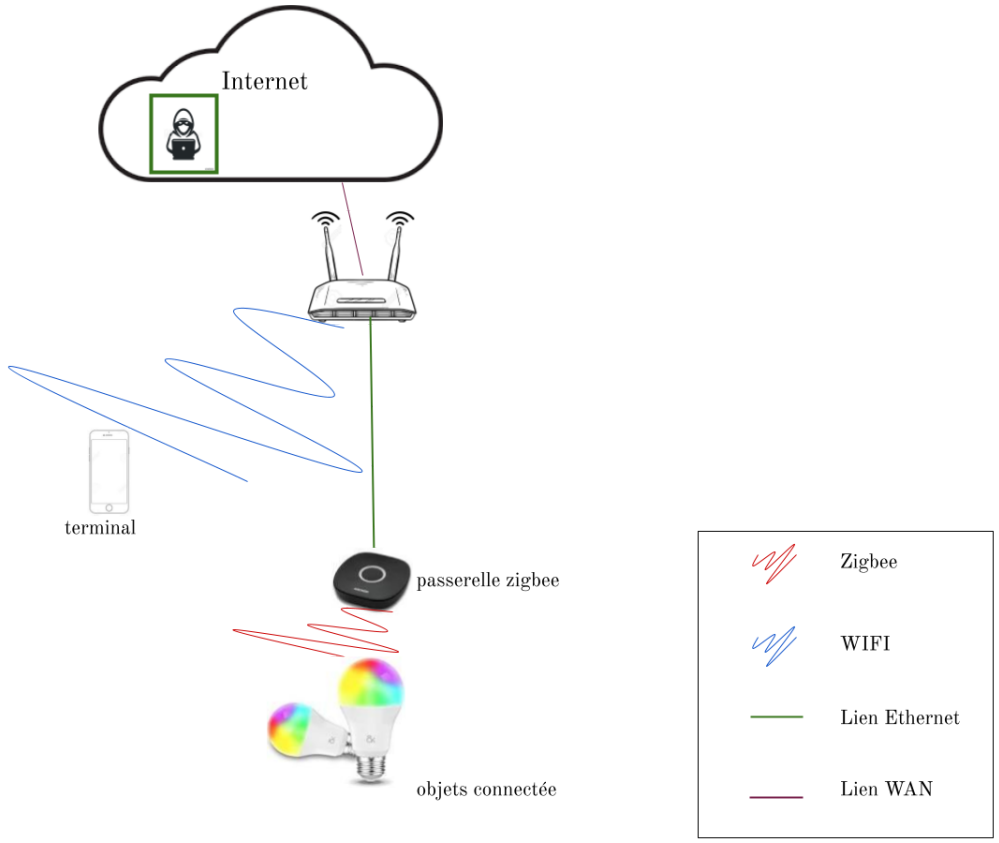


Figure 12: *Architecture de la cible.*

Cette architecture est à considérer dans sa forme générale: les terminaux et les protocoles réseaux peuvent varier selon les cas pratiques (pas forcément smartphone et Zigbee...).

Notre cible pour les tests d'intrusion sera de cette forme. Elle contiendra donc principalement une box internet, des terminaux qui lui seront connectés (IoT, PC, ...), et aussi d'autres passerelles (comme la passerelle zigbee) auxquelles seront connectés d'autres équipements IoT .

II. MODÈLE DE L'ATTAQUANT

Cette section décrit le testeur/attaquant. Plus concrètement, elle décrit les connaissances préalables de l'attaquant sur la cible, ses positions et ses capacités dans chacune d'elles.

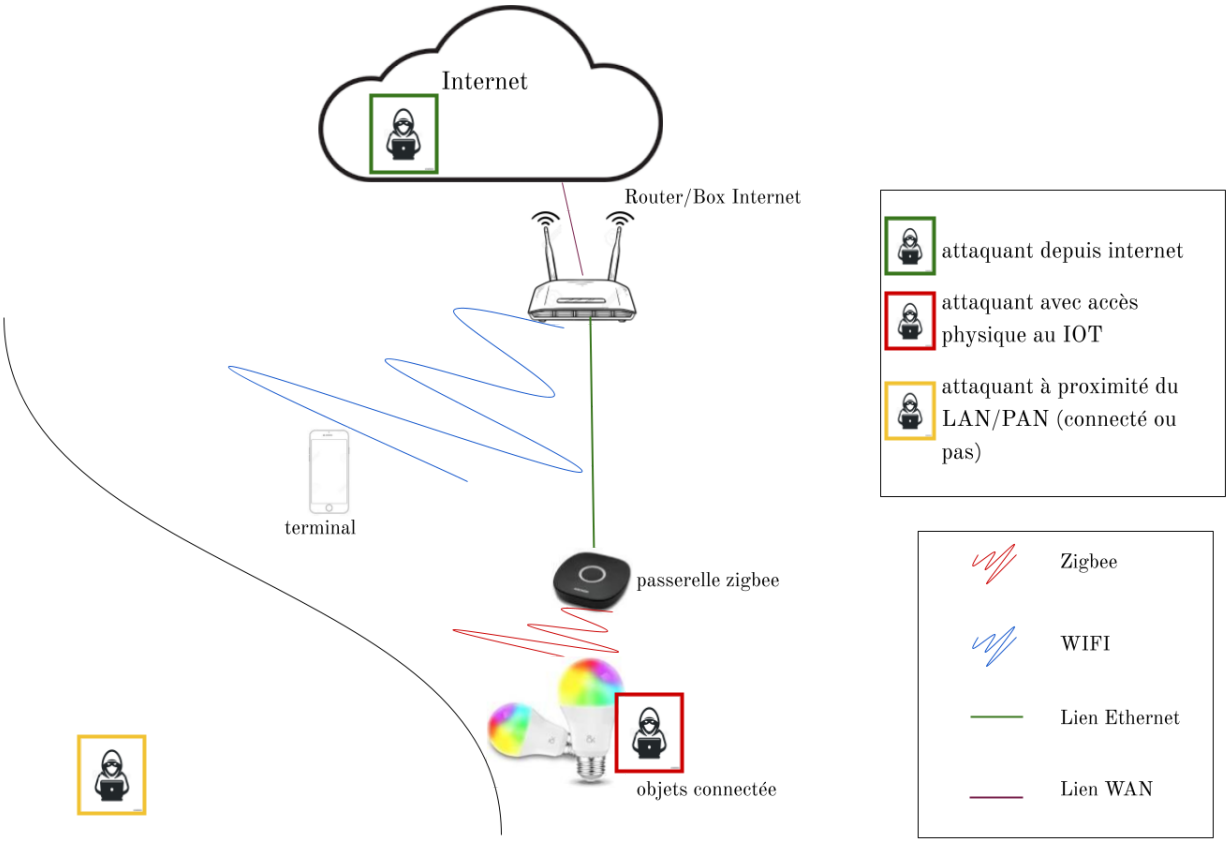


Figure 13: *Modèle de l'attaquant.*

1. Les connaissances et ressources préalables du testeur d'intrusion

Les connaissances et ressources préalables de l'attaquant varient selon les exploits qu'il veut mener, les étapes en cours, les couches qu'il adresse, et de sa position.

De ce fait, le testeur devra se référer aux rubriques *prérequis/entrées* et *outils de* chaque étape.

2. Positions de l'attaquant vis-à-vis des équipements.

Certaines attaques nécessitent une vue particulière sur les équipements et l'environnement.

Ainsi la méthodologie distingue 3 vues: depuis internet, étant dans la zone de couverture du réseau, et ayant accès physique aux équipements.

2.1 Accès Physique

Les attaques sur la couche physique (et souvent firmware) nécessitent une certaine proximité de l'équipement.

Le testeur devra parfois démonter l'équipement pour en analyser les composants, ou pour en altérer le fonctionnement. Il pourra aussi avoir accès aux ports de l'équipement. Cette vue particulière sur l'équipement est souvent nécessaire pour en extraire des données stockées au niveau des puces (Chip).

Ici on est dans l'hypothèse où le testeur est infiltré dans l'environnement, ou dans la chaîne d'approvisionnement ou encore dans l'équipe/le service de maintenance.

2.2 Accès depuis internet

Comme le nom de cette position l'indique, le testeur essaie d'accéder à / compromettre la cible. Il peut tenter d'accéder à la box par l'interface de management à distance (si cette fonctionnalité est activée et si le testeur est en possession de l'adresse publique de la box).

Il peut aussi forger des liens malicieux et user de techniques d'ingénierie sociale pour exploiter des failles au niveau de l'interface web de la box.

2.3 Dans la zone de couverture du réseau

Cette vue est mentionnée au niveau des tests de la couche réseau. En ayant cette vue, le testeur a 2 positions qui vont influencer les choix de ses attaques: connecté et pas connecté (au réseau cible).

Lorsqu'il est en position 'pas connecté', le testeur aura tendance à mener les attaques lui permettant de s'introduire dans le réseau (craquage de mot de passe du point d'accès...). Dans ce premier cas, le testeur est juste dans les environs de la cible (ou du réseau de la cible). Son but est de passer à la vue 'connectée' pour ainsi mener des attaques plus poussées (faire du mouvement latéral par exemple).

Lorsqu'il est connecté, le testeur a un meilleur accès et peut mener d'autres formes d'attaques beaucoup plus élaborées que nous détaillerons dans la méthodologie.

III. LA MÉTHODOLOGIE

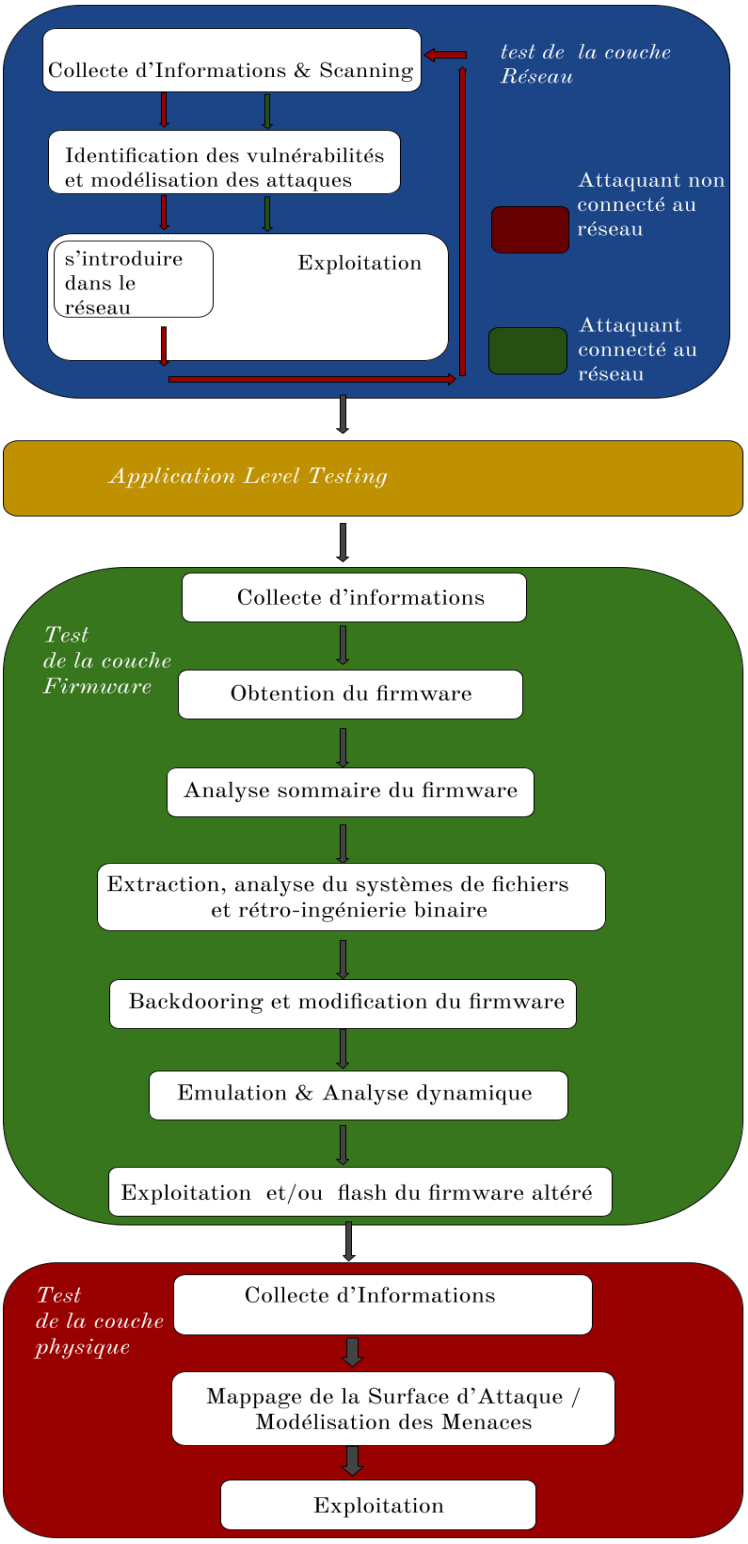


Figure 14: *La méthodologie schématisée*

Les outils mentionnés le long de la méthodologie ne sont que des propositions. Le testeur est libre d'avoir recours aux outils de son choix.

Les tests abordant chaque couche sont quasiment indépendants. Mais l'ordre proposé est l'idéal. En effet, il est par exemple plus judicieux de mener les tests physiques en dernière position car nécessitant parfois de vives interactions comme le démontage. Après quoi les autres tests risquent de ne pas être possibles. La méthodologie suggère de mener les tests de la couche applicative après ceux de la couche réseau car de façon générale, les attaquants commencent par s'introduire dans le système en empruntant la voie du réseau.

Par ailleurs, bien qu'entre les 4 couches il y ait une certaine indépendance (les sorties des unes ne sont pas forcément les entrées des autres), les étapes au sein de chacune d'elles sont dépendantes les unes des autres. Par exemple, au sein de la couche firmware, les étapes sont tout à fait liées et corrélées entre elles.

La méthodologie est accompagnée par deux autres documents que sont l'annexe descriptive des procédures des tests, et la check-list des tests et attaques. La check-list propose des pistes au testeur pour mener des exploits lors de la phase d'exploitation. Les procédures sont abordées brièvement dans la méthodologie et pour le déroulement détaillé le testeur devra se référer à l'annexe descriptive procédures des Tests.

De façon pratique, le testeur aura plus à travailler avec la check-list et l'annexe descriptive des procédures des tests. En effet la check-list reprend l'ensemble des tests (de façon plus synthétique et pratique) et ces tests sont décrits de façon détaillée par l'annexe descriptive des procédures des tests.

A: Test de la couche réseau

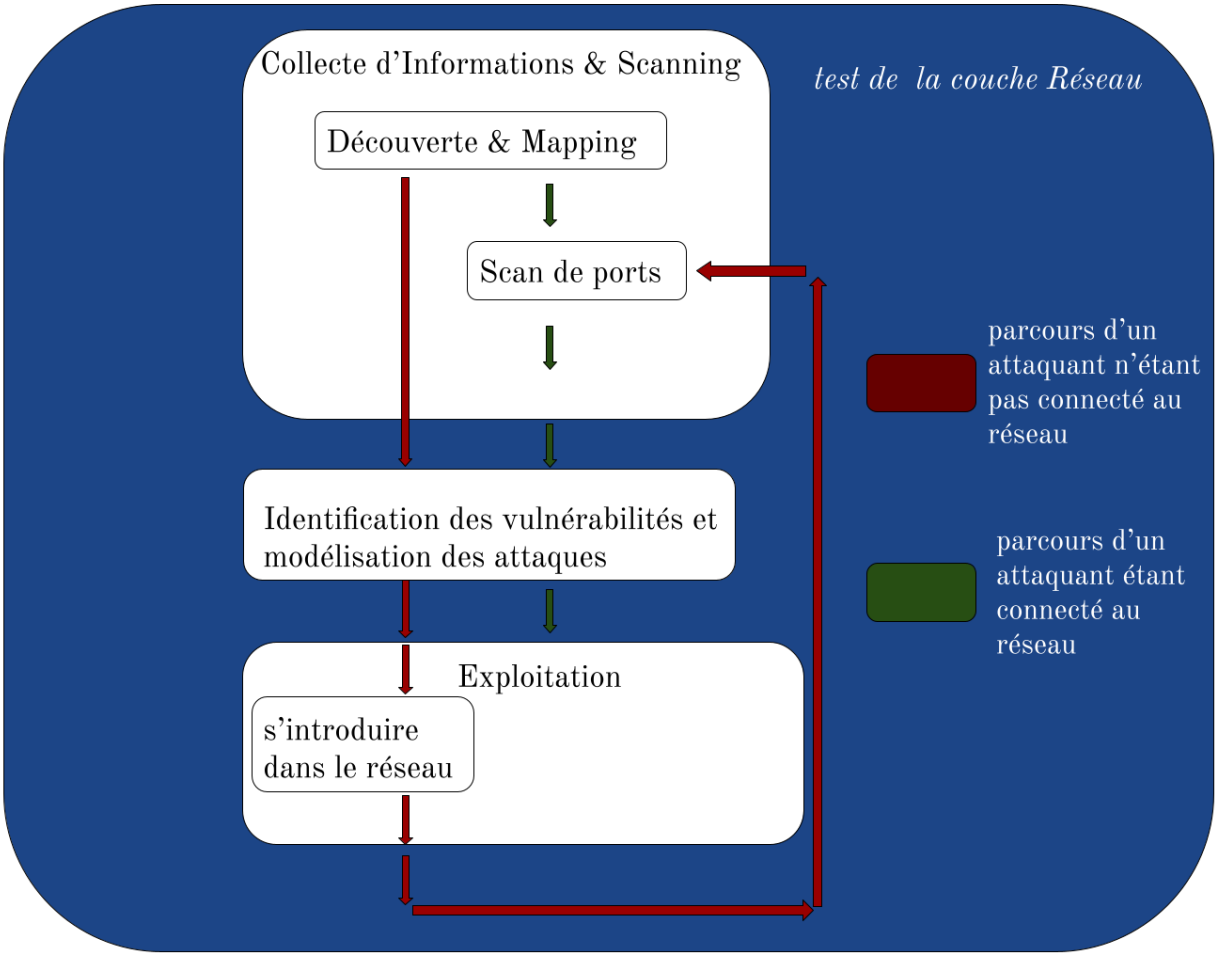


Figure 15: *La méthodologie pour la couche réseau*

Description: À ce stade le testeur teste la robustesse du réseau Cible.

La vue "Dans la zone de couverture du WI-FI mais pas connecté" limite l'attaquant quant à certaines attaques à mener. Pour ce qui est des scans, il ne peut pas attaquer directement les équipements IoT à l'intérieur du réseau vu qu'ils sont dotés d'adresses IP privées converties en une unique adresse IP publique via NAT (généralement dans un contexte home network). Ce qui explique pourquoi dans ce cas le testeur fait fi des étapes de scan. Il devra lors du Mappage de la surface d'attaque et de l'exploitation se pencher sur les attaques visant à s'introduire dans le réseau (craquage de mot de passe, attaques par force brute, attaque par dictionnaire, sniffing...)

Une fois introduit dans le réseau le testeur aura plus d'opportunités d'attaques et pourra approfondir la phase de reconnaissance avec des scans.

Nous rappelons que les procédures sont abordées descriptivement, le déroulement détaillé étant dans l'Annexe des procédures des Tests.

I. Collecte d'informations sur le réseau cible et Scanning

Le but de cette étape est de recueillir des informations sur le réseau qui nous seront utiles dans la suite des tests. Ces informations ont pour but principal de répondre aux interrogations suivantes:

- Quels sont les réseaux à la portée du testeur?
    - Avons-nous un réseau WI-FI actif ?
    - Avons-nous un réseau Zigbee actif ?
    - Des équipements communiquant par Bluetooth / BLE au moment du test?
- Quels sont les protocoles utilisés ?
    - WEP ?
    - WPA?
    - etc
- Quels sont les équipements actifs et quels en sont nos cibles?
- Quelle est la topologie de notre réseau?
- Quels en sont les ports et services ouverts?

1. Découverte et mappage

Objectifs: C'est le lieu pour l'attaquant d'identifier les réseaux ainsi que les équipements IoT actifs et donc potentiellement attaquables. Il pourra aussi recueillir certaines informations comme celles mentionnées en objectif plus haut. Et ensuite en déduire une cartographie si cela est nécessaire.

Cette étape requiert que l'attaquant soit proche de l'environnement à tester ou qu'il soit connecté à un des réseaux de la cible. Le testeur peut se passer de cette partie s'il estime avoir déjà une cartographie du(des) réseau(x) cible(s).

Prérequis / entrées (inputs): L'attaquant doit se trouver dans la zone de couverture du réseau WI-FI.

Outils:

Pour mener à bien cette étape, les logiciels suivants requis sont Vistumbler ou Nmap, KillerBee et Hcitool

Procédure:

Le testeur va identifier les différents terminaux et points d'accès de son environnement cible à l'aide des outils précités et selon les protocoles utilisés sur le réseau: Vistumbler ou Nmap, KillerBee et Hcitool respectivement pour la découverte WI-FI, Zigbee et BLE.

Sorties (outputs): Une cartographie du réseau cible avec des informations telles que les protocoles de chiffrement...

2. Ports et services Actifs

Objectifs: Comme le titre l'indique, il est question pour le testeur d'identifier les ports et services actifs qu'il va cibler pour ses attaques. Appliquer la procédure décrite sur chaque équipement cible.

Prérequis / entrées (inputs):

L'attaquant doit être connecté au réseau et l'équipement ciblé pour le scan doit fonctionner avec le protocole TCP/IP. Par exemple les caméras IP. L'étape précédente est donc nécessaire pour conclure de la faisabilité de cette phase.

Outils: Nmap est l'un des outils par excellence pour mener cette attaque.

Procédure:

Plusieurs formes de scan sont faisables. Toutefois, le scan SYN est plus répandu et se veut discret en raison du fait qu'il n'achève pas le processus de TCP handshake (connexion non établie).

Sorties (outputs): Une liste des ports ouverts

II. Identification des vulnérabilités et modélisation des attaques

Comme le titre l'indique, il sera question ici de découvrir les vulnérabilités des équipements, puis de s'en servir pour créer un scénario d'attaques ou simplement décider de quelles attaques mener.

1. Scan de vulnérabilités

Objectifs: Scanner chaque système du réseau et en identifier des vulnérabilités exploitables.

Prérequis / entrées (inputs):

L'attaquant doit être connecté au réseau et l'équipement ciblé pour le scan doit fonctionner avec le protocole TCP/IP.

Procédure: On pourrait se servir de OpenVAS ou de NMAP pour recueillir des vulnérabilités CVE.

Sorties (outputs): une liste de vulnérabilités connues.

2. Choix des attaques à mener / tests à effectuer

Objectifs: Sélectionner des attaques à mener sur le réseau

Prérequis / entrées (inputs):

Pour cette phase est recommandée la pile d'informations recueillies lors des phases précédentes et La liste des vulnérabilités découvertes (voir collecte d'informations et de scan de vulnérabilité). Les attaquants non connectés au réseau se passeront de ces prérequis.

Outils: le testeur peut utiliser la check-list des attaques fourni en annexe.

Procédure

Le testeur n'aura qu'à y recenser les attaques qu'il pourra mener au vu:

Des informations collectées et des vulnérabilités découvertes lors des phases précédentes, de sa position (vue sur la cible) et celles prérequises par les attaques.

Par exemple, après avoir recueilli plus haut l'information selon laquelle nous avons la présence d'un réseau zigbee, il pourra s'intéresser aux tests de la sous-catégorie "*Attaques Zigbee*" de la check-list.

(optionnel) le testeur peut aussi procéder à une modélisation de menaces et en déduire les attaques à mener pour exploiter les vulnérabilités relatives à ces menaces. La méthodologie propose l'usage de la méthodologie STRIDE pour ce faire.

Sorties (outputs): Une liste d'attaques/tests à mener

III. Exploitation

Au cours de cette étape, le testeur effectue les attaques pour mettre les vulnérabilités du système à nu. La section détails de façon précise comment s'y prendre pour mener à bien chacun des tests.

Les attaques mentionnées ici ne sont bien évidemment pas toutes menées obligatoirement: En effet lors de la phase précédente (*Choix des attaques à mener / tests à effectuer*), le testeur a répertorié les attaques à mener et les failles à exploiter.

Par ailleurs, il faut noter que les tests/attaques de la phase *Exploitation* ne sont pas forcément lié(e)s les un(e)s aux autres. L'objectif étant de compromettre les équipements IoT, cette section ne fait que proposer des attaques pour y parvenir et guider leur mise en œuvre.

Enfin les attaques de cette rubrique ne sont pas mentionnées de façon exhaustive. Nous avons évoqué quelques-uns des nombreux exploits que la méthodologie décrit et propose au testeur pour compromettre sa cible.

1. Exploitation WI-FI

1.1 Sniffing du trafic WI-FI

Objectif: Sniffer une communication WI-FI non cryptée à la recherche de données sensibles (mot de passe...).

Prérequis / entrées (inputs): L'attaquant doit se trouver dans la zone de couverture du réseau WI-FI.

Outils:

Comme Matériels Physiques, la méthodologie propose au testeur un adaptateur WI-FI compatible avec la suite aircrack. Ceci pour l'attaquant à proximité du réseau et n'étant pas connecté, et dont la carte sans-fil ne supporte pas le *monitor mode*.

Pour ce qui est des Logiciels, le testeur pourra recourir à la suite Aircrack (Pour l'attaquant n'étant pas connecté) et Wireshark.

Procédure:

L'attaquant capture le trafic puis l'analyse avec Wireshark. Wireshark peut suffir pour les 2 opérations citées, mais avec airodump (inclus dans la suite aircrack), on peut faire une meilleure analyse des points d'accès et terminaux aux alentours.

Sorties (outputs): Des informations sensibles telles que des mots de passe en clair dans le cas d'un trafic non chiffré.

1.2 craquage de Mot de passe via attaque par dictionnaire

Objectifs: S'introduire dans le réseau (se connecter au point d'accès WI-FI) en tentant plusieurs combinaisons de mots possibles en guise de mots de passe.

Prérequis / entrées (inputs):

L'attaquant doit se trouver dans la zone de couverture du WI-FI et (logiquement) pas connecté.

Outils:

La méthodologie propose comme Matériels Physiques un adaptateur WI-FI (si la carte réseau du testeur ne supporte pas le mode moniteur). Comme Logiciels, elle recommande la suite aircrack.

Procédure:

L'attaquant doit suivre la même procédure de l'attaque Sniffing du trafic WI-FI en se servant de airodump de la suite aircrack (s'arrêter juste avant l'analyse avec Wireshark). Ensuite il pourra lancer l'attaque proprement dite avec aircrack-ng.

Sorties (outputs): Le mot de passe du point d'accès sans fil.

1.3. Attaques Conjointes du DOS sur DHCP, du Serveur Rogue DHCP et du MITM

Objectifs: La finalité ici est faire du MITM

Prérequis / entrées (inputs): L'attaquant doit être connecté au réseau WI-FI.

Outils: Seul le logiciel yersinia est requis (du moins recommandé)

Procédure:

L'attaquant se sert du logiciel requis pour mener successivement: d'abord un DOS de DHCP qui consiste à remplir la plage d'adresse IP avec de fausse adresse pour empêcher les autres terminaux de se voir allouer des adresses par le routeur légitime. Ensuite une attaque de Serveur Rogue DHCP (se fait passer pour un serveur DHCP pour inciter les autres terminaux à se connecter. Pour enfin pouvoir faire un MITM.

Sorties (outputs): Une position dans le réseau (MITM) permettant au testeur de sniffer le trafic et ainsi tenter d'obtenir des données sensibles comme des mots de passe.

2. Exploitation Zigbee

2.1 Sniffing de trafic Zigbee

Objectifs: Capturer le contenu d'un trafic Zigbee et y chercher des informations sensibles

Prérequis / entrées (inputs): L'attaquant doit être dans la zone de couverture du réseau ZIGBEE.

Outils:

La méthodologie propose comme Matériels Physiques: une Clé USB Atmel RzRaven (pré-flashé avec le Firmware Killerbee), et comme logiciel KillerBee.

Procédure:

L'attaquant détermine les hôtes Zigbee aux alentours, détermine le canal de communication et lance le sniffeur. Il pourra ensuite analyser le fichier .pcap et tenter de trouver des données sensibles.

Sorties (outputs): Des données sensibles telles que des mots de passe.

2.2 Attaque Replay

Objectifs: Capturer des paquets d'un trafic et les retransmettre à des fins usurpatoires.

Prérequis / entrées (inputs): l'Équipement devra communiquer par le protocole Zigbee (Avec la fonctionnalité CRC désactivée)

Outils: le Logiciel Attify KillerBee Framework (logiciel disponible ici) est recommandé.

Procédure: Après avoir déterminé le canal de communication de la cible, l'attaquant Capture les paquets puis Lance l'attaque replay proprement dit.

Sorties (outputs): Opérations effectuées sans avoir eu besoin d'authentification / autorisation préalable.

3. Exploitation BlueTooth/BLE

3.1 Sniffing

Objectifs: Capturer des paquets d'un trafic BLE, en recueillir des informations et si possible écrire des requêtes malicieuses au vu des informations obtenues après cette capture.

Prérequis / entrées (inputs): L'attaquant doit être dans la zone de couverture du réseau.

Outils: Les logiciels requis ici sont: Wireshark, Hcitool, Ubertooth One, Gatttool

Procédure:

L'attaquant commence par identifier l'adresse BlueTooth de l'équipement cible (avec Hcitool). Il va ensuite Capturer le trafic (en se servant de Ubertooth One) et Analyser le fichier de capture (avec Wireshark). Optionnellement, il pourra modifier les valeurs des caractéristiques (référencés par les handles) manuellement et tenter de soumettre des requêtes (avec Gatttool).

Sorties (outputs): Des données sensibles telles que des mots de passe.

3.2 Attaque Replay

Objectifs: Capturer les paquets puis les retransmettre (en les modifiant si possible) pour générer des requêtes d'écriture et de lecture.

Prérequis / entrées (inputs): L'attaquant doit être connecté au réseau

Outils: BTLEJuice est le logiciel proposé par la méthodologie pour mener cette attaque.

Procédure:L'attaquant capture les paquets avec BTLEJuice puis rejoue les paquets.

Sorties (outputs): Opérations effectuées sans avoir eu besoin d'authentification / autorisation préalable.

4. Attaques sur VOIP

Les tests/attaques de cette rubrique sont liés et nécessitent d'être menés dans l'ordre proposé.

4.1. Reconnaissance

Objectifs: Recueillir les informations pouvant aider à mener les attaques VOIP.

Prérequis / entrées (inputs): L'attaquant doit être connecté au réseau.

Outils: Comme logiciels, la méthodologie suggère SVMAP et SVNWAR.

Procédure:

Avec SVMAP, l'attaquant lance la découverte (des clients SIP, des PBX, ...), puis passe à l'énumération des numéros d'extensions avec SVNWAR.

Sorties (outputs): Des informations nécessaires pour la suite des tests, notamment les clients SIP, les types et versions des PBX, les numéros d'extensions.

4.2. Interception des flux VOIP/ relecture des conversations

Objectifs: Capturer les paquets puis les retransmettre (en les modifiant si possible) pour générer des requêtes d'écriture et de lecture.

Prérequis / entrées (inputs): L'attaquant se trouve dans la zone de couverture du réseau

Outils: Voir Sniffing du trafic WI-FI

Procédure: L'attaquant Capture le trafic puis l'analyse avec Wireshark.

Sorties (outputs): Des données sensibles telles que des mots de passe, ou des conversations sous forme audio.

4.3. Craquage de Mot de passe SIP

Objectif: Le testeur devra obtenir les mots de passe pour l'authentification SIP (Session Initiation Protocol).

Prérequis / entrées (inputs): L'attaquant: Dans la zone de couverture du réseau.

Outils: Comme Matériel physique, un adaptateur WI-FI est requis (pour les cartes ne supportant pas le mode monitor) et comme logiciels, la suite SIPcrack.

Procédure: Capturer les paquets VOIP avec SIPdump (inclus dans la suite SIPcrack) puis lancer l'attaque avec SIPcrack.

Sorties (outputs): Mots de passe de compte SIP

B. test de la Couche d'application

Se référer à la méthodologie ATHENA pour les tests d'intrusion WEB et Mobiles.

Nous n'avons pas travaillé sur les tests relatifs à cette couche car ATHENA dispose déjà d'une méthodologie bien élaborée à cet effet.

C: Test de la couche Firmware

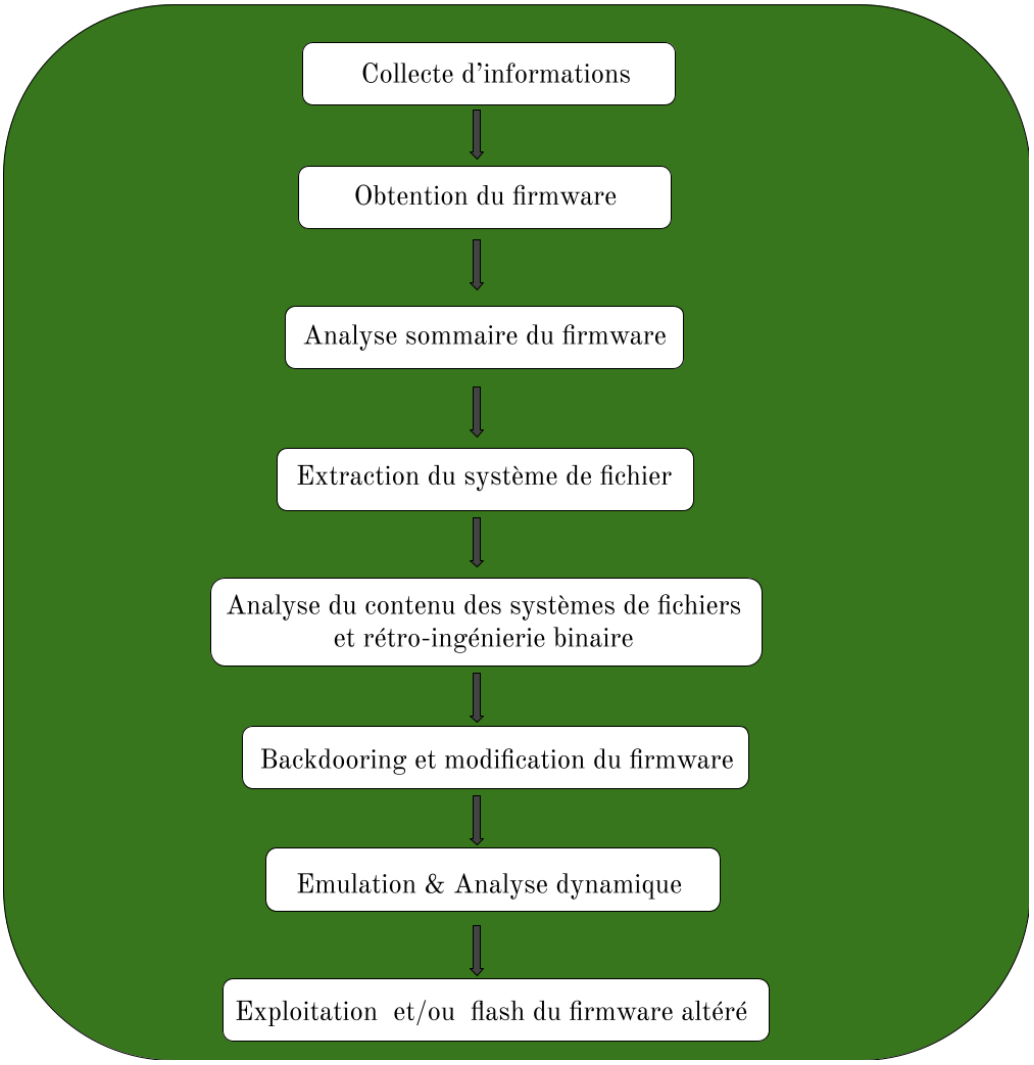


Figure 16: *La méthodologie pour la couche firmware*

Description: cette étape consiste à s'intéresser au micrologiciel/firmware de l'équipement IoT. Le but est de l'obtenir, l'étudier pour en découvrir des vulnérabilités à exploiter. Par ailleurs, le testeur aura à le modifier et/ou y incruster un/des backdoors.

À la fin de tout ce processus, le testeur pourra soit:

- Attaquer le IoT cible au vu des vulnérabilités découvert dans le firmware qu'il a étudié (exploitation)
- Flasher le firmware altéré (modifié, incrusté de backdoors) dans l'équipement cible
- Opter pour les deux options précitées l'une après l'autre.

I. Collecte d'informations

Le testeur acquiert tous les détails techniques et de documentation relatifs au micrologiciel (firmware) de l'appareil cible. Le testeur aura à se focaliser sur:

- Architecture(s) de processeur prise en charge (ARM, MIPS...)
- Plateforme de système d'exploitation (Debian, ...)
- Configurations du Bootloader
- Schémas du matériel
- Fiche technique
- Emplacement du code source (pour les open source)
- Identifiants FCC
- Vulnérabilités connues.

Prérequis / entrées (inputs): Aucun

Outils: La méthodologie propose Internet et la(les) Fiche(s) technique(s)

Procédure: Se servir des outils cités pour recueillir les informations citées en objectifs.

Sorties (Outputs): une pile d'informations qui permettra au testeur d'affiner ses tests/attaques. Par exemple, connaître l'architecture de Processeur sera primordial pour pouvoir suivre l'étape d'émulation et analyse dynamique du firmware.

II. Obtention/Extraction du firmware

Objectifs: Obtenir le firmware en utilisant une ou l'ensemble des 2 méthodes proposées ci-dessous.

Prérequis / entrées (inputs):

L'attaquant a le choix entre plusieurs options pour se procurer le firmware. Notamment par téléchargement et par extraction via les ports de débogage de l'équipement. Pour la seconde méthode(Extraction via les ports de débogage), voir *D. Tests couches Physiques (2.1, 3.3, 4.1)* rubriques *prérequis / entrées (inputs)*.

Outils:

Un navigateur et une connexion internet pour le téléchargement. Pour l'extraction, voir *D. Tests couches Physiques (2.1, 3.3, 4.1)* rubriques *outils*

Procédure:

La méthode du téléchargement du firmware est laissée au soin du testeur. Comme pistes nous proposons: le site de l'équipementier ou les plateformes de dépôt de code source tel que github (cas des firmwares open source)

Pour ce qui est de la seconde méthode: voir *D. Tests couches Physiques (2.1, 3.3, 4.1)* rubriques *procédures*.

Sorties (outputs): Le firmware téléchargé ou extrait de l'équipement. Ce dernier est requis pour effectuer la suite des tests de la méthodologie.

III. Analyse sommaire du firmware

Objectifs: Obtenir quelques caractéristiques du firmware:

Prérequis: Avoir obtenu le firmware

Outils: binwalk et Système Linux sont recommandés.

Procédure: Le testeur n'aura qu'à recourir aux utilitaires file, strings, fdisk et à l'outil binwalk pour obtenir des informations sur le fichier téléchargé.

Sorties (outputs): des informations d'ordre général sur le firmware.

IV. Extraction du système de fichier

Objectifs: comme indiqué en intitulé, on veut extraire les fichiers (pour pouvoir les analyser ultérieurement)

Prérequis / entrées (inputs): Avoir obtenu le firmware

Outils: Binwalk ou autres utilitaires d'extraction de système de fichiers, et un système Linux.

Procédure:

Se servir de Binwalk ou des utilitaires d'extraction spécifiques aux systèmes de fichier (comme unyaffs pour les systèmes de fichier yaffs).

Sorties (outputs): Le système de fichier extrait, c'est-à-dire l'ensemble des fichiers et binaires qui constituent le firmware. C'est l'ensemble de ces fichiers que nous allons manipuler, analyser et modifier pour en découvrir/insérer des vulnérabilités et backdoors.

V. Analyse du contenu des systèmes de fichiers et rétro-ingénierie binaire

Objectifs: Analyser statiquement les fichiers de configuration du système de fichiers ainsi que les binaires pour y repérer des données sensibles ou des vulnérabilités.

Prérequis / entrées (inputs): Avoir obtenu le firmware et en avoir extrait le système de fichier.

Outils:

- Firmwalker
- Hopper
- Système Linux

Procédure:

La procédure laisse au testeur le choix entre une analyse statique manuelle (en ayant recours aux utilitaires tels que grep, cat, find pour repérer des fichiers et contenu sensibles.), et une autre automatisée (avec Firmwalker). Ou encore une combinaison des 2. Après quoi le testeur aborde la rétro-ingénierie (avec Hopper) de quelques binaires qu'il aura jugés pertinents.

Sorties (outputs): A la fin de cette étape on obtient des vulnérabilités et des données sensibles telles que des urls, des identifiants codés en dur, des clefs, des certificats, les routines de déchiffrement du firmware ou même des backdoors installés à des fins de maintenance.

VI. Implantation de backdoors /modification du firmware

Objectifs: Modifier le firmware, y insérer un/des backdoors.

Prérequis / entrées (inputs): Avoir obtenu le firmware et extrait le système de fichier.

Outils:

La méthodologie requiert pour cette étape les outils suivants: FMK (firmware-mod-kit), Buildroot, et Un script qui servira de backdoor. Ici un exemple de script donnant au testeur un shell distant via le port 9999.

Procédure:

On commence par compiler le script du backdoor pour l'architecture du firmware (en se servant de Buildroot). Ensuite on insère le script compilé dans le firmware. On peut procéder à d'éventuelles modifications des fichiers pour y inclure d'autres formes de vulnérabilités. Enfin on crée un nouveau firmware à partir du système de fichier altéré (avec FMK).

Sorties (outputs): On obtient un firmware incrusté de backdoor(s) et altéré de sorte à être vulnérable.

VII. Emulation & Analyse dynamique (facultatif)

Objectifs: Tester les effets des modifications que nous avons effectuées, l'accessibilité de notre (nos) backdoor(s) et l'exploitabilité des vulnérabilités jusque-là découvertes. Le testeur peut passer directement à la phase suivante s'il trouve effectif les modifications opérées dans la phase précédente.

Prérequis / entrées (inputs):

Avoir obtenu le firmware et effectué les modifications de l'étape précédente et être connecté au réseau de l'IoT cible (pour accéder au shell distant initié par notre backdoor).

Outils:

Pour l'émulation on aura besoin de FAT (Firmware Analysis Toolkit) disponible ici et un système UBUNTU de préférence.

Procédure:

Après avoir installé et configuré FAT, le testeur peut exploiter/tester les vulnérabilités découvertes préalablement (Analyse dynamique). Il peut aussi accéder au shell distant initié par notre backdoor au démarrage du firmware.

Sorties (outputs): Des vulnérabilités / backdoors / modifications validées (jugées et confirmées exploitables ou effectives).

VIII. Exploitation et/ou flash du firmware altéré

Objectifs: Attaquer l'équipement physique (et non le firmware émulé comme précédemment) au vu des vulnérabilités découvertes lors des étapes précédentes (analyses statique et dynamique). Le testeur aura à flasher le firmware modifié et exploiter les vulnérabilités et modifications qu'il y aura insérées.

Prérequis / entrées (inputs):

Pour le flash du firmware altéré le premier prérequis est d'avoir suivi la phase Implantation de backdoor /modification du firmware. Le reste des prérequis est mentionné dans les exploits écrire un nouveau firmware via des ports JTAG, et écrire un nouveau firmware dans une mémoire SPI.

Pour l'exploitation, aucun prérequis n'est exigé (mais avoir suivi les phases d'analyse statique et dynamique pour identifier les vulnérabilités est recommandé).

Outils:

Pour le flash du firmware altéré, voir les exploits écrire un nouveau firmware via des ports JTAG, et écrire un nouveau firmware dans une mémoire SPI.

Pour l'exploitation, tout dépendra des vulnérabilités découvertes

Procédure: Le testeur commence par flasher (écrire en mémoire) le firmware altéré puis passe à Exploitation.

Sorties (outputs): Un équipement animé par un firmware truffé de vulnérabilités et de portes dérobées, prêtes à être exploitées (ou déjà exploitées).

D. Test de la couche physique

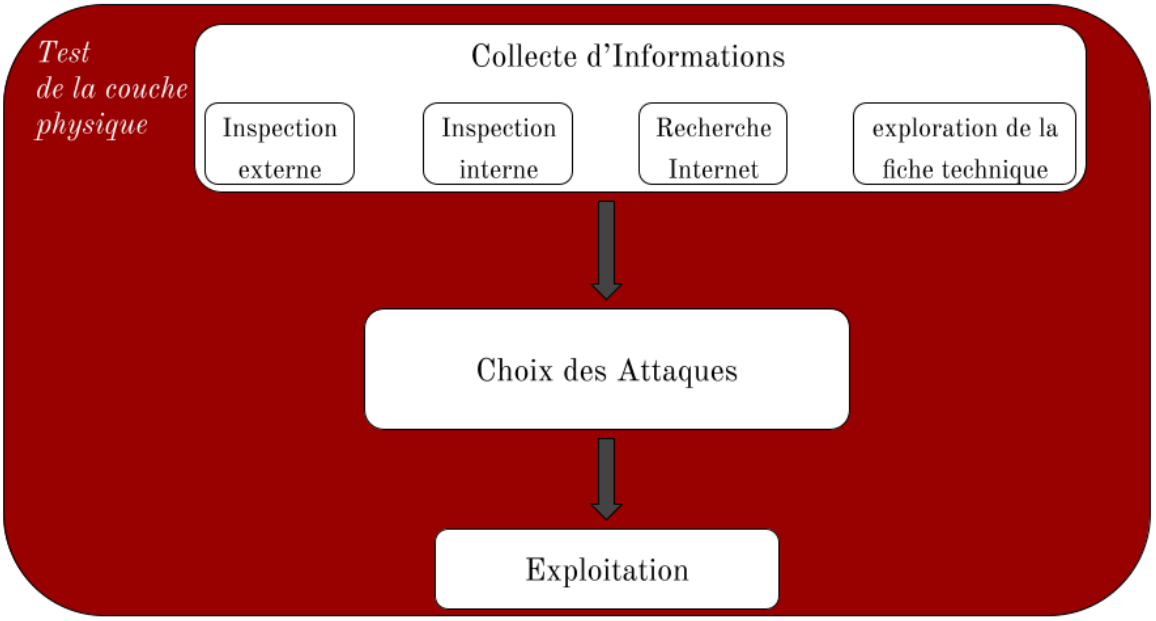


Figure 17: *La méthodologie pour la couche physique*

Description: À ce niveau il est comme le nom l'indique, question de tester la robustesse des équipements vis-à-vis des attaques physiques.

Cette phase s'applique pour chaque équipement IoT cible de l'environnement à tester. Elle nécessite une interaction physique entre le testeur et les équipements cibles.

I. Collecte d'Informations

Le testeur recueille des informations pertinentes sur la couche physique de l'équipement à évaluer en se focalisant sur les éléments suivants:

- Les actifs (pertinents dans le cadre d'une attaque)
  - Ports: JTAG, USB , UART, ...
  - Supports de stockage: Flash, EEPROM...
  - Puces (Chips/SoC): l'architecture utilisée (ARM ?, MIPS ?...), le modèle/la marque...
  - Antennes
  - etc
- Les technologies sous-jacentes
  - les technologies de communication employées via les bus (SPI, I²C...)
- Les versions et vulnérabilités connues
  - Les versions des actifs de la cible et les versions des modèles récents
  - Les vulnérabilités associées connues
- Les informations techniques de l'équipement
  - Puissances et voltages requis

1. Inspection externe

Objectifs: Examiner l'équipement sans avoir besoin de le démonter. Le testeur repère et note exclusivement ce qui se perçoit à première vue sur l'équipement.

Prérequis / entrées (inputs): L'attaquant devra avoir un accès physique à l'équipement.

Outils: Aucun

Procédure:

Les éléments pertinents à noter sont ceux évoqués ci-dessus et que le testeur peut obtenir via un simple examen externe de l'équipement. Particulièrement le testeur aura à relever: Les ports et interfaces externes, la présence d'antennes, les voltages et autres informations comme la FCC ID inscrite souvent au dos de l'équipement.

Sorties (outputs): Une pile d'informations notamment ceux mentionnés en descriptif de la phase de Collecte d'informations ci-dessus.

2. Inspection interne

Objectif: Démonter l'équipement pour en scruter l'intérieur.

Prérequis / entrées (inputs): L'attaquant devra avoir un accès physique à l'équipement.

Outils: De quoi démonter (tournevis de façon générale)

Procédure:

Le testeur devra s'intéresser comme dans le cas précédent aux éléments qui figurent en objectifs plus haut notamment: Les ports et interfaces de communication de la carte mère et les puces et supports de stockage, leurs modèles et architectures

Sorties (outputs): Une pile d'informations notamment ceux mentionnés en descriptif de la phase de Collecte d'informations ci-dessus.

3. Recherches à travers internet

Objectif: À l'aide d'un moteur de recherche, recueillir des informations pertinentes pour un test d'intrusion.

Prérequis / entrées (inputs): le nom/la série/ la version ou le FCC ID du modèle de l'équipement

Outils: Un moteur de recherche et Internet suffisent pour cette étape.

Procédure:

Lancer une recherche sur l'équipement avec le nom/la série/ la version ou le FCC ID. Recenser particulièrement les versions des composants listés, les versions récentes des composants et les vulnérabilités associées connues.

Sorties (outputs): Une pile d'informations notamment ceux mentionnés en descriptif de la phase de Collecte d'informations ci-dessus.

4. Révision de la fiche technique

La fiche technique peut elle aussi fournir des informations précieuses comme les voltages fonctionnels.

En résumé, le testeur devra recueillir une pile d'informations pertinentes en s'aidant des éléments mentionnés en objectifs.

II. Choix des Attaques

Objectifs: S'inspirer de la check-list pour sélectionner des attaques à mener

Prérequis / entrées (inputs): Il est recommandé d'avoir mené la phase précédente de Collecte d'informations (et donc recueilli une pile d'information sur l'équipement).

Outils: La check-list des tests/attaques

Procédure:

En fonction des informations recueillies dans la phase précédente, le testeur aura à sélectionner les tests adaptés. Ainsi ses choix seront fonction des informations collectées lors de la phase précédente et de sa position (vue sur la cible) et celles pré-requises par les attaques.

Par exemple, après avoir recueilli plus haut l'information selon laquelle la carte mère dispose de Ports JTAG, il pourra s'intéresser aux tests de la sous-catégorie "*Attaques JTAG*" de la check-list.

Optionnellement le testeur peut aussi procéder à une modélisation de menaces et en déduire les attaques à mener pour exploiter les vulnérabilités relatives à ces menaces.

La méthodologie propose l'usage de la méthodologie STRIDE pour ce faire.

Sorties (outputs): Une liste de tests/attaques à mener pour compromettre les équipements IoT

III. Exploitation

Les tests/attaques de la phase *Exploitation* ne sont pas forcément lié(e)s les un(e)s aux autres.

1. Exploitation UART

1.1 obtention de shell root sur l'équipement à travers une connexion UART

Objectifs: Obtention de shell root sur l'équipement à travers une connexion UART. Explorer l'équipement à la recherche de données sensibles, tenter des modifications de configurations et consort.

Prérequis / entrées (inputs):

L'attaquant aura besoin d'un accès physique à l'équipement et l'équipement devra bien évidemment disposer de ports UART sur la carte mère.

Outils:

Pour ce qui est des outils matériels, le testeur aura besoin des éléments suivants: un multimètre, le gadget Attify badge et des fils de raccordement

Les logiciels requis sont les suivants: le script Baudrate.py (disponible ici), Minicom ou screen, et le driver de Attify Badge si l'on utilise windows (Disponible ici).

Procédure:

Le testeur devra commencer par identifier les broches UART (à l'aide du multimètre), après quoi il connectera *Attify Badge* au Laptop puis à l'équipement (à l'aide des câbles de raccordement).

Ensuite il identifiera la vitesse de transmission (en exécutant le script) pour pouvoir enfin interagir avec l'équipement et obtenir le shell.

Sorties (outputs): Un shell sur l'équipement via le port UART.

2. Exploitation SPI

2.1 Lecture et écriture à travers la mémoire EEPROM SPI

Objectifs: copier tout le contenu de la mémoire EEPROM (dumper la mémoire) et chercher du contenu sensible, puis écrire un autre contenu sur ladite mémoire.

Prérequis / entrées (inputs):

L'attaquant aura besoin d'un accès physique à l'équipement

L'équipement devra disposer d'une Mémoire EEPROM SPI (normalement vérifiée lors de la collecte d'information).

Outils:

Pour cette phase le testeur utilisera comme matériels physiques: une Pince SOIC (qui servira de support pour la puce mémoire), le gadget Attify Badge et des fils de raccordement.

Il aura aussi besoin d'un script: spiflash.py

Procédure:

Après avoir identifié individuellement les broches SPI de la puce EEPROM, Le testeur va connecter le gadget Attify Badge à la puce EEPROM puis au laptop (pour interagir avec la puce EEPROM).

Ensuite il lance le script de lecture. Après quoi il pourra s'atteler à chercher des données sensibles dans le contenu extrait ou même écrire du nouveau contenu dans la mémoire à l'aide du même script.

Sorties (outputs): S'il s'arrête à l'étape D (de l'annexe des procédés des tests), le testeur achève cette attaque avec le contenu de la mémoire. S'il continue jusqu'à l'étape F, le testeur obtient un équipement animé par un firmware vulnérable.

3. Exploitation JTAG

3.1 Écrire un nouveau firmware

Objectifs: Remplacer le firmware d'origine de l'appareil par un autre qui est par exemple obsolète (fonctionnalités de sécurité moins robustes), modifié...

Prérequis / entrées (inputs):

L'attaquant doit avoir un accès physique à l'appareil et l'équipement devra bien évidemment disposer de ports JTAG sur la carte mère

Outils:

Pour cette phase le testeur utilisera comme matériels physiques JTAGulator et Attify Badge. Les logiciels dont il aura besoin sont OpenOCD, Attify Badge Tools. Il aura également besoin d'un firmware obsolète ou altéré de sorte à être vulnérable.

Procédure:

Le testeur commence par identifier individuellement les broches JTAG (a l'aide de JTAGulator). Ensuite il connecte l'appareil cible aux outils de débogage (Attify badge & Attify badge tools et OpenOCD). Après quoi il pourra flasher le nouveau Firmware.

Sorties (outputs): A la fin de cette étape, le testeur obtient un équipement animé par un firmware vulnérable.

3.2 Lire les données sensibles du firmware

Objectifs: Trouver des données sensibles en temps réel (clés, explorer la valeur des registres en temps réel, fonctions, jeu d'instructions, etc.).

Prérequis / entrées (inputs):

L'attaquant doit avoir un accès physique à l'appareil et avoir extrait ou obtenu le binaire à tester(voir test couche Physique, Exploitation JTAG, 3.3)

Outils:

Pour cette phase, le testeur utilisera comme matériel physique Attify Badge et comme logiciels OpenOCD, GDB-MultiArch et Attify Badge Tools.

Procédure:

Le testeur commence par identifier les broches JTAG cibles pour pouvoir connecter OpenOCD à la cible (avec JTAGulator). Ensuite il exécute le firmware avec *GDB-multiarch*.

Il identifie quelques fonctions stratégiques (fonctions de vérification de mot de passe et de clé...) et analyse leur code assembleur pour localiser une (des) instruction(s) assembleur critique(s) et un (des) emplacement(s) mémoire(s).

Toujours en se servant de *GDB-multiarch le testeur* analyse les registres et lit leur contenu.

Sorties (outputs): des valeurs sensibles commes des mots de passe codés en dur

3.3 Extraire le firmware

Objectifs: Extraire le firmware depuis l'équipement via les ports de communication.

Prérequis / entrées (inputs):

L'attaquant doit avoir un accès physique à l'appareil et avoir suivi les étapes A et B de *Test de la couche physique*, *Exploitation JTAG*, *3.1* qui visent à connecter l'équipement cible aux outils de débogage

Outils:

La méthodologie recommande JTAGulator et Attify Badge pour ce qui est des outils physiques. Les logiciels requis sont OpenOCD, Attify Badge Tools.

Procédure:

Toujours en étant connecté via telnet a OpenOCD Le testeur vérifie l'adresse mémoire à laquelle la mémoire flash démarre et lance proprement l'extraction du firmware.

Sorties (outputs): Le firmware extrait de l'équipement.

4. Exploitation I²C

4.1 Lecture de contenu de la mémoire EEPROM I²C

Objectifs: copier tout le contenu de la mémoire EEPROM (dumper la mémoire) et chercher du contenu sensible.

Prérequis / entrées (inputs):

L'attaquant doit avoir un accès physique à l'équipement

Par ailleurs l'équipement devra disposer d'une Mémoire EEPROM I²C (vérifiable via la fiche technique et/ou un moteur de recherche en se servant du nom du modèle de la mémoire pour lancer une recherche).

Outils:

La méthodologie recommande une pince SOIC (qui servira de support à la puce mémoire), des fils de raccordement et Attify Badge pour ce qui est des outils physiques. Le testeur aura également besoin du script i2ceeprom.py (disponible ici).

Procédure:

Après avoir identifié individuellement les broches I²C de la puce EEPROM, le testeur va connecter la puce EEPROM au matériel Attify Badge et connecter Attify Badge au laptop (pour interagir avec la puce EEPROM).

Après quoi il pourra lancer le script de lecture et chercher des données sensibles dans le contenu lu.

Sorties (outputs): Le contenu de la mémoire et quelques données sensibles.

5. Autres attaques (construites à partir des menaces modélisées, ou spécifiques à des équipements particuliers)

5.1 Le déblocage des modems 3G/4G

Objectifs: Débloquer le modem, c'est-à-dire le rendre utilisable avec des SIM d'opérateurs autres que celui pour lequel il a été initialement configuré.

Prérequis / entrées (inputs):

L'attaquant doit avoir un accès physique à l'équipement et le test est applicable à des modems Huawei de préférence.

Outils:

La méthodologie recommande comme outils physiques un câble quelconque servant à court-circuiter, un câble USB-to-USB et un tournevis pour le démontage.

Pour ce qui est des logiciels, la méthodologie exige:

- USBloader
- Balong usb downloader (disponible via ce lien)
- 2 versions du firmware respectivement récente et obsolète
- DC-unlocker
- Huawei WebUI (disponible ici)
- Pilotes du modem à installer (Huawei Hilink, Huawei Mobile Partner)

Procédure:

Après avoir démonté l'équipement, le testeur le fait redémarrer en le court-circuitant (BootShorting) avec le câble.

Ensuite il fait passer l'équipement en mode usine grâce à balong usb downloader et USBloader. Suite à quoi il opère successivement un downgrade, un firmware et un upgrade du firmware.

Après ces étapes le modem est débloqué mais il a perdu son code IMEI (devient 000000000000000). Il faut donc le réparer avec DC-Unlocker

Sorties (outputs): Modem débloqué

5.2 Le déblocage des Routeurs 3G/4G

Objectifs: Débloquer le routeur, c'est-à-dire le rendre utilisable avec des SIM d'opérateurs autres que celui pour lequel il a été initialement configuré.

Prérequis / entrées (inputs):

L'attaquant devra avoir accès physique à l'équipement et mener le test sur des routeurs Huawei de préférence.

Outils:

Pour ce test est requis comme matériel la SIM d'un autre opérateur.

Pour ce qui est des logiciels, sont requis:

- Putty
- Multicast Upgrade Tool
- Huawei ES186 toolbox
- Huawei Unlock Calculator
- Firmware obsolète.

Procédure:

Le testeur commence par insérer une SIM invalide et connecte le routeur à l'ordinateur. Ensuite via un émulateur comme putty il se connecte au modem via telnet. Avec des commandes du module AT (module pour contrôler les modems) réinitialise le modem.

Avec le firmware obsolète, il opère un downgrade du firmware (avec Multicast Upgrade Tool). Ces étapes font ainsi passer le modem en mode usine.

Toujours via telnet et avec les commandes AT, il recueille le hash du code de déblocage au format hexadécimal et se sert de Huawei Unlock Code Calculator pour en déduire le code de déblocage.

Soumettre le code à l'interface de déblocage de l'équipement: l'équipement est ainsi débloqué.

Sorties (outputs): Routeur débloqué.

CHAPITRE III: Validation de la méthodologie (tests)

La méthodologie étant élaborée, la suite logique est la validation de cette dernière. Cela va consister à mener des tests d'intrusion sur une architecture IOT que nous allons mettre en place. Ces tests d'intrusion seront bien évidemment menés en suivant ladite méthodologie.

I. Architecture / environnement de tests

Nous avons une architecture assez simple. Le but est de réaliser des attaques sur l'environnement en déroulant la méthodologie. Elle est constituée des composants suivants: une carte raspberry pi, une DEL, une plaquette d'essai, des fils de connexions, une box internet, et un smartphone.

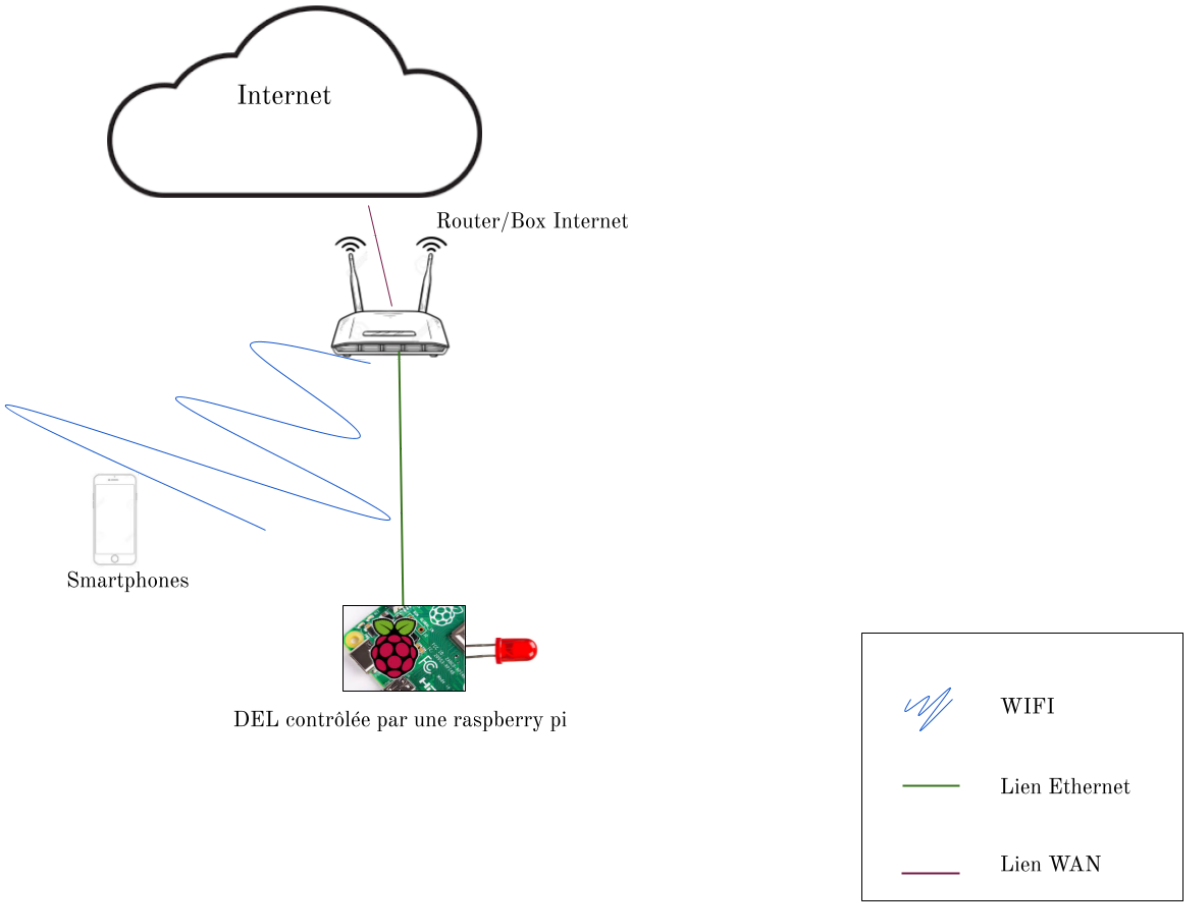


Figure 18: *L'architecture de l'environnement de tests*

1. La carte Raspberry pi



Figure 19: *La carte raspberry pi*

La carte Raspberry est un nano-ordinateur de la taille d'une carte mère. Elle dispose de ports par lesquels interagir avec son l'environnement extérieur, notamment d'autres composants et équipements à contrôler.

Dans notre architecture, notre carte raspberry pi (raspberry pi 3) sera connectée à la box internet. Par ailleurs, elle va abriter un serveur web qui fournira une interface via laquelle le smartphone va manipuler la DEL.

2. La DEL (Diode ÉlectroLuminescente)



Figure 20: *DEL*

L'architecture est constituée d'une DEL, contrôlée par la raspberry pi.

La liaison entre la carte raspberry pi et la DEL est réalisée par une plaquette d'essai et des fils de connexion.

3. Le smartphone

Pour manipuler la DEL, le smartphone va interagir via une connexion WI-FI avec le serveur web abrité par la carte raspberry pi.

4. Montage de l'environnement

Nous avons installé Raspbian ou Raspberry Pi OS comme système d'exploitation sur la Raspberry Pi (voir figure 20 ci-dessous), ainsi que Apache et ses dépendances pour notre serveur web (figure 21).

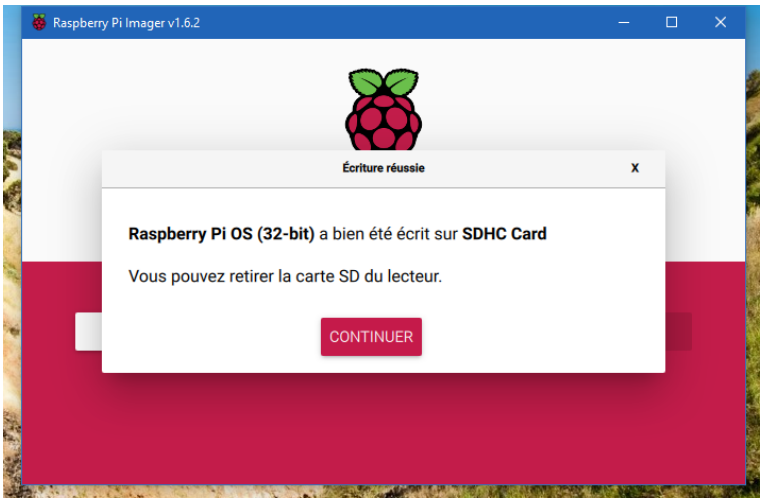


Figure 21: *Raspberry Pi OS installé*

```
pi@raspberrypi:~ $ sudo apt-get install apache2 php libapache2-mod-php
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils libapache2-mod-php7.3 libaprl
  libaprutill libaprutill-dbd-sqlite3 libaprutill-ldap php-common php7.3
  php7.3-cli php7.3-common php7.3-json php7.3-opcache php7.3-readline
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom php-pear
```

Figure 22: *Installation d'apache et de ses dépendances pour mettre en place notre serveur web*

Le code de notre serveur web est disponible en annexe D.

La manipulation de la DEL par la carte Raspberry pi via l'interface web se fait grâce à la librairie Wiringpi. Wiringpi est une librairie de raspberry pi qui permet de manipuler les broches de la carte. Nous y avons fait appel dans notre serveur web (voir annexe D).

```
pi@raspberrypi: ~
pi@raspberrypi:~ $ sudo apt-get install wiringpi
Reading package lists... Done
Building dependency tree
Reading state information... Done
```

Figure 23: *Installation de la librairie wiringpi*

Concrètement nous avons recouru à la library de la façon suivante:

Syntaxe: gpio -g write <*numero_de_la_broche*> <*valeur*> . Le *numéro_de_la_broche* désigne la broche de la carte qui sera liée à la DEL et qui enverra la *valeur* 0 pour l'éteindre et 1 pour l'allumer.

Une fonction comme *system()* ou *shell_exec()* en PHP permet d'exécuter un programme externe, en l'occurrence notre commande ci-dessus.

Appel depuis le code de notre serveur web: On aura donc à utiliser une syntaxe comme celle-ci: system*(*gpio -g write 17 1*)* pour allumer la DEL. (la borne positive de notre DEL est liée à la broche 17 de la carte Raspberry Pi).

Connexion: Nous avons connecté la borne positive de la DEL à la broche 17 (GPIO 17) de la carte Raspberry Pi, puis la borne négative à une résistance. L'autre borne de la résistance sera liée à la masse (une des broches Ground de la Raspberry Pi).

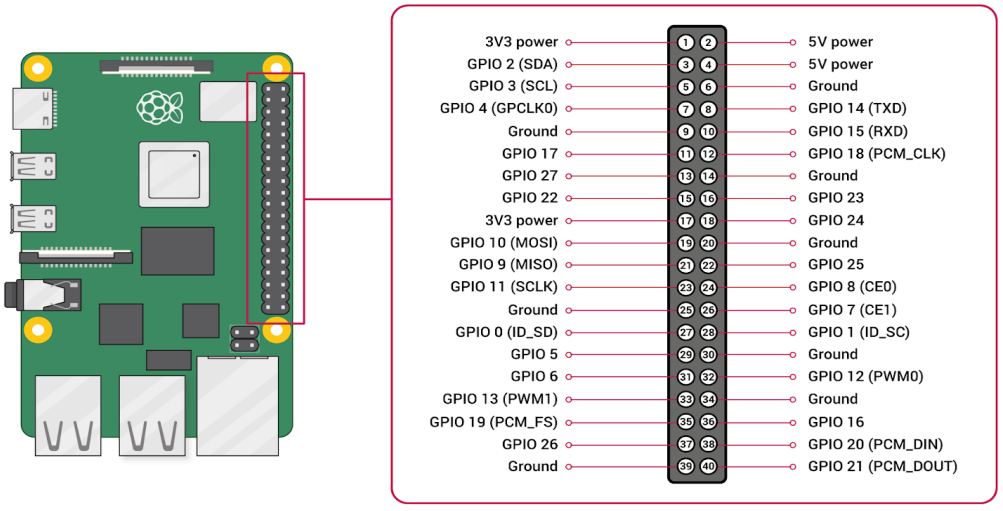


Figure 24: *Les broches de la Raspberry Pi*

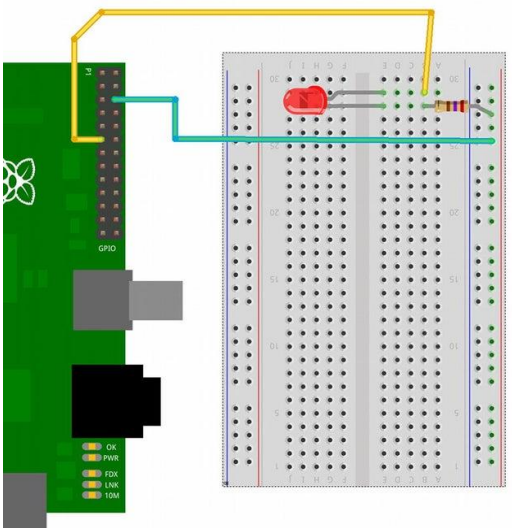


Figure 25: *Schéma du montage*

Une fois la carte raspberry connectée au routeur (par câble ou wifi), on entre son adresse dans un navigateur et on accède à l'interface web.



Figure 26: *Smartphone manipulant la DEL connectée.*

5. Hypothèses de départ

Pour ce test d'intrusion les hypothèses de départ sont les suivantes: le testeur est **hors du réseau**, il ne dispose que de la localisation géographique du réseau cible (il est à proximité, **dans la zone de couverture**).

Pour chaque sous-étape, il dispose des outils et prérequis cités dans la méthodologie. Il n'a pas de cible particulière dans le réseau car n'en n'ayant pas une cartographie préalable.

6. Outils utilisés pour les tests

6.1 La suite aircrack-ng

Il s'agit d'une suite logicielle visant à casser les clés des réseaux WI-FI. Dans cette suite, nous aurons recours à airodump (pour les découvertes de points d'accès et terminaux environnant). Nous aurons également recours à aircrack-ng, casseur de clés portant le même nom que la suite.

Figure 27: *Logo Aircrack-ng*

6.2 Nmap

Nmap est un outil de référence en matière d'audit de sécurité et monitoring réseau. C'est un scanner de ports que nous utiliserons pour détecter les ports actifs des terminaux de notre réseau cible, et aussi pour mener des scans de vulnérabilités.

Figure 28: *Logo Nmap*

6.3 Binwalk

Cet outil permet d'analyser des firmwares, des fichiers binaires, des fichiers compressés, etc, et y déceler/extraire des fichiers et codes exécutables.

Figure 29: *Binwalk*

6.4 Hopper

Hopper est un outil de rétro-ingénierie (reverse engineering). Il permet d'accéder au code assembleur d'un programme. Avec cet outil, il est possible de générer un pseudo code proche de la syntaxe du langage C, à partir du code assembleur.

Figure 30: *Hopper disassembler*

II. Déroulement des tests

Les tests menés ici sont axés sur la couche **réseau** et la couche **firmware**. Les tests de la couche applicative n'ont pas été abordés car ATHENA dispose déjà d'une méthodologie pour les tests d'intrusion web et mobile. Nous n'y avons pas travaillé dans le cadre de ce mémoire. Par ailleurs, par manque d'équipement et matériels (matériels cibles et gadgets de travail), les tests de la couche physique seront abordés ultérieurement et hors du cadre de ce Projet de Fin d'Étude.

A. La couche réseau

Cette section est dédiée aux tests de la couche réseau. Nous rappelons que nous suivons la méthodologie, et que les procédures de chaque étape sont décrites dans l'annexe descriptive des procédures des tests.

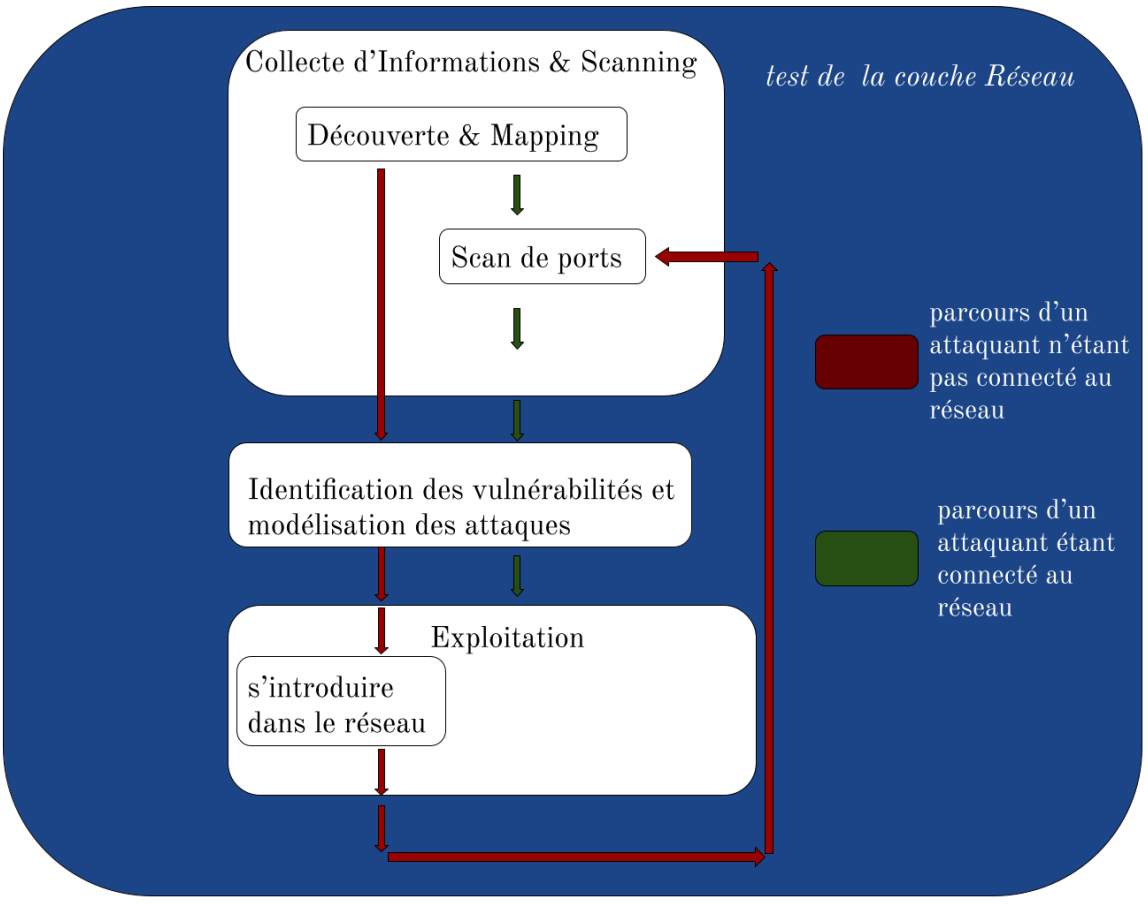


Figure 31: *La méthodologie de pentest réseau à titre de rappel.*

1. Collecte d'informations sur le réseau cible et Scanning

Découverte et mappage: identifions les réseaux à proximité.

Avec l'outil airmon-ng, on passe en mode moniteur (si cette fonction est supportée, le mode moniteur permet à la carte réseau de capturer des paquets ne lui étant pas destinés, et donc de sniffer aux alentours).

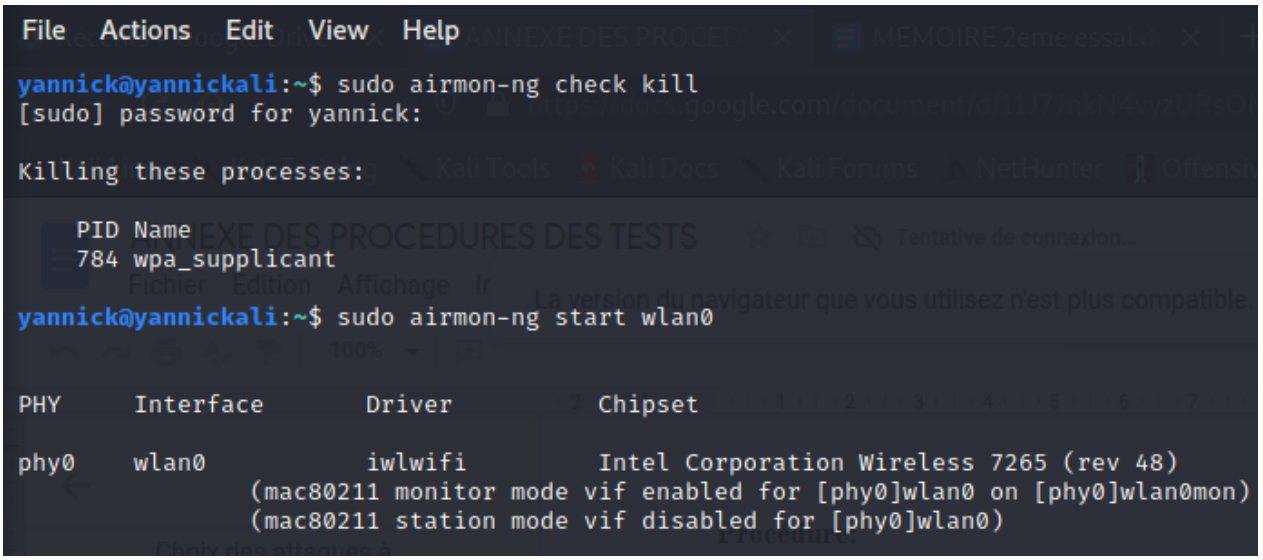


Figure 32: *On interrompt les processus en cours et on passe en mode moniteur.*

Passons à la découverte proprement dite avec l'outil airodump-ng.

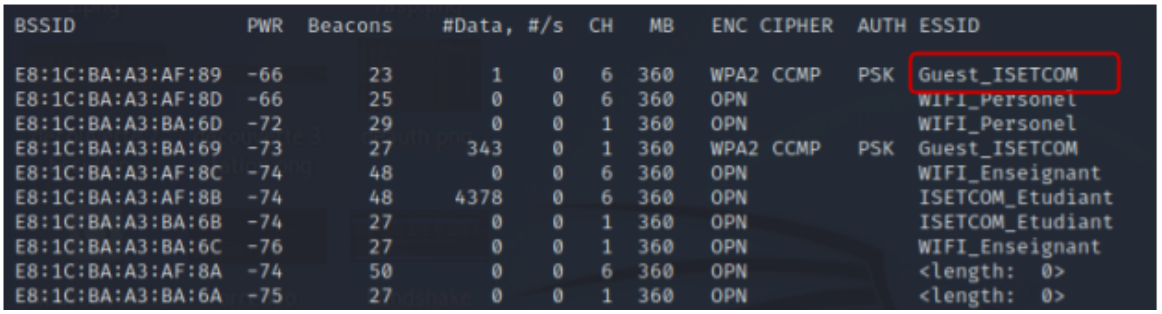


Figure 33: *découverte des points d'accès WI-FI aux alentours.*

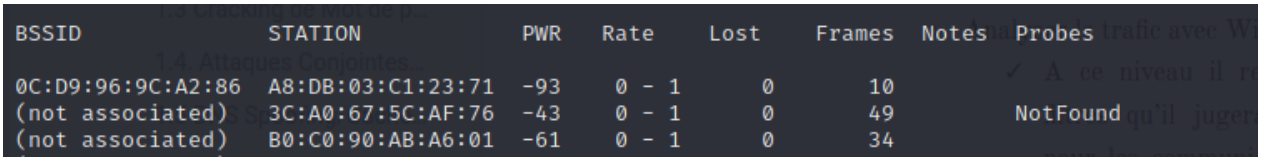


Figure 34: *découvertes des terminaux connectés à chacun de ces points d'accès*

Nous pouvons choisir un des points d'accès et nous intéresser aux terminaux qui lui sont connectés. notamment le point d'accès encadré en rouge (Guest_ISETCOM).

Scan de ports (services actifs)

Comme indiqué dans la méthodologie, nous ne pourrons pas pour l'instant mener cette phase car étant hors du réseau. Pour balayer le réseau, il faut faire un ping sur chaque adresse IP (interne, et donc privée et non accessible directement depuis internet). Par ailleurs, d'une façon générale dans les réseaux de type home network comme le nôtre, ces adresses sont converties en une adresse publique unique via NAT et c'est cette dernière que nous pouvons attaquer éventuellement depuis internet.

2. Identification des vulnérabilités et modélisation des attaques

Scan de vulnérabilités

Nous reviendrons à cette phase une fois que nous nous serons introduits dans le réseau (Pour les mêmes raisons que celles évoquées pour le scan de ports).

Choix des attaques à mener / tests à effectuer

Étant hors du réseau nos choix se limitent aux attaques telles que le sniffing, le craquage de mots de passe, etc, dans le but de s'introduire dans le réseau.

Nous optons donc pour l'attaque par dictionnaire (en vue d'obtenir le mot de passe du point d'accès). Notre point d'accès cible étant déjà découvert, nous allons l'attaquer grâce à l'outil *aircrack-ng*.

Tests	outils
Sniffer une communication WIFI non cryptée à la recherche de données sensibles (mot de passe...)	Logiciels: Wireshark
Cracking de mot de passe via attaque bruteforce	Matériels Physiques: Adaptateur Wifi Logiciels: Wifite, Hashcat
Cracking de mot de passe via attaque par dictionnaire	Matériels Physiques: Adaptateur Wifi Logiciels: la suite Aircrack-ng(Airmon-ng, Airodump-ng, Aireplay-ng, Aircrack-ng), Wireshark
DoS sur DHCP (DHCP Starvation attack) (réseau avec un(e) box internet/point d'accès)	Logiciel: Yersinia
Attaque Serveur rogue DHCP (réseau avec un(e) box internet/point d'accès)	Logiciel: Yersinia
Attaque Man-In-The-Middle (réseau avec un(e) box internet/point d'accès)	Logiciel: Yersinia, Wireshark
DNS Spoofing/Cache Poisonning	Logiciel: Cain&Abel

Figure 35: Aperçu de la *couche réseau - sous catégorie WI-FI* de la check-list. choix de l'attaque *craquage de mot de passe via attaque par dictionnaire*.

3. Exploitation

Attaque par dictionnaire

Avant tout nous devons capturer des paquets de 4-way-handshakes (nécessaires car aircrack-ng en a besoin pour lancer l'attaque). On commence donc par mener une attaque visant à déconnecter les terminaux déjà connectés (pour les obliger à se reconnecter et ainsi capturer les paquets 4-way-handshakes de l'initialisation de leur session). On utilise l'outil aireplay pour ce faire.

```
yannick@yannickali:~$ sudo aireplay-ng --deauth 0 -a E8:1C:BA:A3:AF:89 wlan0mon
23:05:44  Waiting for beacon frame (BSSID: E8:1C:BA:A3:AF:89) on channel 6
23:05:45  Sending 64 directed DeAuth (code 7). STMAC: [B0:73:5D:27:95:FA] [ 0|30
23:05:45  Sending 64 directed DeAuth (code 7). STMAC: [B0:73:5D:27:95:FA] [ 0|46
23:05:46  Sending 64 directed DeAuth (code 7). STMAC: [B0:73:5D:27:95:FA] [ 0|13
23:05:46  Sending 64 directed DeAuth (code 7). STMAC: [B0:73:5D:27:95:FA] [ 0| 6
23:05:47  Sending 64 directed DeAuth (code 7). STMAC: [B0:73:5D:27:95:FA] [ 0|16
23:05:48  Sending 64 directed DeAuth (code 7). STMAC: [B0:73:5D:27:95:FA] [ 0| 1
23:05:48  Sending 64 directed DeAuth (code 7). STMAC: [B0:73:5D:27:95:FA] [ 0| 0
23:05:49  Sending 64 directed DeAuth (code 7). STMAC: [B0:73:5D:27:95:FA] [ 0| 0
23:05:49  Sending 64 directed DeAuth (code 7). STMAC: [B0:73:5D:27:95:FA] [ 0| 1
23:05:50  Sending 64 directed DeAuth (code 7). STMAC: [B0:73:5D:27:95:FA] [ 0| 2
```

Figure 36: *déconnection des terminaux du point d'accès. C'est une forme de déni de service.*

Pendant ce temps, on lance dans un autre terminal une capture de ces paquets. S'intéresser au filtre eapol (il s'agit d'un protocole gérant l'authentification des terminaux)

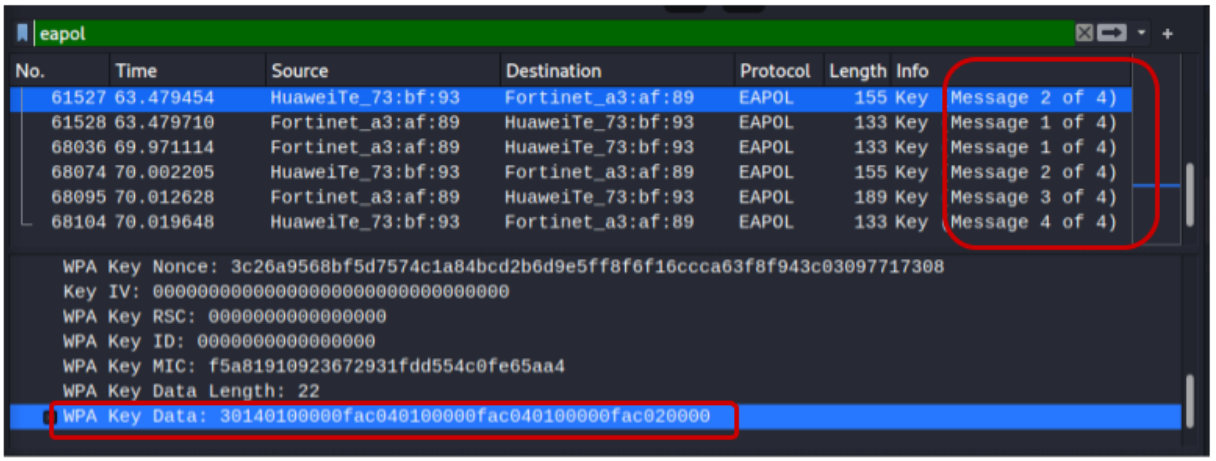


Figure 37: *Nos paquets de 4-way handshakes et la clé que nous souhaitons obtenir (cryptée).*

Nous pouvons maintenant lancer l'attaque (attaque par dictionnaire pour obtenir le mot de passe du point d'accès).

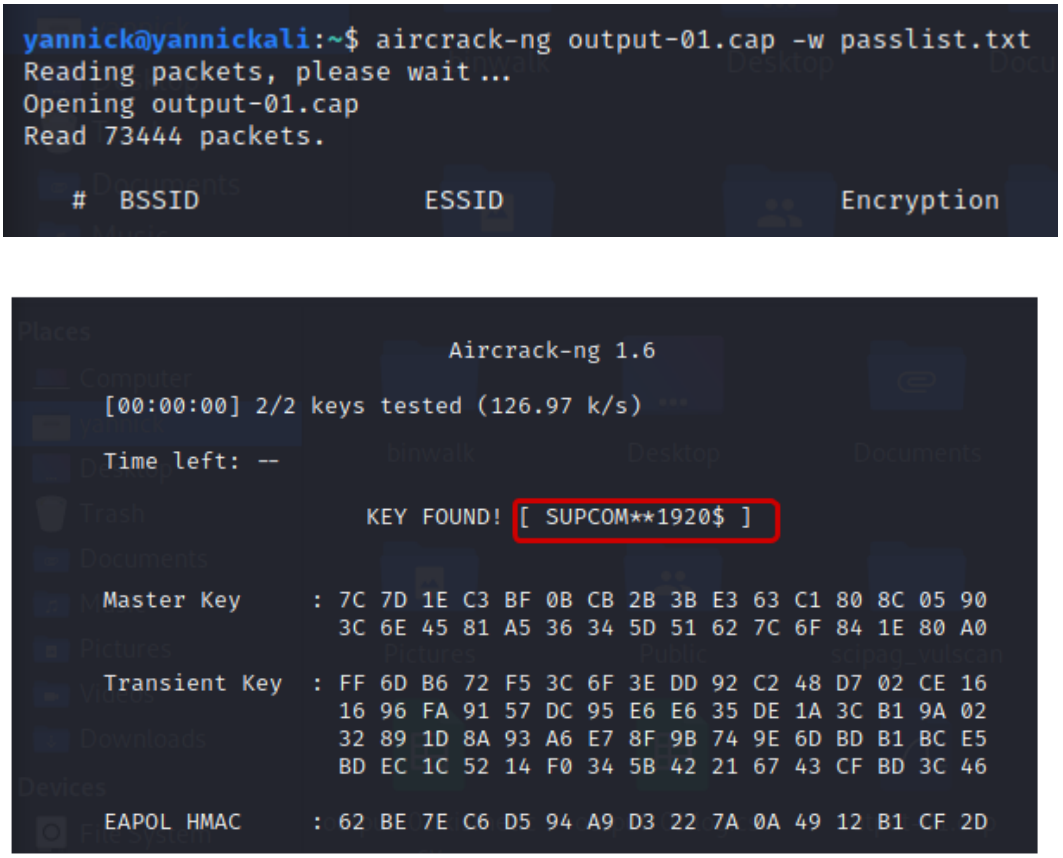


Figure 38: *clé craquée.*

Nous pouvons maintenant nous connecter au réseau cible.

4. Intrusion dans le réseau et retour à la phase de scan de ports

Comme indiqué dans la méthodologie pour un testeur non connecté, une fois introduit dans le réseau, le testeur doit retourner à la sous-étape de s*can de ports* puis continuer le test en tant que testeur appartenant au réseau. Ayant changé de vue sur la cible (passe de non connecté à connecté), il peut mener des attaques plus intéressantes lors de la phase exploitation(voir la check-list).

Après s'être connecté au réseau, nous allons commencer par le scanner ou même approfondir notre phase de découverte.

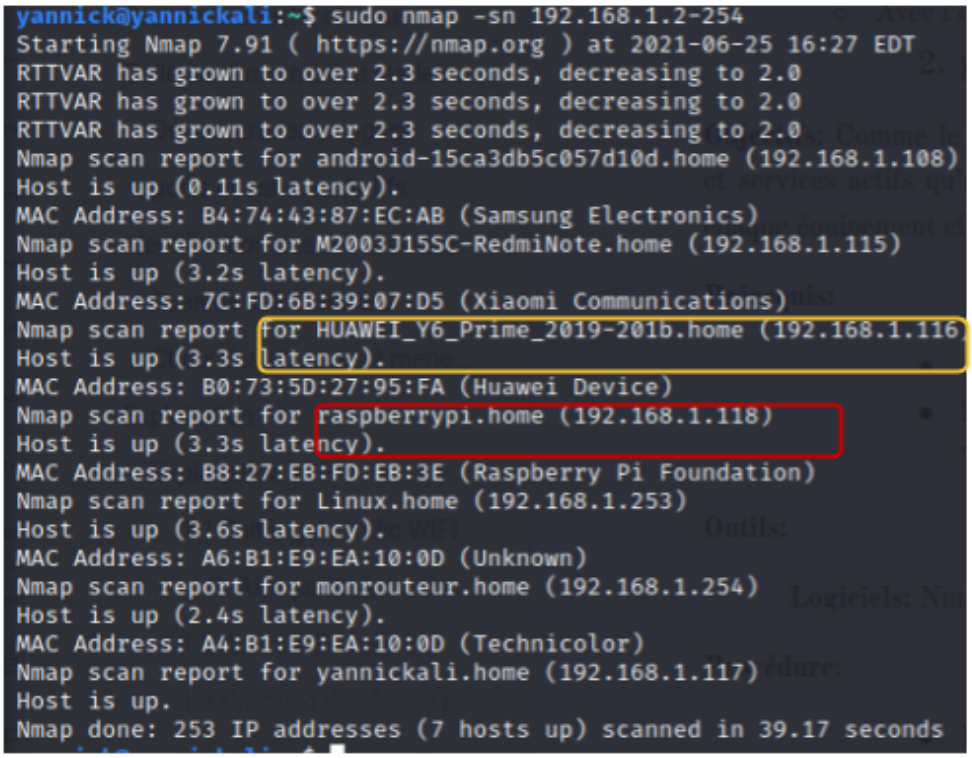


Figure 39: *un IoT découvert (en rouge). Nous allons le cibler ainsi que le smartphone Huawei (en jaune).*

Scan de Ports.

```
yannick@yannickali:~$ nmap 192.168.1.118
Starting Nmap 7.91 ( https://nmap.org ) at 2021-06-25 16:38 EDT
Nmap scan report for raspberrypi.home (192.168.1.118)
Host is up (0.017s latency).
Not shown: 998 closed ports
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
```

Figure 40: *les services ssh et http actif sur le IoT cible*

```
yannick@yannickali:~$ sudo nmap -p 20-24 192.168.1.116
Starting Nmap 7.91 ( https://nmap.org ) at 2021-06-25 16:50 EDT
Nmap scan report for HUAWEI_Y6_Prime_2019-201b.home (192.168.1.116)
Host is up (0.11s latency).

PORT   STATE  SERVICE
20/tcp closed ftp-data
21/tcp closed ftp
22/tcp closed ssh
23/tcp closed telnet
24/tcp closed priv-mail
MAC Address: B0:73:5D:27:95:FA (Huawei Device)

Nmap done: 1 IP address (1 host up) scanned in 0.45 seconds
yannick@yannickali:~$ sudo nmap -p 80 192.168.1.116
Starting Nmap 7.91 ( https://nmap.org ) at 2021-06-25 16:51 EDT
Nmap scan report for HUAWEI_Y6_Prime_2019-201b.home (192.168.1.116)
Host is up (0.28s latency).

PORT   STATE  SERVICE
80/tcp closed http
MAC Address: B0:73:5D:27:95:FA (Huawei Device)

Nmap done: 1 IP address (1 host up) scanned in 0.56 seconds
```

Figure 41: *Scan du smartphone Huawei.*

Scan de vulnérabilités

Conformément à la description de la méthodologie et l'annexe des procédures des tests, nous avons recouru à nmap pour le scan de vulnérabilités.

```
yannick@yannickali:~$ nmap -sV --script=vulscan/vulscan.nse 192.168.1.118
Starting Nmap 7.91 ( https://nmap.org ) at 2021-06-25 16:40 EDT
Nmap scan report for raspberrypi.home (192.168.1.118)
Host is up (0.019s latency).
Not shown: 998 closed ports
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.9p1 Raspbian 10+deb10u2+rpt1 (protocol 2.0)
| vulscan: VulDB - https://vuldb.com:
| [130671] gsi-openssh-server 7.9p1 on Fedora /etc/gsissh/sshd_config privile
|
| MITRE CVE - https://cve.mitre.org:
| [CVE-2010-4755] The (1) remote_glob function in sftp-glob.c and the (2) pro
8.1, NetBSD 5.0.2, OpenBSD 4.7, and other products, allow remote authenticate
expressions that do not match any pathnames, as demonstrated by glob expressi
2010-2632.
| [CVE-1999-0661] A system is running a version of software that was replaced
6, (2) util-linux 2.9g, (3) wuarchive ftpd (wuftpd) 2.2 and 2.1f, (4) IRC cli
|
| SecurityFocus - https://www.securityfocus.com/bid/:
| [102780] OpenSSH CVE-2016-10708 Multiple Denial of Service Vulnerabilities
| [101552] OpenSSH 'sftp-server.c' Remote Security Bypass Vulnerability
| [94977] OpenSSH CVE-2016-10011 Local Information Disclosure Vulnerability
| [94975] OpenSSH CVE-2016-10012 Security Bypass Vulnerability
| [94972] OpenSSH CVE-2016-10010 Privilege Escalation Vulnerability
| [94968] OpenSSH CVE-2016-10009 Remote Code Execution Vulnerability
| [93776] OpenSSH 'ssh/kex.c' Denial of Service Vulnerability
| [92212] OpenSSH CVE-2016-6515 Denial of Service Vulnerability
| [92210] OpenSSH CBC Padding Weak Encryption Security Weakness
| [92209] OpenSSH MAC Verification Security Bypass Vulnerability
| [91812] OpenSSH CVE-2016-6210 User Enumeration Vulnerability
```

Figure 42: *Scan de vulnérabilités sur la raspberry pi.*

Choix des attaques

Nous aurions pu exploiter quelques-unes des nombreuses vulnérabilités détectées par scan précédemment. Mais nous ne nous y attarderons pas dans le cadre de ce projet.

Pour aller plus loin

Pour rappel, nous avions détecté un IoT (Raspberry pi). Les équipements IoT sont souvent négligés d'un point de vue des pratiques hygiéniques de sécurité. On peut tenter une connexion ssh à la raspberrypi avec les identifiants par défaut (utilisateur: pi, mot de passe: raspberry).

```
yannick@yannickali:~$ ssh pi@192.168.35.151
pi@192.168.35.151's password:
Linux raspberrypi 5.10.17-v7+ #1414 SMP Fri Apr 30 13:18:35 BST 2021 armv7l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Jun 26 11:52:37 2021

SSH is enabled and the default password for the 'pi' user has not been changed.
This is a security risk - please login as the 'pi' user and type 'passwd' to set a new password.

pi@raspberrypi:~ $
```

Figure 43: *Connexion réussie via ssh avec les identifiants par défaut de la carte raspberry.*

Lors des scans de ports, nous avions détecté le port 80 de la carte raspberry qui était actif. On peut donc soupçonner la présence d'un serveur web abrité par la carte depuis lequel des objets connectés sont manipulés. Pour ce faire, vérifions l'existence d'une page web dans le dossier /var/www/html pour en analyser le contenu

```
pi@raspberrypi:~ $ ls /var/www/html/
index.php  index.php.save
pi@raspberrypi:~ $
```

Figure 44: *présence d'une page web.*

Cette page est potentiellement celle qui permet le contrôle d'objets connectés. Pour en être sûr on peut en analyser le contenu.

```
            <input type="submit" style = "font-size: 16 pt" value="eteindre" name="
            <input type="submit" style = "font-size: 16 pt" value="allumer" name="
            <input type="submit" style = "font-size: 16 pt" value="faire clignoter"
    </form>
    <?php
    shell_exec("/usr/local/bin/gpio -g mode 17 out");
    if(isset($_GET['off']))
    {
            echo "La LED est eteinte";
            shell_exec("/usr/local/bin/gpio -g write 17 0");
    }
    else if(isset($_GET['on']))
    {
            echo "la LED est eteinte";
            shell_exec("/usr/local/bin/gpio -g write 17 1");
    }
    else if(isset($_GET['blink']))
    {
            echo "La LED clignote";
            for($x = 0;$x<=4;$x++)
            {
                    shell_exec("/usr/local/bin/gpio -g write 17 1");
                    sleep(1);
                    shell_exec("/usr/local/bin/gpio -g write 17 0");
                    sleep(1);
```

Figure 45: *Le code confirme nos soupçons.*

Dans le cadre de notre test d'intrusion, nous pouvons modifier malicieusement ce code, comme par exemple y insérer un shell distant inverse via une connexion ssh (ssh reverse shell). Il constitue une forme de backdoor car il nous permettra d'avoir la possibilité d'accéder au serveur web et ainsi à la raspberry ultérieurement.

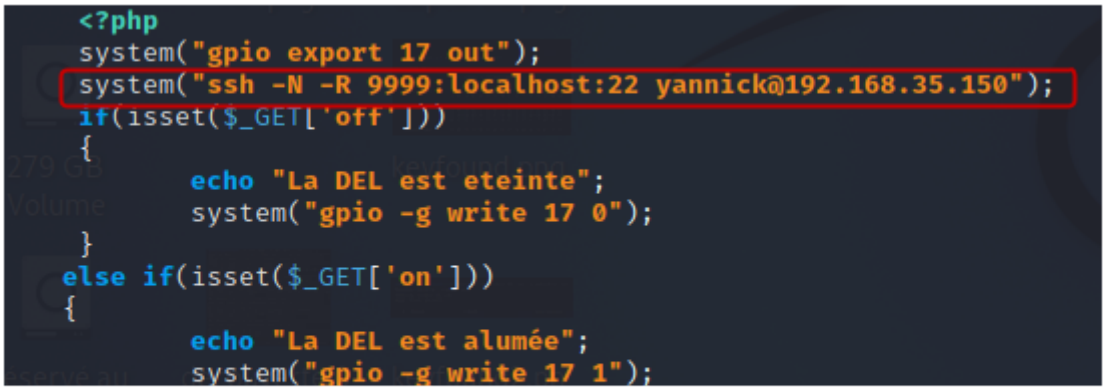


Figure 46: *Insertion d'un code malicieux dans le serveur web. Cette ligne de code fait initier une connexion ssh par la victime vers l'attaquant. On a ainsi un shell distant inverse (ssh reverse shell).*

NB: pour que la connexion ssh soit possible il faut que le testeur mène quelques opérations préalables, comme générer une clé pré-partagée dans la raspberry pi, puis la transférer vers son répertoire .ssh/authorized_keys. Cela est indiqué sur la figure ci-dessous. L'avantage avec les reverse shells est qu'ils permettent de contourner les pare-feux vu que la connexion est initiée par la victime. En effet les pare-feux filtrent généralement les connexions entrantes, mais sont très souvent configurés pour filtrer moins rigoureusement les connexions sortantes).

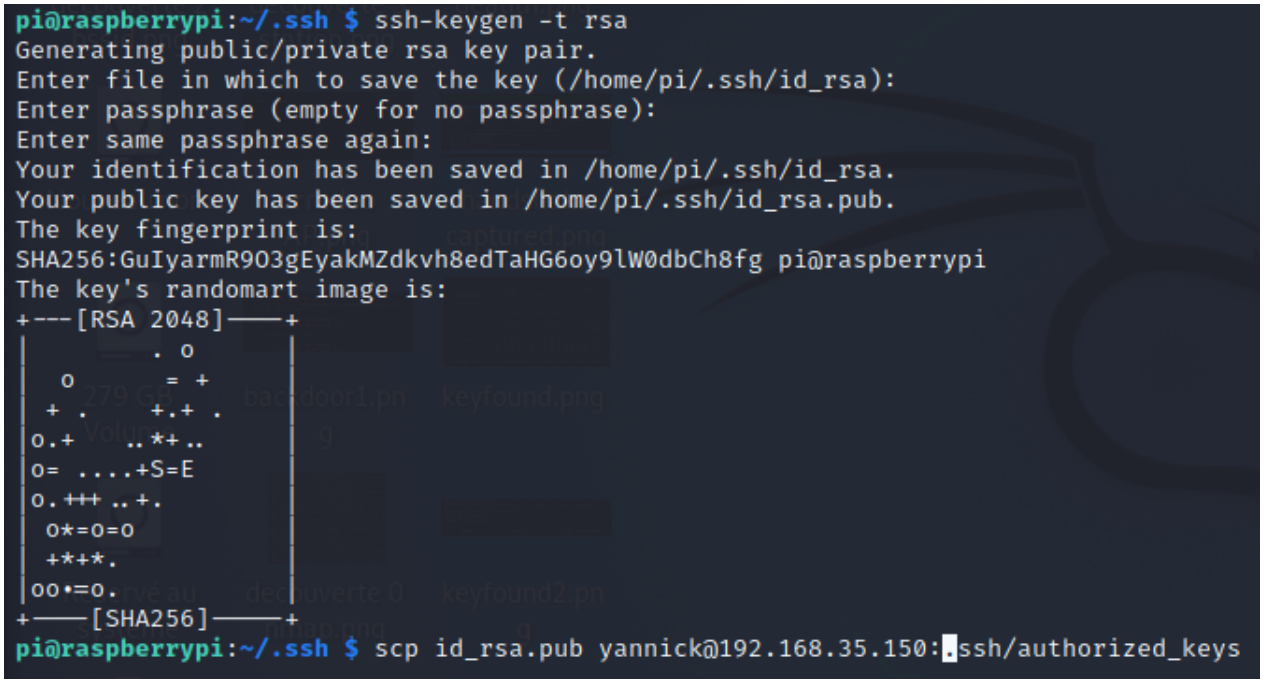


Figure 47: *Génération d'une clé pré-partagée et transfert vers la machine de l'attaquant. Le but est de rendre effectif le shell ssh inverse.*

5. Conclusion du test d'intrusion au niveau de la couche réseau.

Au sortir de cette phase de notre test d'intrusion, nous retenons que la méthodologie nous a permis d'abord d'**analyser** notre réseau cible. Cela dans le but d'en découvrir des **cibles potentielles** (points d'accès, terminaux mobiles, IoT, PC).

Ensuite nous nous sommes **introduits dans le réseau** en menant un crack de **mot de passe** via attaque dictionnaire. Cela nous a permis d'avoir une meilleure vue sur les équipements pour mieux les découvrir, et en identifier des **vulnérabilités**. Les vulnérabilités découvertes pourront être exploitées à la suite par le testeur.

Enfin nous avons compromis un IoT découvert dans notre réseau cible en y insérant un **backdoor** permettant un shell distant inverse (**remote reverse shell**). Le testeur pourra à l'avenir s'introduire furtivement et mener d'autres attaques plus compromettantes et effectuer du mouvement latéral a partir du système compromis.

Comme contre-mesures, nous proposons une **bonne politique de mot de passe**, meilleur moyen de se protéger contre les attaques de craquage de clés. Cela nous aurait également éviter de compromettre la raspberry pi qui fonctionnait avec les identifiants par défauts.

B. La couche firmware

Pour la couche firmware, nous avons opté pour le firmware DWR 932 de D-Link (DWR-932_fw_revB_2_02_eu_en_20150709 en long). Nous avons limité les attaques de cette couche aux analyses de rétro-ingénierie et de recherches de vulnérabilités. Ceci en raison de l'absence de moyens / équipements physiques pour mener les attaques, de gadgets pour extraire puis flasher le firmware ou encore en raison du risque d'endommager une box et en perdre la garantie.

Nous allons donc nous passer de la dernière phase, celle du flash (réécriture) du firmware dans l'équipement.

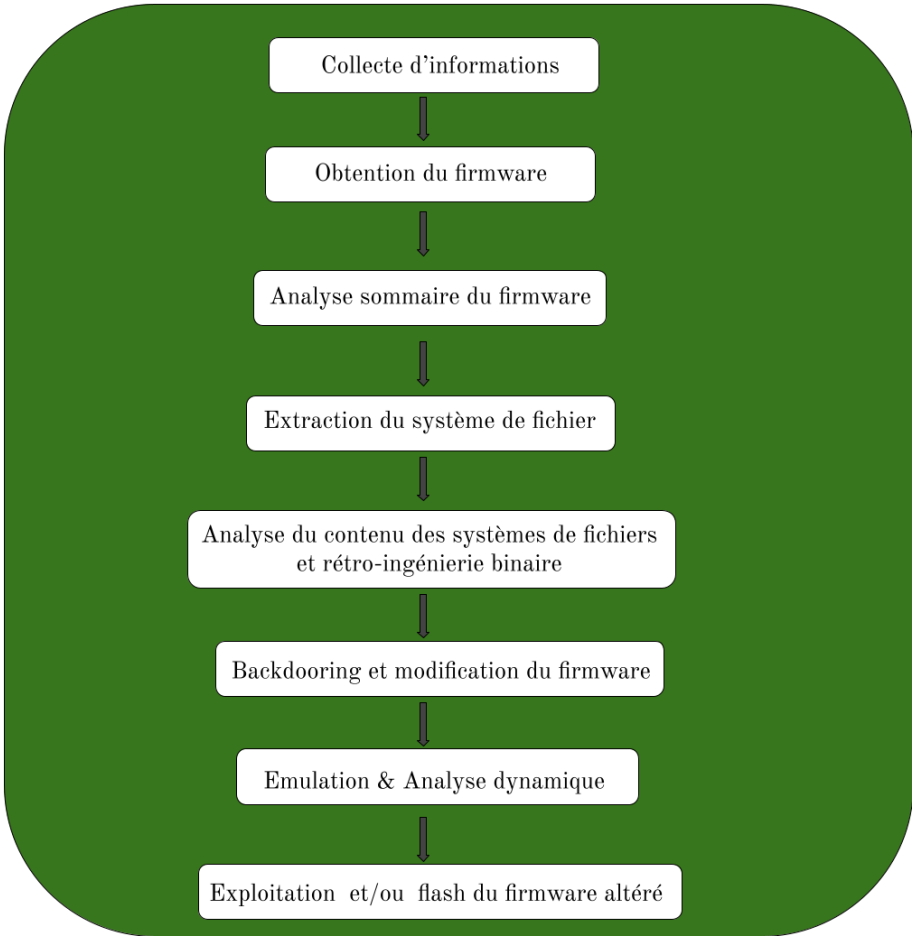


Figure 48: *La méthodologie firmware à titre de rappel.*

1. La collecte d'information

En suivant les pistes proposées par la méthodologie dans l'annexe des procédures, intéressons-nous respectivement à:

L'architecture de processeur prise en charge:

Le site openwrt.org nous fournit la fiche technique suivante qui nous informe quant à l'architecture prise en charge:

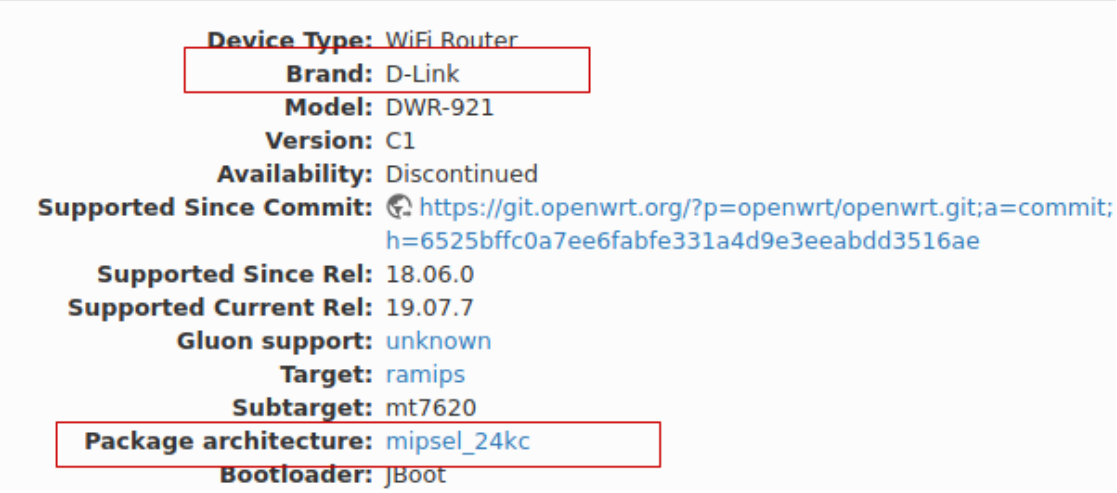
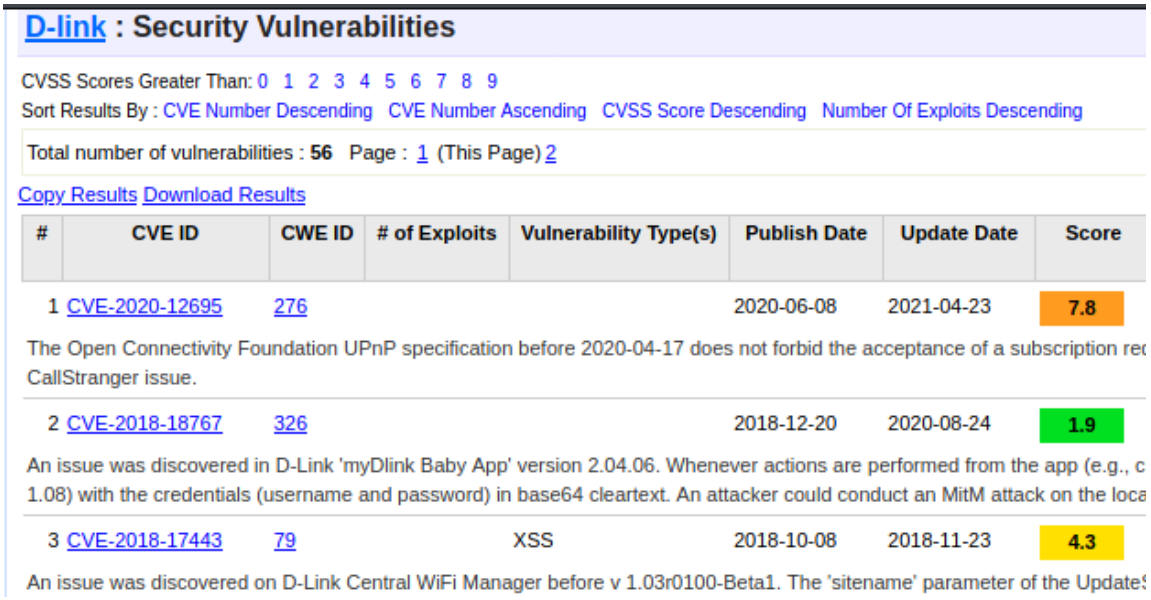


Figure 49: *Architecture des modèles de la série DWR de D-LINK est donc de la famille MIPS*

Aux vulnérabilités Connues:

La plateforme cvedetails répertorie chaque vulnérabilité connue sous une appellation caractéristique (CVE-*annee_de_decouverte-nombre_arbitraire*).



D-link : Security Vulnerabilities

CVSS Scores Greater Than: 0 1 2 3 4 5 6 7 8 9

Sort Results By : CVE Number Descending CVE Number Ascending CVSS Score Descending Number Of Exploits Descending

Total number of vulnerabilities : **56** Page : 1 (This Page) 2

Copy Results Download Results

#	CVE ID	CWE ID	# of Exploits	Vulnerability Type(s)	Publish Date	Update Date	Score
1	CVE-2020-12695	276			2020-06-08	2021-04-23	7.8
	The Open Connectivity Foundation UPnP specification before 2020-04-17 does not forbid the acceptance of a subscription rec CallStranger issue.						
2	CVE-2018-18767	326			2018-12-20	2020-08-24	1.9
	An issue was discovered in D-Link 'myDlink Baby App' version 2.04.06. Whenever actions are performed from the app (e.g., c 1.08) with the credentials (username and password) in base64 cleartext. An attacker could conduct an MitM attack on the loca						
3	CVE-2018-17443	79		XSS	2018-10-08	2018-11-23	4.3
	An issue was discovered on D-Link Central WiFi Manager before v 1.03r0100-Beta1. The 'sitename' parameter of the Update						

Figure 50: *vulnérabilités connues des routeurs D-Link*

Bilan de la phase de collecte d'informations

Nous savons que notre firmware a comme architecture **MIPS**. Cette information est normalement utile pour les phases d'implantation de backdoor (Implantation de backdoor) et d'émulation du firmware.

Nous avons aussi une pile de **vulnérabilités** découvertes sur la plateforme ci-dessus. Nous pouvons nous intéresser aux identifiants CVE de 2015 jusqu'au-delà (la version de votre firmware date de 2015 comme le nom nous laisse l'entendre).

2. Obtention du firmware

D'après la méthodologie nous avons le choix entre 2 options: l'extraction depuis l'équipement ou le téléchargement (depuis le site de l'équipementier par exemple). Dans notre cas, il est évident que nous opterons pour la seconde option.

Sur le site de DLINK nous pouvons télécharger le firmware.

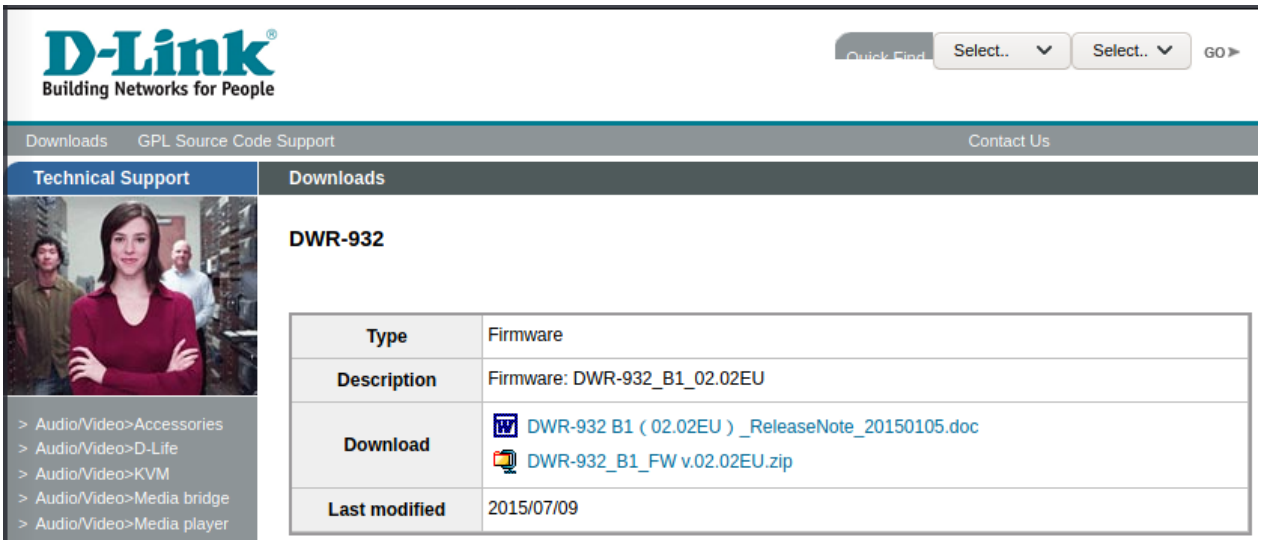


Figure 51: *Téléchargement du firmware*

3. Analyse sommaire du firmware

Nous avons eu recours à l'utilitaire binwalk.

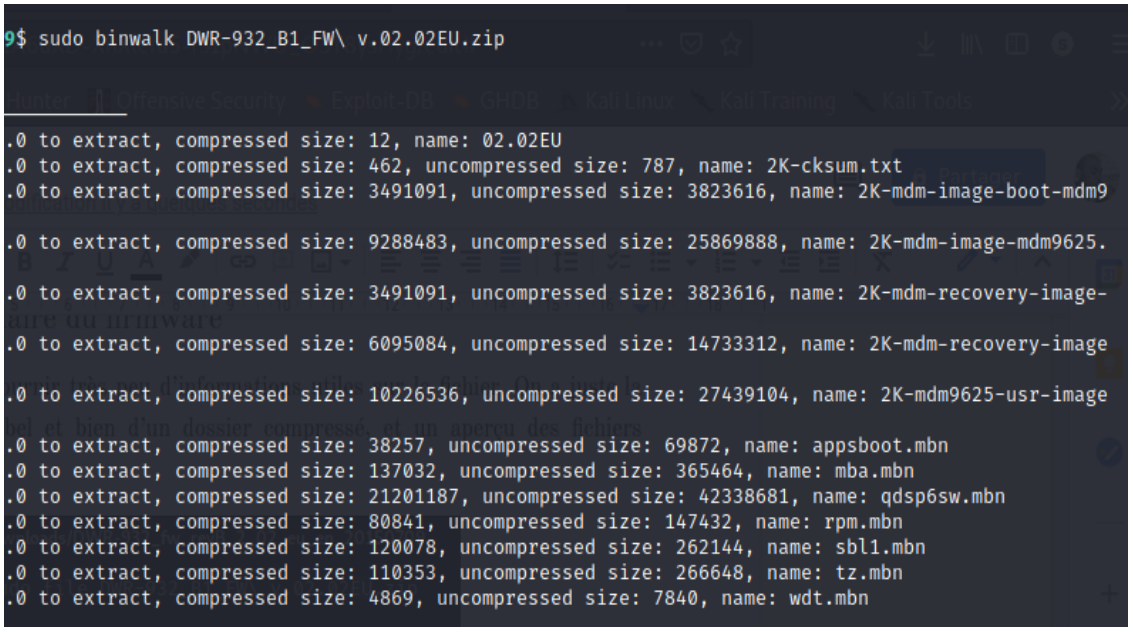


Figure 52: *Informations sommaires sur le fichier téléchargé.*

Les utilitaires semblent fournir très peu d'informations utiles sur le fichier. On a juste la confirmation qu'il s'agit bel et bien d'un dossier compressé, et un aperçu des fichiers compressés qu'il contient.

NB: à l'intérieur du fichier .zip originel se trouve un autre fichier .zip. C'est lui qui contient le système de fichiers.

4. Extraction du système de fichier

Comme le titre l'indique, il s'agit ici d'extraire les fichiers du firmware. Cela va permettre de le parcourir à la recherche de code non compilé vulnérable, etc.

D'abord nous remarquons que nous avons affaire à un système de fichiers yaffs.

```
yannick@yannickali:/root/Downloads/DWR-932_fw_revB_2_02_eu_en_20150709$ ls -la
total 116384
drwxr-xr-x 2 root root     4096 Jun 24 13:20 .
drwxr-xr-x 5 root root     4096 Jun 24 13:20 ..
-rw-r--r-- 1 root root        0 Jan  3  2015 02.02EU
-rw-r--r-- 1 root root      787 Jan  3  2015 2K-cksum.txt
-rw------- 1 root root 27439104 Jan  3  2015 2K-mdm9625-usr-image.usrfs.yaffs2
-rw-r--r-- 1 root root  3823616 Jul 18  2014 2K-mdm-image-boot-mdm9625.img
-rw-r--r-- 1 root root 25869888 Jul 18  2014 2K-mdm-image-mdm9625.yaffs2
-rw-r--r-- 1 root root  3823616 Jul 18  2014 2K-mdm-recovery-image-boot-mdm9625.img
-rw-r--r-- 1 root root 14733312 Jul 18  2014 2K-mdm-recovery-image-mdm9625.yaffs2
-rwxr-xr-x 1 root root    69872 Jul 18  2014 appsboot.mbn
-rwxr-xr-x 1 root root   365464 Jul 18  2014 mba.mbn
-rwxr-xr-x 1 root root 42338681 Jul 18  2014 qdsp6sw.mbn
-rwxr-xr-x 1 root root   147432 Jul 18  2014 rpm.mbn
-rwxr-xr-x 1 root root   262144 Jul 18  2014 sbl1.mbn
-rwxr-xr-x 1 root root   266648 Jul 18  2014 tz.mbn
-rwxr-xr-x 1 root root     7840 Jul 18  2014 wdt.mbn
```

Figure 53: *contenu du fichier .zip originel après avoir été décompressé.*

Nous allons donc recourir à l'utilitaire unyaffs pour extraire le système de fichier. Commençons par celui-ci *2K-mdm-image-mdm9625.yaffs2*.

```
yannick@yannickali:/root/Downloads/firmware$ sudo unyaffs 2K-mdm-image-mdm9625.yaffs2  root-yaffs
yannick@yannickali:/root/Downloads/firmware$
```

Figure 54: *Extraction du système de fichier dans un nouveau répertoire nommé root-yaffs.*

NB: Nous avons renommé le dossier *DWR-932_fw_revB_2_02_eu_en_20150709* en *firmware*. Il s'agit du même répertoire.

```
yannick@yannickali:/root/Downloads/firmware/root-yaffs$ ls -la
total 96
drwxr-xr-x 21 root root  4096 Jul 18  2014 .
drwxr-xr-x  3 root root  4096 Jun 24 13:59 ..
drwxr-xr-x  2 root root 12288 Jul 18  2014 bin
drwxr-xr-x  2 root root  4096 Jul 18  2014 boot
-rw-r--r--  1 root root    38 Jul 18  2014 build.prop
drwxr-xr-x  2 root root  4096 Jul 18  2014 cache
drwxr-xr-x  2 root root  4096 Jul 18  2014 config
drwxr-xr-x  2 root root  4096 Jul 18  2014 config2
drwxr-xr-x  2 root root  4096 Jul 18  2014 dev
drwxr-xr-x  2 root root  4096 Jul 18  2014 disk
drwxr-xr-x 31 root root  4096 Jul 18  2014 etc
drwxr-sr-x  3 root root  4096 Jul 18  2014 home
drwxr-xr-x  4 root root  4096 Jul 18  2014 lib
lrwxrwxrwx  1 root root    12 Jul 18  2014 linuxrc -> /bin/busybox
drwxr-xr-x 10 root root  4096 Jul 18  2014 media
drwxr-xr-x  2 root root  4096 Jul 18  2014 mnt
drwxr-xr-x  2 root root  4096 Jul 18  2014 proc
drwxr-xr-x  2 root root  4096 Jul 18  2014 sbin
lrwxrwxrwx  1 root root    11 Jul 18  2014 sdcard -> /media/card
drwxr-xr-x  2 root root  4096 Jul 18  2014 sys
drwxrwxrwt  2 root root  4096 Jul 18  2014 tmp
drwxr-xr-x  2 root root  4096 Jul 18  2014 usr
drwxr-xr-x  7 root root  4096 Jul 18  2014 var
drwxr-xr-x  3 root root  4096 Jul 18  2014 WEBSERVER
lrwxrwxrwx  1 root root     8 Jul 18  2014 www -> /usr/www
```

Figure 55: *Aperçu du système de fichier extrait*

5. Analyse du contenu des systèmes de fichiers et rétro-ingénierie binaire

Maintenant que nous avons l'ensemble des fichiers du firmware, nous allons le parcourir à la recherche de vulnérabilités et de code/configuration vulnérable.

Commençons par effectuer une analyse statique Manuelle

Essayons de lister les fichiers de configurations:

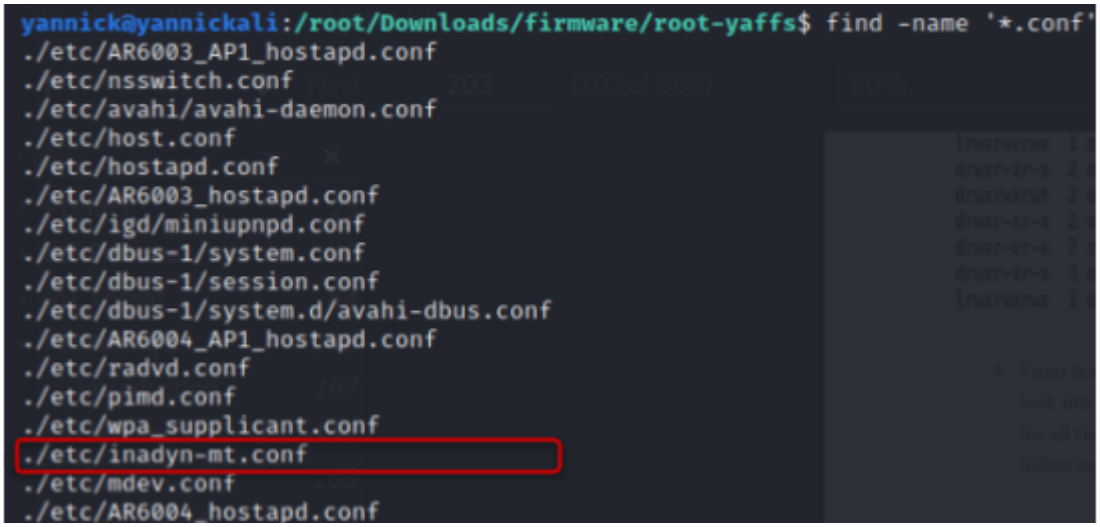


Figure 56: *liste des fichiers de configuration*

On peut ensuite parcourir le contenu de chacun d'entre eux. Certains d'entre eux contiennent des données que nous jugeons sensibles notamment: *./etc/inadyn-mt.conf* encadré en rouge. En effet en affichant son contenu on obtient ceci:

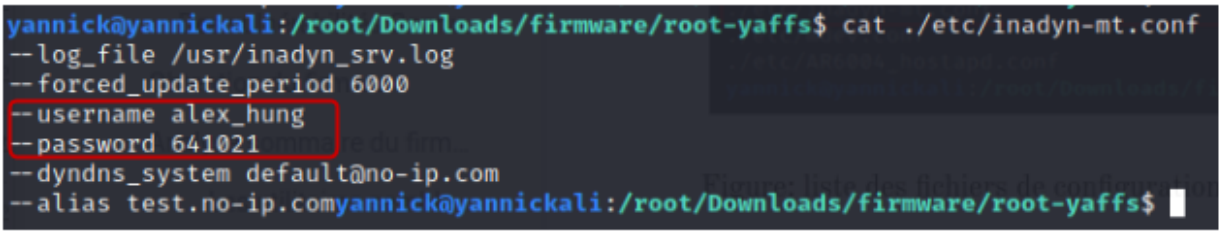


Figure 57: Identifiants trouvés dans le fichier *./etc/inadyn-mt.conf*

Parcourons les fichiers */etc/passwd* et */etc/shadow*

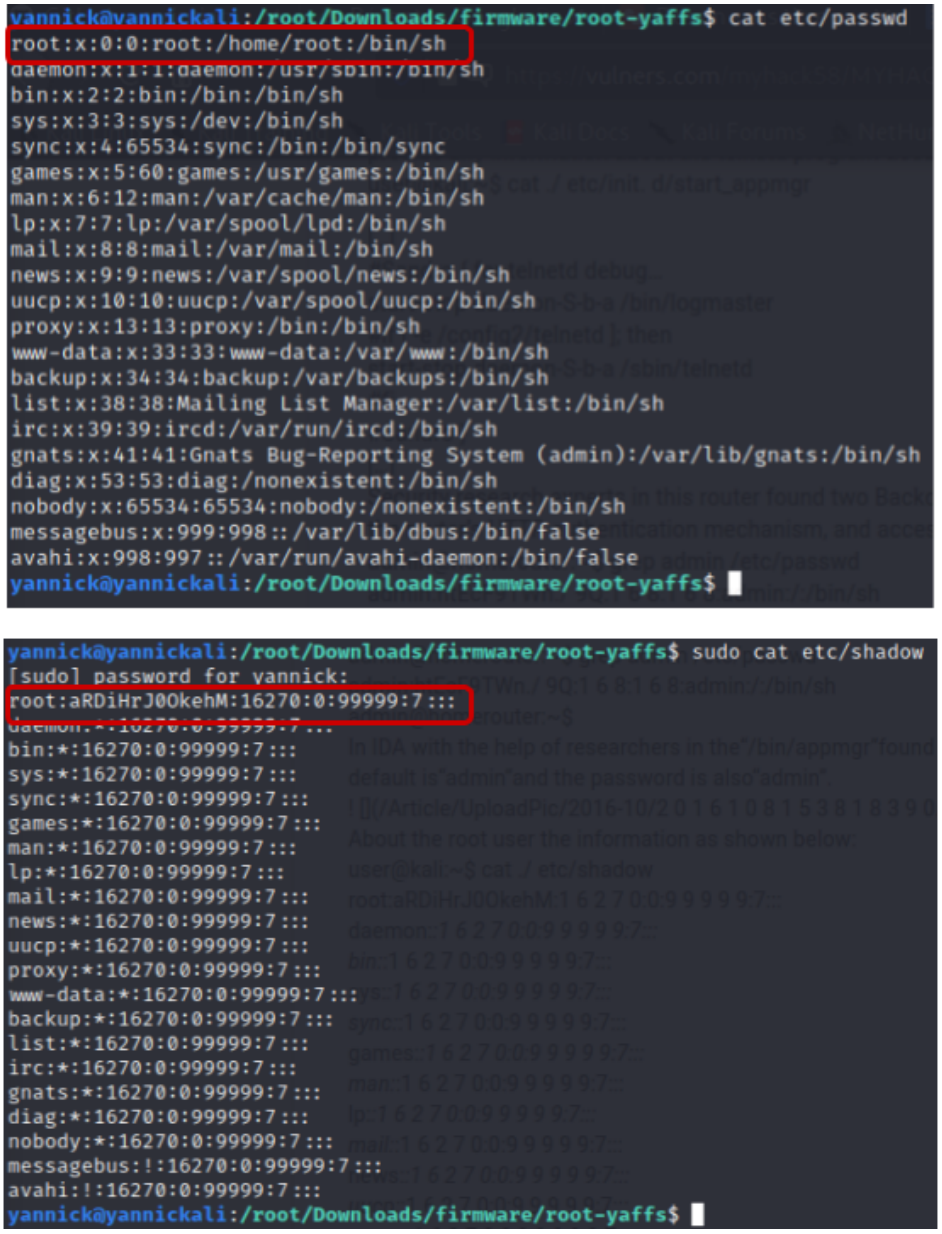


Figure 58: Un compte Root identifié: le hash dans */etc/shadow* va nous permettre de cracker le mot de passe de l'utilisateur Root.

Nous nous trouvons en présence d'un compte Root. Tentons d'en identifier le mot de passe avec l'utilitaire John The Ripper.

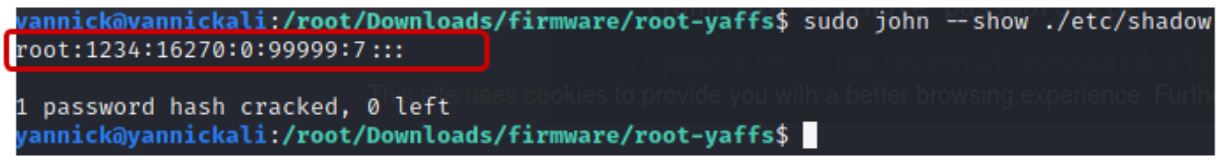


Figure 59: *mot de passe obtenu. Le compte root a comme mot de passe 1234.*

Menons une analyse statique automatisée

On peut recourir à firmwalker pour lister les fichiers les plus sensibles du système de fichiers. Après quoi on peut au besoin se livrer à une exploration approfondie de chacun d'eux. Nous ne nous y attarderons pas car dans la méthodologie cela est laissé aux soins du testeur.

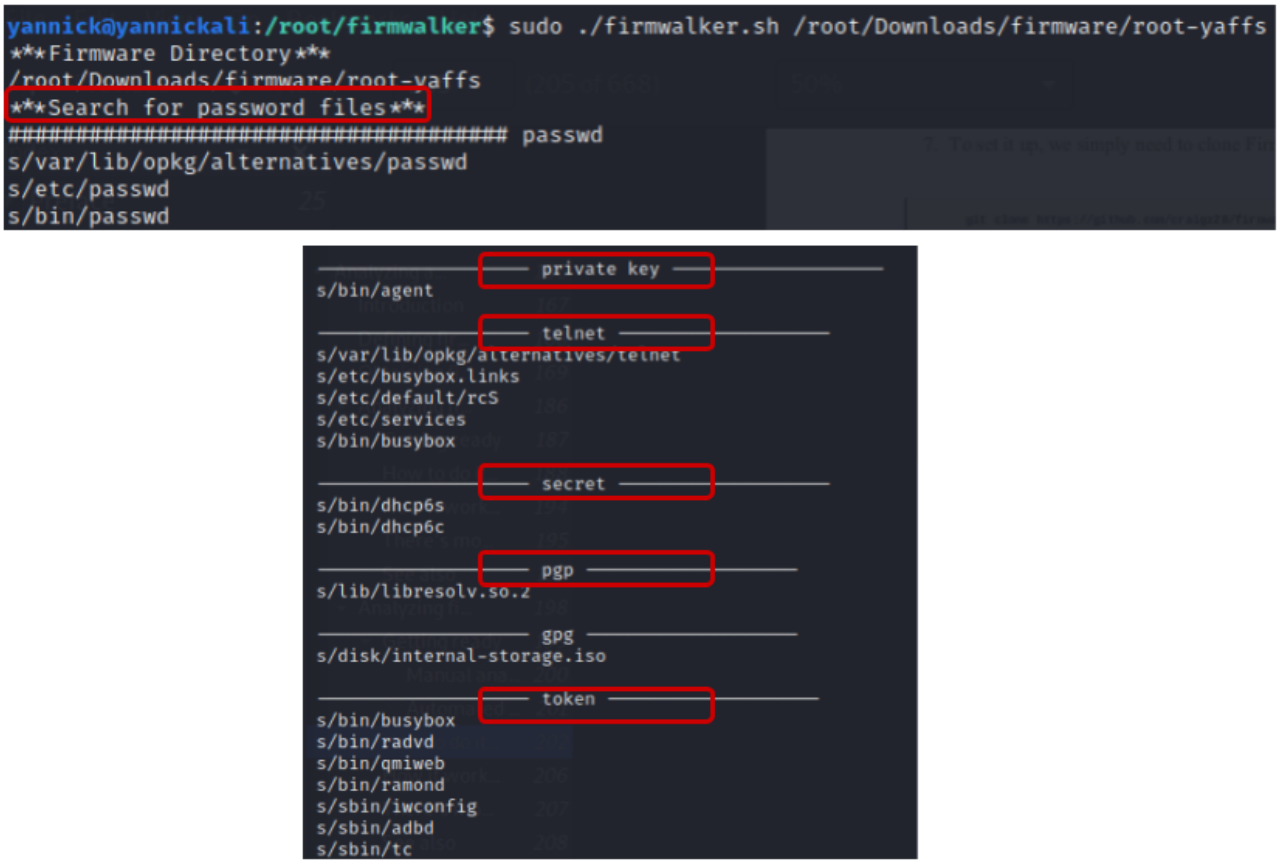


Figure 60: *Fichiers sensibles découverts par l'outil Firmwalker.*

Menons la phase de rétro-ingénierie binaire

Pour ce faire nous aurons recours à l'outil Hopper.

Commençons par lancer Hopper.

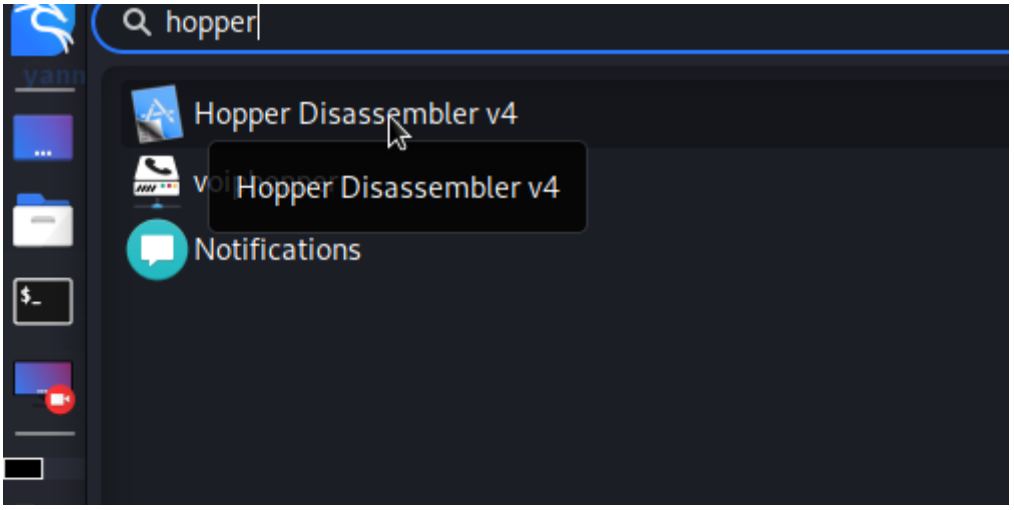


Figure 61: *Hopper depuis le menu.*

Ensuite chargeons un des fichiers binaires sur lesquels nous souhaitons mener notre rétro-ingénierie (reverse engineering). Intéressons-nous au fichier *appmgr* dans */bin*

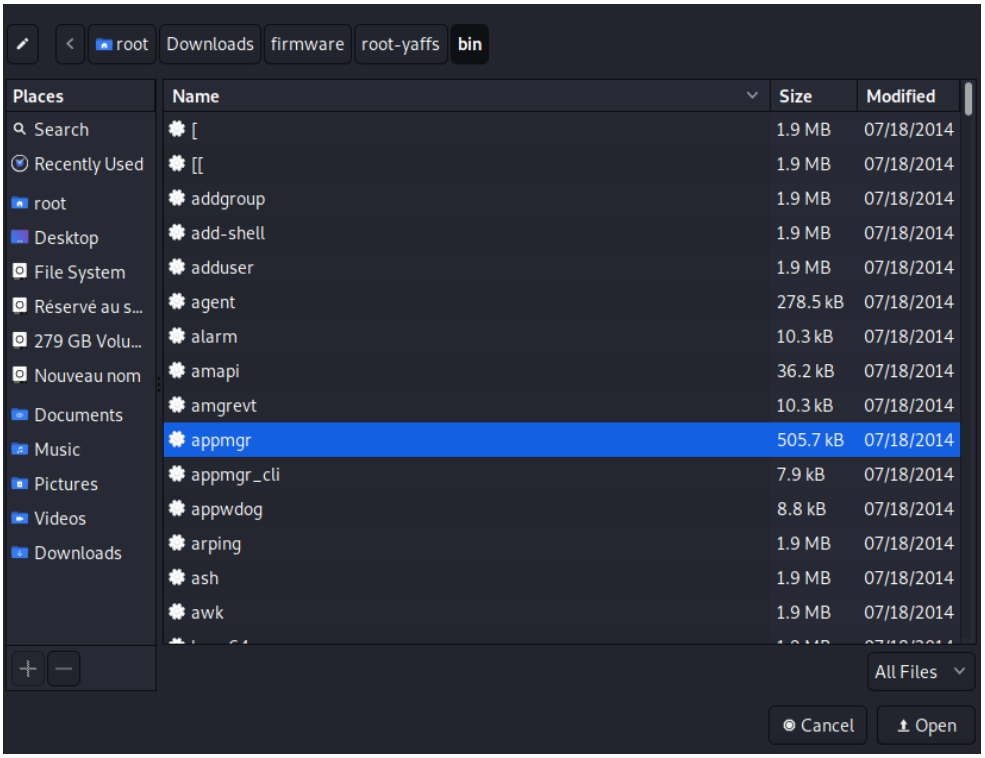


Figure 62: *Chargement du binaire appmgr*

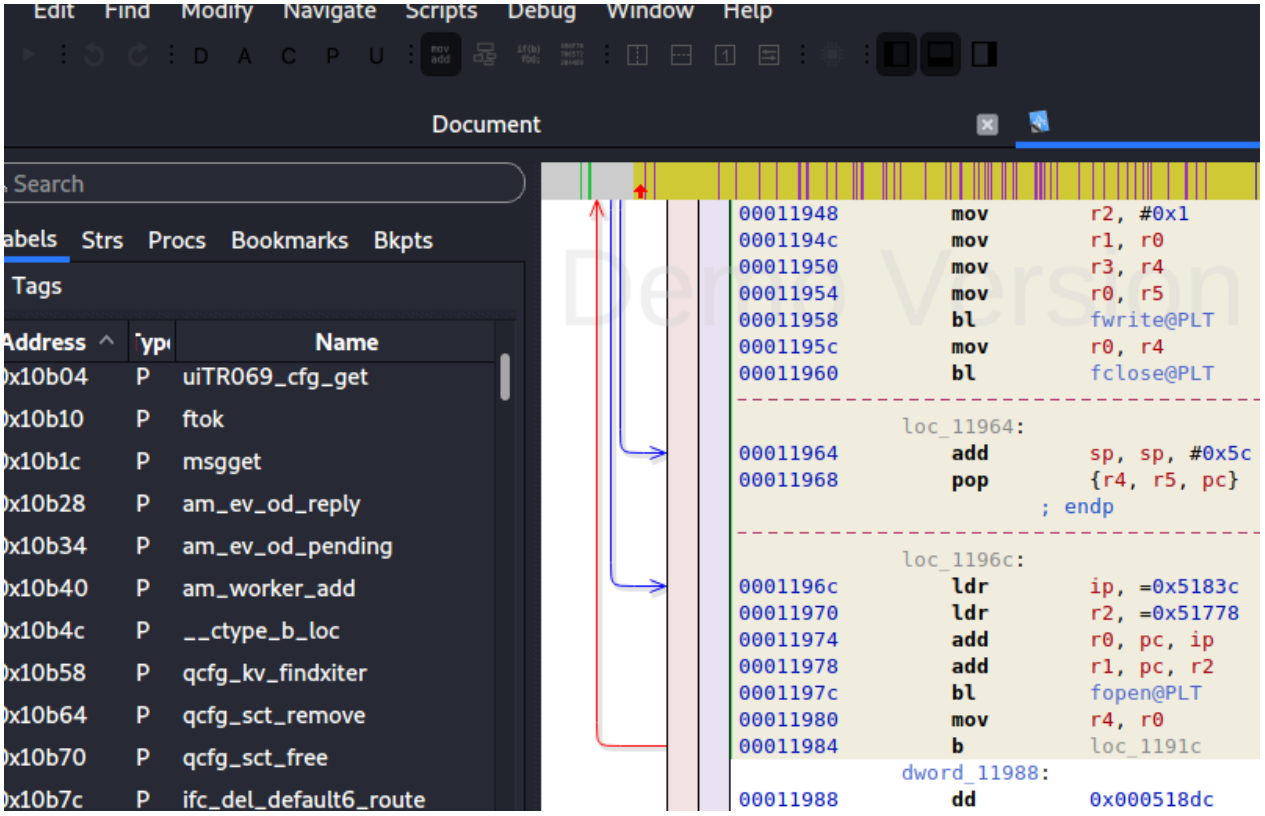


Figure 63: *Hopper disassembler* nous fournit le code assembleur du binaire (à droite), ainsi que les fonctions qui le composent (à gauche).

Les comptes que nous avons identifiés précédemment sont probablement des comptes d'accès à distance incrustés dans le firmware à des fins éventuelles de maintenances. Ils constituent en quelque sorte des backdoors implantées expressément. Il s'agit d'hypothèses que nous pourrons vérifier.

En faisant une recherche de mots-clés comme **ssh**, **telnet**, ... dans chacun des binaires du système via l'outil hopper, on peut remarquer ceci:

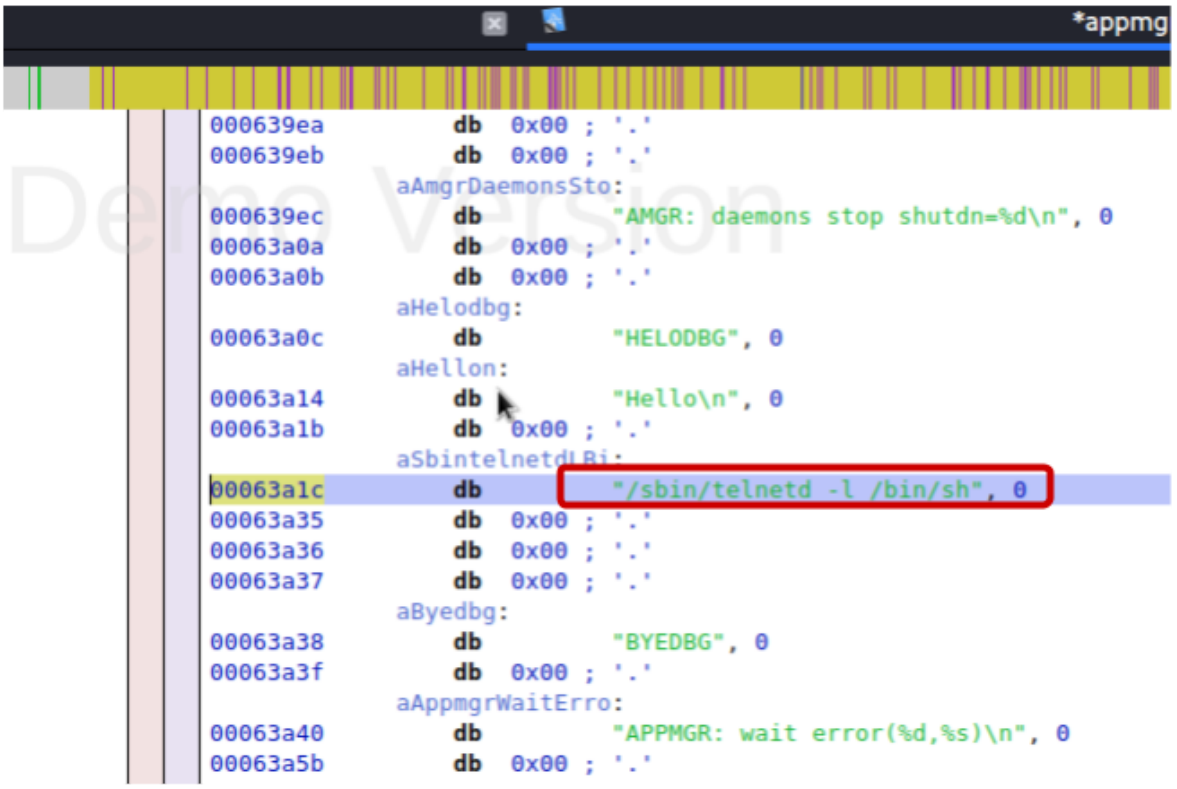


Figure 64: *le firmware semble fournir des services d'accès à distance (en l'occurrence telnet).*

Toujours grâce à Hopper, tentons de générer **le pseudo-code** de la fonction de laquelle est issue l'instruction impliquée. (la fonctionnalité *show pseudo-code of procedure* de hooper transforme le code assembleur en une syntaxe proche du langage C).



Figure 65: *Pseudo-code de la fonction de laquelle provient l'instruction lançant le daemon telnetd*

Dans l'encadré numéro 1 en jaune on peut noter que le programme reçoit un message provenant d'un socket (rôle de la fonction *recvfrom*).

Ensuite dans l'encadré numéro 2 en rouge, une comparaison de chaînes de caractères (rôle de la fonction *strncmp*) est faite. Notons la chaîne codée en dur **HELODBG** avec laquelle est faite la comparaison.

On entre dans une boucle selon le résultat de cette comparaison (encadré numéro 3 en vert).

Suite à quoi est lancé le service telnet (encadré numéro 4 en bleu). On pourra accéder au firmware via telnet avec le compte Root identifié plus haut.

Bilan de la phase d'analyse du système de fichiers

Au sortir de cette étape, nous avons identifié **plusieurs comptes,** parmi lesquels un **compte Root** dont nous avons craqué le **mot de passe** (1234). Nous avons aussi découvert l'existence d'un **backdoor** accessible via telnet avec la chaîne de caractères HELODBG comme clé. Enfin nous avons découvert plusieurs autres fichiers potentiellement sensibles.

6. Conclusion de la phase de test du firmware

Comme nous l'avons évoqué plus haut, nous avons limité cette phase de notre test d'intrusion aux attaques de rétro-ingénierie (reverse Engineering) et de découvertes de vulnérabilité / mauvaises pratiques d'hygiène de sécurité.

Pour ce qui est par exemple de la phase de **d'implantation de backdoors**, Il est évident que nous nous en passerons en raison de l'existence des backdoors déjà découverts dans le firmware. La phase d'**émulation** est une phase facultative que le testeur mène pour valider ses découvertes/altérations malicieuses avant de s'attaquer au firmware de l'équipement cible réel. Enfin la phase d'exploitation et de réécriture du firmware n'est évidemment pas menable sans équipement physique.

En somme, en suivant la méthodologie, nous avons été capables de collecter des **vulnérabilités connues** sur le firmware puis d'en extraire le système de fichiers pour l'analyser. Nous avons ainsi été en mesure, toujours en suivant la méthodologie, d'en **découvrir des vulnérabilités**, des **fichiers hautement sensibles**, des **valeurs codées en dur**, des **backdoors** et des **comptes** mal protégés dont un ayant un **accès privilégié**.

Comme contre-mesures, nous proposons un **cryptage** plus fort du firmware. En effet cela rend quasi impossibles les attaques d'analyse et de rétro-ingénierie du firmware. Par ailleurs, il serait judicieux d'**éviter les identifiants et valeurs sensibles codées en dur** et en clair.

CONCLUSION GENERALE

Notre étude visait à élaborer une méthodologie de test d'intrusion en milieu IoT. Il ressort que nous avons été à même d'en concevoir une ayant la caractéristique d'aborder l'ensemble des surfaces d'attaques couramment rencontrées dans les milieux IoT. Elle adresse le concept de tests d'intrusion IoT en 4 phases, selon les surfaces d'attaques IoT.

Cette méthodologie qui résulte de notre travail est basée sur divers standards et méthodologies de la sphère IoT, notamment le FSTM (Firmware security Testing methodology), ou encore l'ISVS (IoT Security Verification Standard) de OWASP.

Par ailleurs, nous avons été à même de vérifier l'efficacité de ladite méthodologie au travers de tests pratiques, en nous focalisant sur les canaux d'attaques des couches réseau et firmware.

Nous jugeons pertinent d'inclure pour des travaux futurs, des rubriques adressant les contre-mesures des différentes attaques exploitées en vue de guider les testeurs dans l'élaboration d'un rapport de qualité. De même, toujours dans le cadre de travaux ultérieurs, nous envisageons la conception d'une plateforme d'automatisation de la tâche de test d'intrusion guidée par cette méthodologie.

BIBLIOGRAPHIE & WEBOGRAPHIE

[1] Top 5 Penetration Testing Methodologies and Standards visité le 30/06/2021 à 17h49

[2] OISSG, *Information Systems Security Assessment Framework (ISSAF) draft 0.2.1B*, publié le 01 May 2006. disponible ici

[3] Penetration Testing Execution Standard, visité le 08/03/2021 à 17h35

[4] ISTR 2019: Internet of Things Cyber Attacks Grow More Diverse *[consultée le 15/03/2021 à 11h38]*

[5] IoT devices more at risk of cyber attack than ever - report [consultée le 12/03/2021 à 11h02]

[6] The OWASP IoT Top 10 List of Vulnerabilities | InfoSec Insights [consultée le 15/03/2021 à 11h44]

[7] Pooja Anand, Yashwant Singh, Arvind Selwal, Pradeep Kumar Singh, Raluca Andreea Felseghi and Maria Simona Raboaca, "IoVT: Internet of Vulnerable Things? Threat Architecture, Attack Surfaces, and Vulnerabilities in the Internet of Things and Its Applications towards Smart Grids", *MDPI journals*, Septembre 2020

[8] Samer Moein, Fayez Gebali, Issa Traore, "Analysis of Covert Hardware Attacks", *Journal of Convergence*, Septembre 2014.

[9] SAMER MOEIN, FAYEZ GEBALI, "A Formal Methodology for Quantifying Overt Hardware Attacks", *Conference Paper* , Fevrier 2015

[10] Christoph Dobraunig, Maria Eichlseder, Thomas Korak, Stefan Mangard, Florian Mendel et Robert Primas SIFA: "Exploiting Ineffective Fault Inductions on Symmetric Cryptography", 2018. disponible ici

[11] Hagai Bar-El, "Known Attacks Against Smartcards", 2015 disponible ici

[12] Saul St. John, Thunderbolt: Exposure and Mitigation, publié en 2013 disponible ici

[13] Eclypsium, "Anatomy of a firmware attack", décembre 2019 disponible ici

[14] John heasman, "Hacking extensible firmware", *blackhat 2007, 2007*

[15] Breaking BLE — Vulnerabilities in pairing protocols leave Bluetooth devices open for attack [consultée le 30 /06/ à 18h27]

[16] Attacking bluetooth smart devices: introducing a new BLE proxy tool [consultée le 30 /06/ à 18h31]

[17] IoT-Security-Verification-Standard-ISVS/V5-Hardware_Platform_Requirements.md at master · OWASP/IoT-Security-Verification-Standard-ISVS [consultée le 15/03/2021 à 11h51]

[18] OWASP Firmware Security Testing Methodology - Firmware Security Testing Methodology [consultée le 15/03/2021 à 11h54]

[19] https://www.rapid7.com/blog/post/2017/05/10/iot-testing-methodology/ visité le 15/05/2021 à 11h51

[20] Muhammad Burhan, Rana Asif Rehman, Bilal Khan et Byung-Seo Kim, IOT Elements, Layered Architectures and Security Issues: A Comprehensive Survey, *MDPI journal* le 24 Août 2018

[21] en.wikipedia.org/wiki/Internet_of_things visité le 09/03/2021 à 16h35

[22] oracle.com/internet-of-things/what-is-iot/ visité le 09/03/2021 à 16h39

[23] internetofthingsagenda.techtarget.com/definition/Internet-of-Things-IoT visité le 09/03/2021 à 16h40

ANNEXES

Les annexes suivantes reprennent les tests de la checklist. **A** pour les tests physiques, **B** pour les tests réseau, **C** pour les tests firmware.

ANNEXES A1: Tests physiques, sous catégorie collecte d'informations

Tests	outils
Inspection externe	aucun
Inspection interne	de quoi démonter (tournevis de façon générale)
Recherches à travers internet	Moteur de recherche et connexion internet
Révision de la (des) fiche(s) technique(s)	aucun

ANNEXES A2: Tests physiques, sous catégorie Exploitation JTAG

Tests	outils
extraire le firmware via les ports JTAG	materiels physiques: Jtagulator/Jtagenum, Attify-badge/BusPirate, logiciels: OPENOCD, GDB-multiarch Attify-badge tools
Lecture de données sensibles en mémoire (clé/mot de passe pour authentification.) via debogage JTAG	materiels physiques: Jtagulator/Jtagenum, Attify-badge/BusPirate, logiciels: OPENOCD, GDB-multiarch Attify-badge tools
Ecriture d'un firmware non-à-jour ou modifié sur une/des puces (écraser le firmware originel)	materiels physiques: Jtagulator/Jtagenum, Attify-badge/BusPirate logiciels: OPENOCD, GDB-multiarch, Attify-badge tools

ANNEXES A3: Tests physiques, sous catégories Exploitation UART, SPI et I2C

Tests	outils
obtention de shell root sur l'équipement à travers une connexion UART et exploitation de ce shell	materiels physiques: Multimètre, Attify badge, Entête et Fils de raccordement. Logiciels: Le script Baudrate.py, Minicom ou screen
lire le contenu de la mémoire EEPROM et explorer le binaire obtenu (trouver des données sensibles...)	materiels physiques: pince SOIC, Attify Badge, fils de raccordement Logiciels: le script spiflash.py
écriture d'un nouveau contenu sur une/des puces EEPROM (contenu originel altéré)	
lire le contenu de la mémoire EEPROM et explorer le binaire obtenu (trouver des données sensibles...)	materiels physiques: pince SOIC, Attify Badge, fils de raccordement Logiciels: le script i2ceeprom.py

ANNEXES A4: Tests physiques, sous catégorie tests spécifiques à des équipements

Tests	outils
Débloquer un Routeur 3G/4G (le rendre utilisable avec d'autres operateurs)	Logiciels: Putty, Multicast Upgrade Tool, Huawei ES186 toolbox, Huawei Unlock Calculator
Débloquer un modem 3G/4G (le rendre utilisable avec d'autres operateurs)	Matériels physiques: câble quelconque servant à court-circuiter, câble USB-to-USB, tournevis pour le démontage.
	Logiciels: USBloader, balong usb downloader, 2 versions du firmware respectivement récente et obsolète, DC-unlocker, Huawei WebUI, pilotes du modem (Huawei Hilink, Huawei Mobile Partner)

ANNEXES B1: Tests Réseaux, sous catégorie collecte d'informations

Tests	outils
Découverte et mappage	Logiciels: Vistumbler, Nmap , KillerBee, Hcitool Matériels Physiques: Clé USB Atmel RzRaven
Identification des ports et services Actifs	Nmap
Scan de vulnérabilités	Logiciel: OpenVas ou Nmap

ANNEXES B2: Tests Réseaux, sous catégories Attaques Bluetooth/BLE et Zigbee

Tests	outils
Sniffer une communication BLE à la recherche de données sensibles et emmettre des requêtes malicieuses	Logiciels: Wireshark, Hcitool, Ubertooth, Gatttool
Attaque replay sur un réseau BLE	Logiciels: BTLEJuice

Sniffer une communication à la recherche de données sensibles	Matériels Physiques: Atmel RzRaven USB Stick Logiciels: KillerBee
Attaque replay	Logiciels: Attify KillerBee Framework

ANNEXES B3: Tests Réseaux, sous catégorie Attaques WI-FI

Tests	outils
Sniffer une communication WIFI non cryptée à la recherche de données sensibles (mot de passe...)	Logiciels: Wireshark
Cracking de mot de passe via attaque bruteforce	Matériels Physiques: Adaptateur Wifi Logiciels: Wifite, Hashcat
Cracking de mot de passe via attaque par dictionnaire	Matériels Physiques: Adaptateur Wifi Logiciels: la suite Aircrack-ng(Airmon-ng, Airodump-ng, Aireplay-ng, Aircrack-ng), Wireshark
DoS sur DHCP (DHCP Starvation attack) (réseau avec un(e) box internet/point d'accès)	Logiciel: Yersinia
Attaque Serveur rogue DHCP (réseau avec un(e) box internet/point d'accès)	Logiciel: Yersinia
Attaque Man-In-The-Middle (réseau avec un(e) box internet/point d'accès)	Logiciel: Yersinia, Wireshark
DNS Spoofing/Cache Poisonning	Logiciel: Cain&Abel

ANNEXES B4: Tests Réseaux, sous catégorie Attaques VOIP

Tests	outils
Reconnaissance	Logiciels: Svnmap, Svmap
Interception des flux VOIP/ relecture des conversations	Voir Sniffing du trafic WIFI
bruteforce de mot de passe de clients VOIP (SIP)	Matériels: Adaptateur wifi Logiciels: SIPcrack et SIPdump (qui est inclu dans la suite SIPcrack)
Flooding sur VOIP (SIP/SDP)	Logiciel: Inviteflood

ANNEXES C1: Tests Réseaux, sous catégorie Collecte d'informations et Analyse

Tests	outils
Collecte d'informations	Internet, Fiche technique
Obtention/Extraction du Firmware	Dépend de la méthode utilisée (si l'extraction se fait via JTAG, SPI, ou I²C voir les tests respectifs de la couche physique. Sinon télécharger le firmware s'il est disponible sur internet...)
Analyse du firmware	Logiciels: Binwalk
Extraction du Système de Fichier	Logiciel: Binwalk
Analyse du système de fichier (à la recherche de vulnérabilités, de fichiers *.conf* vulnérables, de données sensibles, de Backdoors de maintenance...)	Logiciel: Firmwalker
Rétro-ingénierie binaire	Logiciel: Hooper

ANNEXES C2: Tests Réseaux, sous catégories Implantation de backdoor & Emulation, Analyse dynamique et Exploitation

Tests	outils
Backdooring	Logiciels: FMK, Buildroot
Emulation etAnalyse dynamique	Logiciel: FAT (Firmware Analysis Toolkit)
Exploitation et/ou flash du firmware altéré	Voir comment écrire un nouveau firmware en mémoire ici (JTAG) et ici (SPI)

ANNEXE D: Code du serveur Web faisant appel au module GPIO.

```
<title>INTERFACE DE CONTROL DE IOT</title>
</head>
	<body>
	<center><b><font size = '20'>INTERFACE DE CONTROLE DE LA DEL</b>
	  <form method="get" action="index.php">
		  <input type="submit" style = "font-size: 16 pt" value="eteindre" name="off">
		  <input type="submit" style = "font-size: 16 pt" value="allumer" name="on">
		  <input type="submit" style = "font-size: 16 pt" value="faire clignoter" name="blink">
	  </form>
	  <?php
	  system("gpio export 17 out");
	  if(isset($_GET['off']))
	  {
		  echo "La DEL est eteinte";
		  system("gpio -g write 17 0");
	  }
	 else if(isset($_GET['on']))
	 {
		  echo "La DEL est alumee";
		  system("gpio -g write 17 1");
	 }
	 else if(isset($_GET['blink']))
	 {
		  echo "La DEL clignote";
			  for($x = 0;$x<=4;$x++)
		  {
			  system("gpio -g write 17 1");
			  sleep(1);
			  system("gpio -g write 17 0");
			  sleep(1);
		  }
	 }
	 ?>
```